

Scoping Literature Review

Accelerating Tensor Network Contraction for Quantum Circuit Simulation using Processing-in-Memory Architectures

Contents

1. Introduction	3
1.1. Motivation and target gap	3
1.2. Review scope	4
2. Research Questions	4
3. Methodology	4
3.1. Review design	4
3.2. PICOC framework	5
3.3. Tiered evidence-mapping strategy	5
3.4. Inclusion criteria	6
3.5. Exclusion criteria	6
3.6. Tiered eligibility logic	7
3.7. Screening fallback protocol	7
4. Search Strategy and Record Identification	8
4.1. Search execution	8
4.2. Search log	8
4.3. Search completeness and fast-moving literature limitations	9
5. Screening and Eligibility	9
5.1. Title and abstract screening	9
5.2. Full-text eligibility assessment	9
5.3. PRISMA-ScR flow	11
5.4. Evidence map	12
6. Quality Appraisal	12
7. Data Extraction	13
8. Evidence Synthesis	14
8.1. Answers to the research questions	14
8.2. Cross-tier synthesis	17
9. Design Implications	18
9.1. Literature-derived implementation implications	18
9.2. Representative UPMEM constraints for tensor-network contraction	19
9.3. Cost-model implications	20
9.4. Implications for future prototypes	20
10. Limitations	21
11. Conclusion	22
Bibliography	22
12. Appendix A – PICOC, Search Strings, and Search Validation	36
12.1. Purpose of the appendix	36
12.2. PICOC keyword groups	36
12.3. From PICOC to tiered searches	37
12.4. Database-level limits and search log	38
12.5. Search-string validation during revision	39
12.6. Exact search strings	40
13. Appendix B – Screening Log	42
13.1. Purpose of the screening log	42

13.2. Screening summary by tier	42
13.3. Records advanced to full-text assessment	42
13.4. Records excluded during screening	48
14. Appendix C – Full-Text Review	64
14.1. Purpose of the full-text review	64
14.2. Full-text eligibility summary	64
14.3. Studies included after full-text review	64
14.4. Studies excluded after full-text review	68
15. Appendix D – Quality Appraisal	68
15.1. Purpose of the quality appraisal appendix	68
15.2. Quality appraisal rubric	69
15.3. Quality appraisal cards	69
16. Appendix E – Data Extraction Cards	83
16.1. Purpose of the data extraction appendix	83
16.2. Extraction schema	83
16.3. Extraction cards	84

1. Introduction

1.1. Motivation and target gap

Quantum computing has been a rapidly evolving field since Feynman’s initial proposal in 1982 [1]. Although current devices remain in the Noisy Intermediate-Scale Quantum (NISQ) era [2], theoretical algorithms demonstrate computational advantages that are not expected to be achievable by classical systems in general. Until fault-tolerant quantum computers are realized at scale, classical simulation of quantum circuits remains an essential part of the quantum-computing stack. Simulators are used for hardware verification, algorithmic validation, benchmarking, and the development of hybrid quantum-classical workflows [3].

A major milestone occurred in 2019, when Google reported quantum supremacy using the Sycamore processor, estimating that a classical supercomputer would require 10,000 years to reproduce a calculation performed by the quantum device in 600 seconds [4]. Subsequent work showed, however, that the classical baseline can move rapidly. Improved algorithms and implementations substantially reduced the claimed gap. For example, later tensor-network-based simulations demonstrated that the classical cost of reproducing Sycamore-style circuits can be reduced dramatically when improved algorithms, numerical representations, and hardware-aware implementations are used [5]. This history shows that quantum advantage is not evaluated against a fixed classical reference point. It is continually redefined by improvements in algorithms, representations, and hardware-aware simulation methods.

Much of this classical progress relies on tensor-network (TN) contraction. As Cicero et al. [3] discuss, traditional state-vector simulation stores 2^n complex amplitudes and therefore quickly exhausts available memory. Tensor networks—such as Matrix Product States (MPS) or Projected Entangled Pair States (PEPS)—avoid this direct capacity limit by representing a quantum state or circuit as a graph of local tensors. However, exact simulation still requires contracting the network. The bottleneck is then shifted toward contraction order, intermediate tensor growth, and data movement. Tensor-network contractions are often expressed as Einstein-summation tensor expressions involving high-order tensors and large index dimensions; these expressions are commonly lowered to dense matrix multiplications or related tensor kernels., and modern kernels may become limited less by arithmetic throughput than by the cost of moving large arrays between memory and processing units: the so-called “memory wall” [6].

Processing-in-Memory (PIM) [7] is an architectural paradigm designed to mitigate the memory wall by placing computation near memory banks. Commodity PIM architectures, such as the digital DRAM-based platform provided by UPMEM [8], have shown potential for memory-bound workloads. This review focuses on **digital** PIM, with emphasis on UPMEM-style architectures, to distinguish the target hardware model from analog in-memory computing approaches such as ReRAM crossbars.

At present, the direct literature connecting tensor-network quantum circuit simulation and digital PIM is extremely sparse. The executed database search did not identify any study that directly implements tensor-network quantum circuit simulation on UPMEM-style digital PIM. The closest adjacent work is **PIMutation** by Lee et al. [9], which uses real UPMEM devices for quantum circuit simulation, but relies on a full state-vector representation rather than a tensor-network representation. This approach encounters a memory limit in the low-30-qubit range, and its partitioning strategy is not robust for highly entangled circuits. This motivates the central question of the review: whether quantum circuit simulation via tensor-network contractions can be mapped onto memory-constrained digital PIM devices.

1.2. Review scope

Quantum circuit simulation, tensor-network methodology, and Processing-in-Memory are mature independent domains, but their intersection remains sparse. In particular, no direct study of tensor-network quantum circuit simulation on UPMEM-style digital PIM was identified by the executed database search and eligibility process.

Because the direct target domain is largely empty, a systematic effectiveness review is not possible. Instead, this work is structured as a Scoping Literature Review using a proxy strategy and reported according to PRISMA-ScR (Preferred Reporting Items for Systematic reviews and Meta-Analyses extension for Scoping Reviews). The review maps three adjacent evidence domains: quantum circuit simulation on PIM; mathematical and systems frameworks for tensor-network contraction on traditional memory hierarchies; and UPMEM proxy workloads that expose memory-capacity, data-movement, and communication constraints relevant to tensor-network execution. By charting these proxy domains, the review aims to identify the evidence required to adapt quantum tensor networks for PIM environments and to clarify which design questions remain untested.

2. Research Questions

The review is organized around four research questions, which correspond to the main evidence domains examined in the study.

Research Questions (RQ)

- RQ1** What are the primary computational and memory bottlenecks in tensor-network contractions for quantum circuits?
- RQ2** How do existing contraction heuristics and execution models perform on traditional CPU/GPU memory hierarchies?
- RQ3** How have quantum circuit simulators been adapted for Processing-in-Memory (PIM) architectures?
- RQ4** What performance insights can Processing-in-Memory workloads provide for tensor-network simulations?

3. Methodology

3.1. Review design

This work is structured as a Scoping Literature Review rather than a conventional systematic literature review, because the executed search did not identify enough direct studies to support a comparative synthesis of tensor-network quantum circuit simulation on digital PIM. The review therefore follows a proxy-based evidence-mapping strategy and is reported according to PRISMA-ScR (Preferred Reporting Items for Systematic reviews and Meta-Analyses extension for Scoping Reviews).

The review protocol was organized using the PICOC framework: Population, Intervention, Comparison, Outcome, and Context. The formal search was conducted across three academic databases: IEEE Xplore, the ACM Digital Library, and Scopus. These databases were selected because they cover the main domains required by this review: quantum simulation, tensor-network algorithms, high-performance computing, computer architecture, and Processing-in-Memory systems.

3.2. PICOC framework

Population Quantum circuit simulation workloads, with primary focus on tensor-network representations and adjacent state-vector evidence used only for comparison where relevant.

Intervention Digital Processing-in-Memory architectures and algorithms adapted for PIM constraints, including slicing, bond cutting, contraction-path optimization, quantization, numerical representation, and data-layout adaptation.

Comparison Conventional CPU/GPU and distributed-memory implementations, together with adjacent non-tensor-network quantum-simulation approaches on digital PIM.

Outcome Performance, bandwidth utilization, energy efficiency, memory feasibility, intermediate tensor size, numerical accuracy, and data-movement behavior.

Context Digital Processing-in-Memory hardware, with emphasis on UPMEM-style architectures and their execution constraints.

3.3. Tiered evidence-mapping strategy

A strict search for the direct target intersection was first used to test whether the exact research area was represented in the selected databases and search strings. This strict target was defined as tensor-network quantum circuit simulation combined with digital Processing-in-Memory. The strict target search did not identify any direct studies, which justified the use of a tiered proxy strategy.

Because the strict target intersection was not represented in the retrieved literature, the review used a tiered evidence-mapping strategy. Tier 0 served as a baseline gap test for direct studies on tensor-network quantum circuit simulation using digital PIM. Tiers 1–3 then provided proxy evidence by relaxing one part of the strict target at a time: Tier 1 retained quantum simulation and PIM but allowed non-tensor-network representations; Tier 2 retained tensor-network contraction but allowed conventional CPU/GPU or distributed-memory hardware; and Tier 3 retained digital PIM hardware but allowed non-quantum memory-bound proxy workloads.

Tier 0 *Strict target intersection* — Tensor-network quantum circuit simulation on digital Processing-in-Memory. This tier was used as a baseline test of the direct research gap in the executed search and did not identify any direct studies.

Tier 1 *Quantum simulation on PIM* — Quantum circuit simulation combined with Processing-in-Memory, regardless of whether the simulator uses a state-vector or tensor-network representation. This tier captures direct quantum-simulation-on-PIM evidence, but not necessarily direct tensor-network evidence.

Tier 2 *Tensor-network algorithmic analogs* — Tensor-network contraction, slicing, contraction-path optimization, and related tensor-algebra methods developed for CPU, GPU, or distributed-memory systems. This tier captures algorithmic evidence relevant to contraction planning, but not direct PIM evidence.

Tier 3 *UPMEM hardware proxy workloads* — Digital PIM and UPMEM studies on memory-bound workloads, such as SpMV, GEMM, graph analytics, sparse tensor operations, and host-DPU communication. This tier captures hardware-constraint evidence only; it is not treated as evidence about quantum simulation performance.

Tier 0 establishes that no direct target study was identified by the executed search. Tiers 1–3 provide the proxy evidence used in the remainder of the review.

3.4. Inclusion criteria

The inclusion criteria define the topical, architectural, and methodological boundaries used to select studies relevant to the review questions.

The final database searches were executed in November 2025. Therefore, studies published after November 18, 2025 are outside the scope of this review.

- IC1** *Topical relevance* — The study addresses at least one of the review’s target computational domains: tensor-network contraction, quantum circuit simulation, state-vector simulation, tensor algebra, or memory-bound multidimensional array computation.
- IC2** *Target architecture* — The study uses, simulates, models, or directly informs digital Processing-in-Memory architectures, with emphasis on UPMEM, or analyzes distributed-memory systems with explicit data-movement constraints.
- IC3** *Algorithmic focus* — Exclude studies that use tensor networks primarily to analyze physical phenomena, rather than to study computational implementation, contraction cost, memory complexity, scheduling, or data movement. Examples include papers focused only on condensed-matter models, phase transitions, or ground-state properties.
- IC4** *Publication date* — The study was published between January 1, 2016 and November 18, 2025.
- IC5** *Source quality* — The study is a peer-reviewed conference paper, journal article, or reputable preprint in a discipline directly relevant to the review’s computational, quantum-simulation, or computer-architecture scope.

3.5. Exclusion criteria

The exclusion criteria were used to remove studies that were structurally incompatible with the review scope or insufficiently relevant to the target hardware and computational model.

- EC1** *Analog or non-digital in-memory hardware* — Exclude studies whose primary contribution depends on analog, mixed-signal, ReRAM, memristor, phase-change-memory, or crossbar-based computation, unless they provide hardware-independent insights into data movement, tiling, numerical representation, or workload mapping that are transferable to digital PIM architectures such as UPMEM.
- EC2** *Irrelevant workload* — Exclude studies focused exclusively on application-specific workloads, such as deep-neural-network (DNN) inference or training, transformer acceleration, or relational databases, unless they provide transferable evidence about dense or sparse tensor/matrix computation, graph-like memory access, numerical representation, data layout, or data movement relevant to tensor-network simulation.
- EC3** *Pure physics theory* — Exclude studies that use tensor networks primarily to analyze physical phenomena, without providing transferable evidence about computational implementation, contraction cost, memory complexity, scheduling, or data movement.
- EC4** *Shared-memory dependence* — Exclude studies whose main method depends on tightly coupled shared-memory execution, such as cache-coherent shared memory, shared virtual memory, atomic updates, or fine-grained worker-to-worker communication, in a way that cannot be transferred to isolated UPMEM DPUs with host-mediated communication.
- EC5** *Quality or format limitation* — Exclude records that are non-English, abstract-only, poster-only, non-technical, or lack sufficient methodological detail for screening, eligibility assessment, or data extraction.

3.6. Tiered eligibility logic

Because the strict target search did not identify direct studies, the review applies the inclusion criteria differently across the three formal evidence tiers introduced in Section 3.3.

Tier 1 *Quantum simulation on PIM* — Studies must satisfy IC1, IC2, IC4, and IC5. IC3 is desirable but not mandatory, because this tier includes both tensor-network and state-vector quantum simulators on PIM. State-vector PIM studies are used as adjacent quantum-simulation evidence rather than as direct tensor-network evidence.

Tier 2 *Tensor-network algorithmic analogs* — Studies must satisfy IC1, IC3, IC4, and IC5. IC2 is not required, because this tier captures tensor-network contraction, slicing, and scheduling methods developed for CPU, GPU, or distributed-memory systems. These studies are used as algorithmic analogs rather than as evidence about PIM performance.

Tier 3 *UPMEM hardware proxy workloads* — Studies must satisfy IC2, IC4, and IC5. IC1 and IC3 are not required, because this tier is intended to capture evidence about digital-PIM execution constraints rather than evidence about quantum simulation or tensor-network algorithms directly. Eligible studies may therefore use non-quantum proxy workloads, provided that they expose constraints relevant to a future UPMEM-based tensor-network implementation, such as host-DPU transfer, MRAM-WRAM movement, WRAM-aware tiling, dense or sparse matrix/tensor kernels, graph-like memory access, load balancing across DPUs, or host-mediated communication. These studies are used only as hardware-constraint evidence, not as direct evidence that PIM accelerates quantum circuit simulation.

For Tier 3, a proxy workload was retained only when it provided substantive evidence about one or more transferable digital-PIM constraints relevant to tensor-network execution. These constraints included host-DPU transfer, MRAM-WRAM movement, WRAM-limited tiling, integer-oriented execution or floating-point emulation costs, numerical representation, load balancing across DPUs, and host-mediated communication. Tier 3 studies were not treated as direct evidence about quantum simulation; they were used only to identify hardware constraints that a future tensor-network implementation would need to respect.

Application-level AI, LLM, or accelerator studies were excluded unless they provided transferable evidence about general tensor kernels, numerical representation, host-DPU transfer, WRAM-aware tiling, sparse or graph-like memory access, or other digital-PIM constraints relevant to tensor-network execution. This rule distinguishes application-specific acceleration claims from transferable hardware or kernel-level evidence.

The exclusion criteria were applied according to a transferability rule. Each tier admits a different type of proxy evidence, but a study was retained only when its main contribution transferred to that tier’s purpose. Tier 2 retained CPU/GPU or distributed-memory studies only when they provided transferable algorithmic evidence for tensor-network contraction, such as slicing, contraction ordering, scheduling, partitioning, memory use, or data movement. Tier 3 retained non-quantum UPMEM proxy workloads only when they exposed transferable digital-PIM constraints, such as host-DPU transfer, MRAM-WRAM movement, WRAM-aware tiling, numerical representation, load balancing, or host-mediated communication. Studies were excluded when their contribution was too application-specific or too dependent on tightly coupled shared-memory execution to inform a future UPMEM-based tensor-network implementation.

3.7. Screening fallback protocol

During screening, several records used ambiguous terminology for Processing-in-Memory, near-memory processing, tensor operations, or accelerator architecture. When title and abstract

information was insufficient to determine eligibility, the methodology and experimental setup sections of the full text were consulted. This fallback procedure was used to distinguish digital PIM from analog or ReRAM-based in-memory computing, and to assess whether a non-quantum workload provided transferable evidence for tensor-network simulation.

4. Search Strategy and Record Identification

4.1. Search execution

The formal database searches were conducted in November 2025 in IEEE Xplore, the ACM Digital Library, and Scopus. The search strategy followed the tiered evidence-mapping logic defined in Section 3.3. A strict baseline search was first used to test the direct target intersection of tensor-network quantum circuit simulation and digital Processing-in-Memory. Because the strict target search returned no direct studies in the selected databases, the review used a proxy-based search strategy to map adjacent evidence domains relevant to the research questions.

Because the strict target search returned no direct studies in the selected databases, the review used a proxy-based search strategy to map adjacent evidence domains relevant to the research questions. The subsequent database searches operationalized the three proxy evidence tiers. Tier 1 queries combined quantum-simulation terms with digital Processing-in-Memory terms. Tier 2 queries combined tensor-network and tensor-contraction terms with terms related to contraction planning and parallel or distributed execution, including slicing, contraction-path optimization, graph or hypergraph partitioning, memory-constrained scheduling, distributed-memory execution, load balancing, and communication cost. Tier 3 queries combined UPMEM or digital-PIM terms with data-movement-sensitive proxy workloads, including dense and sparse matrix/tensor kernels, SpMV, GEMM, graph analytics, quantization, numerical representation, and host-DPU communication. The full database-specific search strings are provided in Section 12.

4.2. Search log

The tiered searches identified 316 raw records. Database-level filters were then applied where supported by the database interface, including publication date, language, and publication type. After these filters, 186 records remained for deduplication and screening. The distribution of records across databases and evidence tiers is shown in Table 1, while the exact search strings and database-level limits are reported in Section 12.4.

Database	Tier	Raw hits	After limits
IEEE Xplore	1 Quantum simulation on PIM	0	0
IEEE Xplore	2 Algorithmic analogs	1	1
IEEE Xplore	3 Hardware proxies	31	31
ACM DL	1 Quantum simulation on PIM	52	6
ACM DL	2 Algorithmic analogs	77	27
ACM DL	3 Hardware proxies	131	98
Scopus	1 Quantum simulation on PIM	1	1
Scopus	2 Algorithmic analogs	0	0
Scopus	3 Hardware proxies	23	22
Total		316	186

Table 1: Search log by database and evidence tier.

The tier labels in Table 1 describe the search strata used to retrieve records; final inclusion by tier is reported after screening and full-text assessment in Section 5.

4.3. Search completeness and fast-moving literature limitations

The formal database search was limited to IEEE Xplore, the ACM Digital Library, and Scopus. These databases were selected because they provide indexed coverage of computer architecture, high-performance computing, tensor computation, and quantum-computing venues relevant to the review scope. The formal search did not include arXiv, Google Scholar, dblp, GitHub repositories, or systematic backward and forward citation chasing.

This choice improves reproducibility because the executed database searches can be reported as explicit Boolean strings with database-level limits. However, it also limits completeness. Quantum simulation, tensor-network methods, compiler systems, and Processing-in-Memory architectures are fast-moving areas, and relevant work may appear first as preprints, software artifacts, workshop papers, or repository-based implementations before being indexed in the selected databases.

Therefore, the absence of direct studies should be interpreted narrowly. This review claims that no direct study of tensor-network quantum circuit simulation on UPMEM-style digital PIM was identified by the executed database search and eligibility process. It does not claim that no such work exists anywhere. The review should be understood as a structured baseline for the thesis project. Later implementation work may add targeted related work through backward citation chasing, forward citation chasing, arXiv checks, or artifact searches where needed, but such additions would be reported separately from the formal SLR corpus.

5. Screening and Eligibility

5.1. Title and abstract screening

Following the execution of the search strategy and duplicate removal, 161 unique records advanced to title and abstract screening. During this phase, records were evaluated against the inclusion and exclusion criteria defined in Section 3.4 and Section 3.5.

Several records, especially in the Processing-in-Memory and computer-architecture literature, used ambiguous terminology for hardware model, workload type, or execution constraints. In particular, some abstracts did not clearly distinguish digital DRAM-based PIM from analog or ReRAM-based in-memory computing, while others described tensor operations or accelerator workloads without enough detail to determine relevance.

To reduce false positives before the full eligibility phase, the fallback protocol described in Section 3.7 was applied. When title and abstract information was insufficient, the methodology or experimental setup sections of the full text were briefly consulted. This procedure was used only to determine screening eligibility, not to perform full data extraction.

After title and abstract screening, 124 records were excluded. The remaining 37 candidate studies advanced to full-text eligibility assessment.

5.2. Full-text eligibility assessment

The 37 candidate studies were assessed through full-text review. During this phase, studies were evaluated for whether they provided actionable algorithmic, architectural, or empirical evidence relevant to the review questions.

Four studies were excluded during full-text assessment. In Tier 1, one candidate was excluded because it proposed a custom silicon architecture rather than using or evaluating commodity digital PIM hardware for quantum simulation. In Tier 2, one study was excluded because it focused on circuit-design-oriented hypergraph partitioning rather than tensor-network contraction, tensor-algebra execution, or transferable memory/communication-aware scheduling methods, and another

was excluded because it was a broad survey without sufficient experimental or methodological detail for data extraction. In Tier 3, one early benchmarking record was excluded because it did not provide sufficient experimental setup detail and was superseded by a later, more complete publication.

The Tier 1 full-text assessment is important for interpreting RQ3. Two Tier 1 candidate records advanced to full-text review, but only PIMutation was included in the final corpus because it directly evaluates quantum circuit simulation on commodity UPMEM hardware. Therefore, the final Tier 1 evidence base consists of one included study.

After full-text eligibility assessment, the final included corpus contained 33 studies: 1 from Tier 1, 10 from Tier 2, and 22 from Tier 3.

5.3. PRISMA-ScR flow

The study selection process is summarized in Figure 1. The search identified 316 raw records, which were reduced to 186 records after database-level limits and 161 unique records after deduplication. Title and abstract screening excluded 124 records. Full-text assessment was conducted on 37 candidate studies, of which 4 were excluded. The final corpus contained 33 included studies: 1 from Tier 1, 10 from Tier 2, and 22 from Tier 3.

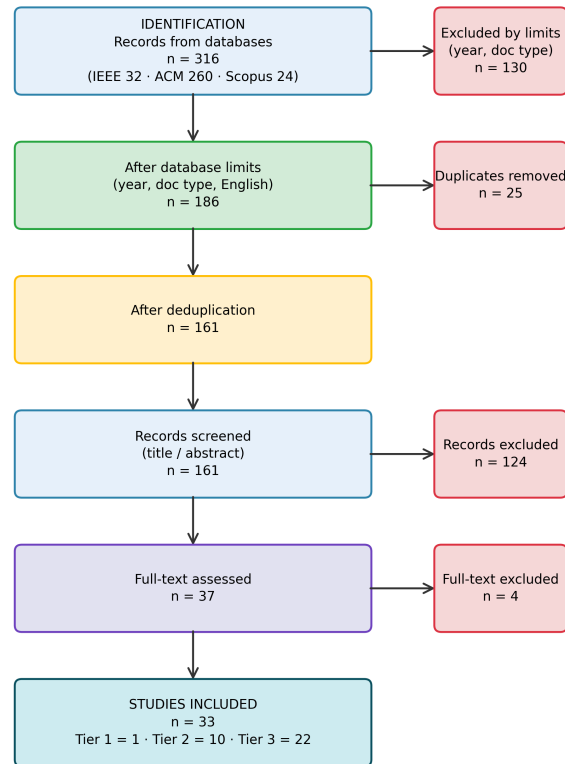


Figure 1: PRISMA-ScR flow diagram summarizing the study selection process.

5.4. Evidence map

To inspect the thematic coherence of the included corpus, a citation and topic map was generated for the included studies. The map provides a visual overview of relationships between the algorithmic tensor-network literature, quantum-simulation studies, and UPMEM hardware proxy papers.

Figure 2 shows no visible interconnection between the evidence tiers, except for one link between the Tier 1 study, PIMutation [9], and the Gómez-Luna Benchmarking paper [6]. In particular, PIMutation is not visibly connected to the Tier 2 tensor-network contraction literature. This supports the review’s proxy-based synthesis: the review connects adjacent literatures conceptually, not because they already form a connected citation/topic network.

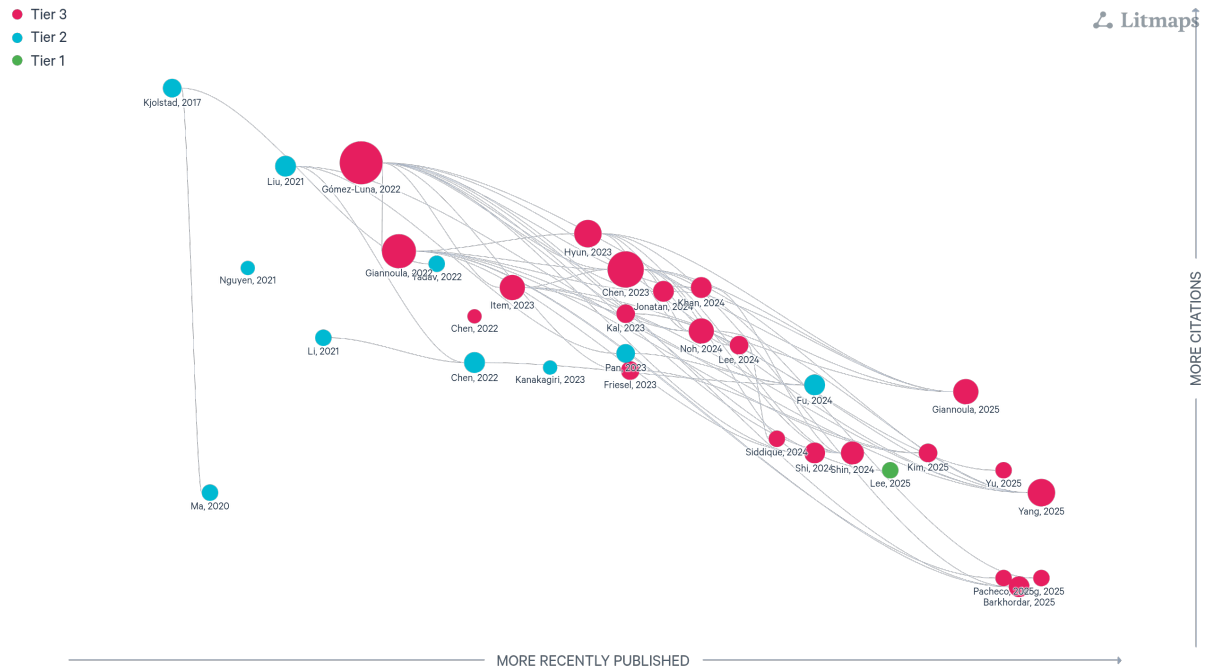


Figure 2: Evidence map of the included studies.

6. Quality Appraisal

Quality appraisal of the included literature was conducted using an eight-criterion rubric evaluated on a 0–2 scale. The rubric was designed as a review-specific quality, transparency, and transferability assessment rather than as a general ranking of paper quality. It evaluates how clearly and reproducibly each study supports the review questions, and how transferable its evidence is to future PIM-aware tensor-network simulation.

Each criterion was scored using the same three-point scale: 0 when the criterion was absent or not assessable, 1 when it was partially addressed, and 2 when it was clearly addressed with sufficient detail. The eight assessed dimensions were: clarity of research aims, methodological reproducibility, presence of CPU/GPU or otherwise relevant baseline comparisons, explicit treatment of data movement, hardware or execution-model realism, availability of open-source artifacts, discussion of limitations, and depth of contribution to the review questions.

Each included study received a maximum possible score of 16 points. Studies scoring between 14 and 16 points were classified as high quality. Studies scoring between 10 and 13 points were classified as medium quality. Studies scoring below 10 points were classified as low quality.

The quality appraisal summary is shown in Table 2. The appraisal should be interpreted as a structured transparency and reproducibility tool rather than as an absolute ranking of paper quality. This is important because the included corpus combines different types of evidence, including quantum simulation, tensor-network algorithms, compiler frameworks, UPMEM benchmarking, dense and sparse matrix/tensor workloads, graph workloads, and communication frameworks. Consequently, criteria such as hardware realism, baseline quality, and contribution depth are not identical across all evidence tiers.

The final appraisal classified 14 studies as high quality, 16 as medium quality, and 3 as low quality. The only Tier 1 study, PIMutation, was classified as medium quality under the 14–16 high-quality threshold. All 33 studies remained in the final corpus and were included in data extraction. The quality appraisal was used descriptively rather than as an exclusion criterion. In practice, studies with clearer methodology, stronger baseline comparisons, explicit data-movement analysis, and available artifacts or source code provided more actionable evidence for the review. Lower-scoring studies were still retained when relevant, but their findings were interpreted more cautiously when drawing implementation-specific conclusions. The complete per-paper scoring records and evidence notes are provided in Section 15.

Literature tier	Total studies	High (14–16)	Medium (10–13)	Low (< 10)
Tier 1: Quantum simulation on PIM	1	0	1	0
Tier 2: Tensor-network algorithmic analogs	10	2	8	0
Tier 3: UPMEM hardware proxy workloads	22	12	7	3
Overall corpus	33	14	16	3

Table 2: Summary of quality appraisal scores across the included studies.

7. Data Extraction

Following quality appraisal, a standardized data extraction process was used to collect the information required to answer the review questions. For each included study, an extraction card was created using a common schema. The goal was not only to summarize each paper, but also to identify algorithmic, architectural, and empirical evidence that could transfer to tensor-network quantum circuit simulation on digital PIM.

The extraction schema included the following categories:

Study type Identifies how the study functions within the review, such as compiler framework, optimization study, systems paper, benchmark, characterization study, architecture proposal, or communication framework. Publication format is handled separately through the source-quality criterion.

Computational domain Quantum simulation, tensor-network contraction, tensor algebra, dense and sparse matrix computation, graph processing, host-DPU communication, or another memory-bound proxy workload.

Aim and RQ link Main objective of the study and the review question or questions it primarily informs.

Method Algorithmic, compiler, architectural, or systems technique proposed or evaluated by the study.

Platform Target architecture or evaluation environment, such as CPU, GPU, distributed-memory system, UPMEM, digital PIM simulator, or another PIM-like platform.

Constraints Explicit memory, communication, arithmetic, and data-movement constraints, including memory capacity, intermediate tensor size, host-DPU transfer, MRAM-WRAM movement, lack of native floating-point support, and inter-DPU communication.

Evaluation evidence Reported baselines, datasets or benchmark circuits, workloads, performance metrics, bandwidth measurements, energy results, and scalability limits.

Reported results Main performance, energy, scalability, correctness, or accuracy findings relevant to the review.

Reported limitations Limitations, bottlenecks, threats to validity, scope restrictions, and reproducibility information reported by the study, including source-code availability, benchmark suites, simulator artifacts, datasets, or absence of released code where relevant.

Review relevance Main contribution to the review questions and implications for PIM-aware tensor-network simulation.

When a final peer-reviewed publication omitted implementation details that were available in an earlier preprint, artifact repository, benchmark suite, or project documentation, these sources were cross-referenced where relevant. The source of extracted information was recorded in the corresponding extraction card. The full extraction records are provided in Section 16.

8. Evidence Synthesis

The synthesis of the 33 included studies combines evidence from quantum simulation, tensor-network algorithms, and digital Processing-in-Memory proxy workloads. Because no included study directly maps tensor-network quantum circuit simulation onto UPMEM-style PIM, the synthesis is organized around the four research questions and then integrated across the three evidence tiers.

The final included corpus contains 1 Tier 1 study, 10 Tier 2 studies, and 22 Tier 3 studies. Tier 1 should therefore be interpreted carefully: although the Tier 1 search produced several candidate records, only PIMutation remained after screening and full-text eligibility assessment. Tier 1 consequently provides direct but case-based evidence for quantum simulation on digital PIM, not direct evidence for tensor-network simulation on PIM. Tier 2 provides algorithmic evidence for tensor-network contraction, slicing, contraction-path optimization, and memory-aware execution on conventional architectures. Tier 3 provides hardware-constraint evidence about UPMEM-style execution, including host-DPU transfer, MRAM-WRAM movement, limited WRAM capacity, lack of native floating-point support, and host-mediated communication.

8.1. Answers to the research questions

RQ1 *What are the primary computational and memory bottlenecks in tensor-network contractions for quantum circuits?*

The Tier 2 tensor-network and tensor-algebra studies consistently point to four recurring bottleneck categories: intermediate tensor growth, contraction-order sensitivity, slicing overhead, and data movement [10], [11], [12], [13]. Tensor-network simulation avoids storing the full 2^n state vector directly, but exact contraction can still create large intermediate tensors. The size and lifetime of these intermediates depend on circuit structure, entanglement pattern, contraction order, slicing strategy, and data layout.

The Tier 2 studies show that tensor-network simulation is not governed only by arithmetic complexity. In large-scale quantum circuit simulators, memory pressure is reduced by choosing contraction paths that avoid large intermediates, slicing selected indices into independent subtasks, tracking the lifetime of intermediate tensors or tensor edges, and distributing the resulting work across many processors [5], [10], [11], [14]. Compiler and tensor-algebra studies support the same conclusion from a more general tensor-computation perspective: loop ordering, tensor formats, and contraction scheduling can strongly affect memory traffic and intermediate storage, even when the mathematical tensor expression is unchanged [13], [15], [16].

For quantum circuit simulation, many tensor contractions are eventually lowered to dense or structured linear-algebra kernels such as GEMM or Einsum-like operations. On conventional CPU/GPU systems, these kernels can be highly optimized, but performance may still be limited when intermediate tensors exceed cache or device-memory capacity [12]. This makes tensor-network contraction a memory- and communication-sensitive workload, rather than a purely compute-bound workload.

RQ2 *How do existing contraction heuristics and execution models perform on traditional CPU/GPU memory hierarchies?*

Existing contraction heuristics and execution models have shown strong results when their cost assumptions match the target architecture. AutoHOOT, TACO, DISTAL, and related tensor-algebra systems provide useful foundations for contraction planning, tensor expression optimization, loop generation, and distributed tensor execution [13], [16], [17]. Quantum circuit simulators on Sunway and GPU systems further show that slicing, contraction-path refinement, mixed precision, and hierarchical parallelism can reduce memory pressure and improve throughput on conventional high-performance systems [5], [10], [11], [12].

However, these methods are designed around conventional hardware assumptions. They usually assume hardware-native floating-point arithmetic, mature dense-linear-algebra libraries, cache hierarchies or high-bandwidth GPU memory, and relatively efficient inter-node or device communication. Their optimization objectives typically prioritize FLOPs, peak intermediate tensor size, arithmetic intensity, or memory-capped execution.

These objectives are useful but incomplete for UPMEM-style PIM. On UPMEM, host-DPU transfer, MRAM-WRAM movement, WRAM capacity, DPU-local instruction cost, and host-mediated synchronization become first-order architectural constraints. Therefore, existing CPU/GPU contraction heuristics provide an algorithmic starting point, but their cost models cannot be transferred directly to PIM-aware execution without modification.

RQ3 *How have quantum circuit simulators been adapted for Processing-in-Memory architectures?*

The evidence base for RQ3 is extremely limited. The final corpus contains only one study that directly adapts quantum circuit simulation to commodity digital PIM hardware: PIMutation [9]. Therefore, RQ3 cannot be answered through cross-study synthesis in the same way as RQ1, RQ2, or RQ4. It is answered as a sparse-evidence finding.

PIMutation implements state-vector quantum circuit simulation on real UPMEM hardware. It demonstrates that commodity digital PIM can execute quantum simulation workloads, but it does not use a tensor-network representation. Its contribution is therefore most useful as evidence about the feasibility and limitations of mapping quantum simulation to UPMEM-style hardware, rather than as direct evidence for tensor-network contraction.

Because state-vector simulation requires storage proportional to 2^n , PIMutation remains limited to the low-30-qubit range on the evaluated hardware. The study also reports that its vector-partitioning

strategy is effective mainly when the circuit structure permits separability; for highly entangled circuits, the partitioning becomes less effective [9]. This limitation motivates the tensor-network direction, because tensor-network simulation does not necessarily store the full state vector explicitly. It may therefore support larger qubit counts when the circuit structure, slicing strategy, and contraction path keep intermediate tensors manageable. However, this possibility is not validated by PIMutation, because PIMutation uses a state-vector representation rather than tensor-network contraction.

The answer to RQ3 is therefore that quantum circuit simulation has been adapted to commodity digital PIM only in a very limited form in the included literature: as a state-vector UPMEM implementation. This finding motivates, but does not validate, the investigation of tensor-network representations as a possible alternative for future PIM-aware quantum simulation.

RQ4 *What performance insights can Processing-in-Memory workloads provide for tensor-network simulations?*

The Tier 3 proxy studies identify several hardware constraints that are transferable to the design of future tensor-network execution on UPMEM. First, host-DPU data transfer can achieve only a fraction of theoretical peak bandwidth, making host-mediated movement a major bottleneck [18]. Second, each DPU has limited WRAM, which forces careful tiling and limits the size of local intermediate tensors [6], [19]. Third, the lack of native floating-point support can make floating-point-heavy kernels costly on integer-oriented DPUs [20], [21]. Fourth, the absence of direct inter-DPU communication means that reductions, broadcasts, and synchronization must be organized through the host or through specialized communication frameworks [22], [23].

Proxy workloads also show which operation types are most informative for future tensor-network prototypes. GEMM-oriented PIM studies provide evidence for dense tensor-kernel mapping [19], [24]. SpMV, SpMM, graph, and sparse tensor studies provide evidence for irregular memory access, partitioning, and load balancing [25], [26], [27]. Communication studies provide evidence for the cost of moving slices, operands, and partial reductions between host and DPUs [18], [22].

These findings do not show that PIM will accelerate tensor-network quantum simulation. Rather, they identify the hardware constraints that any future PIM-aware tensor-network simulator would need to address: locality, transfer volume, tile size, numerical representation, and host-mediated coordination.

8.2. Cross-tier synthesis

PIM constraint	Evidence source	Transferability to TN contraction	Implication for future implementation
Host-DPU transfer	UPMEM transfer characterization [18]	Direct hardware constraint	Contraction planning should minimize host-mediated movement of operands, slices, and partial results.
Small WRAM scratchpad	UPMEM benchmark and tiling studies [6], [19]	Direct hardware constraint	Tensor kernels must be tiled so that local working sets and temporary buffers fit within WRAM.
MRAM-WRAM movement	UPMEM workload studies [6], [19]	Direct hardware constraint	Cost models should include local memory movement, not only host-DPU transfer and arithmetic work.
No native floating-point support	Floating-point and numerical PIM studies [20], [21]	Partial numerical proxy	Complex-valued tensor arithmetic may require software floating point, fixed point, block floating point, or hybrid host-DPU execution. This remains an untested issue for quantum tensor-network workloads.
No direct inter-DPU communication	UPMEM quantum and communication studies [9], [22]	Direct architectural constraint	Slicing and reduction strategies should avoid fine-grained inter-DPU dependencies and use planned host-mediated aggregation.

Table 3: Transferability of UPMEM proxy evidence to PIM-aware tensor-network contraction.

Across the three evidence tiers, the literature motivates a cautious design hypothesis: future tensor-network simulation on digital PIM would benefit from extending conventional CPU/GPU contraction-planning cost models with PIM-specific transfer, locality, and synchronization terms. This is not validated performance evidence; rather, it follows from the mismatch between the assumptions of conventional contraction planners and the constraints exposed by UPMEM proxy studies.

The evidence therefore motivates, but does not validate, a transfer-aware Host–PIM execution model. In this model, the host CPU would perform global orchestration, including contraction-path selection, slicing, scheduling, tensor-layout decisions, and synchronization. DPUs would execute local tiled kernels that reuse data near memory and minimize unnecessary host transfers. The role of the host would not be eliminated; rather, it would become the planner and coordinator for a transfer-aware execution pipeline.

The central design hypothesis is that host-DPU transfer volume, MRAM-WRAM movement, DPU-local reuse, numerical representation, and synchronization frequency should become first-class optimization targets. PIM is attractive because it can reduce movement between memory and conventional processors when data is reused near the DPUs. However, this benefit depends on

keeping operands, intermediate tiles, and partial results local. In some cases, a PIM-aware planner may prefer additional local arithmetic or recomputation if this avoids expensive host-DPU transfers or host-mediated reductions. Conversely, a contraction path that is efficient on a GPU may be inefficient on UPMEM-style PIM if it repeatedly materializes large intermediates, exceeds WRAM-friendly tile sizes, or routes partial results through the host.

The reviewed evidence does not justify a general claim that PIM will outperform CPUs or GPUs for tensor-network quantum simulation. It supports a narrower and more testable conclusion: PIM may be worth evaluating for selected tensor-network workloads when circuit structure, slicing choices, tensor layouts, and numerical representation provide enough locality to offset host-DPU transfer and DPU arithmetic limitations. The next section translates this hypothesis into design implications for future PIM-aware tensor-network prototypes.

9. Design Implications

The evidence synthesis suggests that tensor-network quantum circuit simulation on UPMEM-style Processing-in-Memory should not be treated as a direct port of existing CPU/GPU contraction frameworks. Existing contraction planners optimize important objectives such as FLOP count, peak intermediate size, slicing depth, and memory-capped execution. However, the reviewed studies indicate that a UPMEM-oriented design would also need to account for host-DPU transfers, WRAM capacity, MRAM-WRAM movement, tensor layout, numerical representation, and host-mediated synchronization.

The implications in this section should be interpreted as literature-derived design hypotheses for future prototype development. They do not show that PIM will necessarily outperform CPU or GPU tensor-network simulators.

9.1. Literature-derived implementation implications

The reviewed studies suggest several design considerations that future PIM-aware tensor-network simulators would need to test.

Separate global planning from local execution Contraction-path search, slicing decisions, tensor-layout planning, and global scheduling are likely to remain host-side tasks. UPMEM DPUs have limited local memory and are not well suited to global graph optimization or dynamic contraction planning. The host is therefore the natural location for computing the contraction plan and dispatching local work to the DPUs.

Use operation-specific execution routes A single generalized DPU kernel is unlikely to be efficient across all tensor-network operations. Dense contractions, tensor reshaping or permutation, elementwise transformations, sparse or irregular tensor kernels, and communication-heavy reductions may require different execution strategies. Frameworks such as SimplePIM [28], ATiM [19], and SparseP [25] illustrate the importance of workload-specific mapping strategies on PIM. Graph-processing studies provide additional proxy evidence for irregular memory access, partitioning, and load balancing on UPMEM-style hardware [20], [26], [29], but they do not directly evaluate quantum tensor-network workloads.

Optimize data movement, not only arithmetic A contraction path that is efficient on a CPU or GPU may be inefficient on UPMEM-style PIM if it repeatedly materializes large intermediates, moves partial results through the host, or fails to reuse data locally. Host-DPU transfer volume, MRAM-WRAM movement, DPU-local reuse, and synchronization frequency should therefore be treated as design constraints alongside FLOP count and peak intermediate size.

Design for WRAM-aware tiling Tensor operations dispatched to DPUs would need to be tiled to fit the small WRAM scratchpad. This requires explicit control over tile size, memory layout, buffering, and local reuse. Productivity frameworks may be useful for prototyping, but performance-critical tensor kernels are likely to require hardware-aware tiling and careful management of MRAM–WRAM transfers.

Treat numerical representation as a design variable UPMEM DPUs do not provide native floating-point arithmetic, so conventional FP32 or FP64 tensor kernels may be costly. Future work should investigate whether fixed-point, block-floating-point, quantized, or mixed-representation formats can reduce execution cost while preserving acceptable numerical accuracy for quantum simulation. This remains an open issue because the included studies do not directly evaluate complex-valued tensor-network arithmetic on UPMEM.

Avoid fine-grained inter-DPU dependencies UPMEM DPUs do not communicate directly. Algorithms requiring frequent peer-to-peer synchronization are therefore poorly matched to the architecture. Reductions over sliced tensor results should be planned as host-mediated operations or mapped onto communication frameworks such as PID-Comm [22].

9.2. Representative UPMEM constraints for tensor-network contraction

The Tier 3 proxy studies identify several UPMEM constraints that are relevant to a future tensor-network implementation. These constraints do not prove that UPMEM will accelerate tensor-network quantum simulation, but they define the architectural limits that such an implementation would need to respect. Table 4 summarizes the constraints used to motivate the cost-model discussion in Section 9.3.

Constraint	Cost term	UPMEM-specific issue	Main implication
Host–DPU traffic	$B_{\text{host-DPU}}$	Bidirectional host–DPU transfers can dominate when data is repeatedly moved [6], [18].	Batch transfers; avoid repeated slice movement.
MRAM–WRAM movement	$B_{\text{MRAM-WRAM}}$	DPU kernels require explicit movement through the MRAM–WRAM hierarchy [6], [19].	Tile for WRAM reuse.
DPU-local work	I_{DPU}	DPU execution includes integer-oriented local work, address calculation, and kernel overhead [6], [18].	Use kernels with enough local reuse.
Synchronization	N_{sync}	Fine-grained direct DPU-to-DPU communication is unavailable or limited; coordination is host-mediated or framework-managed [22], [23].	Prefer independent slices and coarse reductions.
Numerical representation	E_{num}	Floating-point or complex arithmetic may require software support or alternative numerical formats [21].	Evaluate fixed-point, block floating point, or mixed precision.
WRAM pressure	P_{WRAM}	Small per-DPU WRAM limits the active working set available to each DPU [6], [19].	Choose WRAM-friendly tile sizes.

Table 4: Representative UPMEM constraints motivating the PIM-aware cost terms.

9.3. Cost-model implications

The constraints summarized in Table 4 motivate extending conventional tensor-network planning objectives with PIM-specific transfer, locality, synchronization, and numerical-representation terms. A first-order PIM-aware scoring term can be written as:

$$C_{\text{PIM}} \approx \alpha \cdot B_{\text{host-DPU}} + \beta \cdot B_{\text{MRAM-WRAM}} + \gamma \cdot I_{\text{DPU}} + \delta \cdot N_{\text{sync}} + \eta \cdot E_{\text{num}} + \theta \cdot P_{\text{WRAM}}$$

The expression should be read as a cost assigned to a candidate execution plan, not as a calibrated performance predictor. A future planner could estimate these terms for alternative contraction paths, slicing strategies, tiling strategies, data layouts, and Host–DPU mappings, then prefer the plan with the lower combined cost.

In this decomposition, $B_{\text{host-DPU}}$ represents bidirectional traffic between host memory and DPU memory, $B_{\text{MRAM-WRAM}}$ represents local movement within the DPU memory hierarchy, I_{DPU} represents DPU-local execution work, N_{sync} represents synchronization and host-mediated coordination events, E_{num} represents numerical-representation overhead, and P_{WRAM} represents penalties caused by limited WRAM capacity.

The coefficients α , β , γ , δ , η , and θ are architecture-specific penalty weights rather than universal constants. In a future prototype, they could be estimated from microbenchmarks measuring host–DPU bandwidth, MRAM–WRAM transfer cost, DPU-local kernel cost, synchronization overhead, software floating-point or alternative-format overhead, and WRAM-sensitive tiling behavior.

The terms are not fully independent. Numerical-representation overhead may increase DPU-local work, WRAM pressure may increase MRAM–WRAM movement, and host–DPU traffic may be partially hidden by batching or overlapping execution. These interactions would need to be measured in a prototype implementation.

This scoring term changes how an efficient contraction plan would be evaluated for PIM. A path with more arithmetic may be preferable if it reduces transfers or keeps intermediate data local. Conversely, a path that minimizes FLOPs may be unattractive on UPMEM-style PIM if it repeatedly materializes large tensors, exceeds WRAM-friendly tile sizes, or forces frequent host-mediated reductions.

The reviewed studies do not instantiate or validate this scoring term for tensor-network quantum simulation on UPMEM. Instead, they justify the inclusion of its terms. The expression should therefore be interpreted as a literature-derived planning abstraction and a starting point for future prototype evaluation, not as a validated performance model.

9.4. Implications for future prototypes

The evidence does not imply that PIM will universally outperform CPU or GPU simulators. Instead, it suggests a narrower and more testable hypothesis: selected tensor-network contractions may benefit from near-memory execution when circuit structure, slicing strategy, tensor layout, and numerical representation provide enough locality to offset host–DPU transfer, limited WRAM capacity, and DPU arithmetic constraints.

Future prototypes should therefore begin with controlled circuit families and evaluate correctness, memory footprint, transfer volume, numerical error, and scalability before making strong claims about speedup. The most important outcome of an initial prototype may not be raw acceleration, but a clearer understanding of which parts of tensor-network quantum simulation are compatible with UPMEM-style execution constraints.

10. Limitations

This review has several limitations. First, screening, study selection, quality appraisal, and data extraction were conducted by a single reviewer. This introduces a risk of subjective selection and extraction decisions, because no second reviewer independently checked the inclusion decisions, quality scores, or extracted evidence.

Second, the review was not based on a formally pre-registered protocol. The protocol was developed before and during the search process, but it was not registered in an external repository such as OSF. To mitigate this limitation, the review documents the PICOC framework, inclusion and exclusion criteria, tier-specific selection rules, search strings, search logs, screening decisions, full-text decisions, quality appraisal records, and data extraction cards in the main text and appendices.

Third, the search strategy was limited to IEEE Xplore, the ACM Digital Library, and Scopus. It did not include a systematic search of arXiv, Google Scholar, dblp, GitHub repositories, or backward and forward citation chasing. This limits the strength of claims about the complete absence of direct studies, especially because quantum simulation, tensor-network methods, and Processing-in-Memory are fast-moving fields with substantial preprint and artifact activity. The review therefore claims only that no direct studies were identified by the executed database search and selection process, not that no such work exists anywhere.

Fourth, the database search strings were targeted rather than exhaustive. The tiered search strategy was developed after the strict target search returned no direct records, and each tier used a selected subset of the broader PICOC keyword space. During revision, search-string validation showed that some queries could have been broader or more consistently aligned with the full PICOC keyword groups. In particular, broader Tier 3 terms such as GEMM, matrix multiplication, tensor kernels, host-DPU communication, collective communication, and data transfer may retrieve additional digital-PIM proxy studies. Search validation also showed that matching result counts do not necessarily imply identical record sets, and that database results can change over time as new records are indexed. The review should therefore be interpreted as a mapping of the evidence identified by the executed and documented protocol, rather than as an exhaustive search of all possible digital-PIM proxy workloads.

Fifth, the quality appraisal contains an element of reviewer judgment. The included corpus combines different types of evidence, including quantum-simulation studies, tensor-network algorithm papers, compiler frameworks, UPMEM benchmark studies, graph workloads, dense and sparse matrix/tensor workloads, and communication frameworks. Although a common eight-criterion rubric was applied to all included studies, the meaning of criteria such as baseline quality, hardware realism, and contribution depth differs across tiers. The quality scores should therefore be interpreted as a structured transparency and reproducibility tool rather than as an absolute ranking of paper quality.

Sixth, the direct target intersection is sparse. No included study directly implements tensor-network quantum circuit simulation on UPMEM-style digital PIM. As a result, the synthesis relies on proxy evidence from three adjacent domains: quantum simulation on PIM, tensor-network contraction on conventional architectures, and digital-PIM proxy workloads on UPMEM. This proxy strategy is appropriate for a scoping review, but it limits the strength of conclusions that can be drawn about actual tensor-network performance on PIM.

Seventh, the Tier 3 proxy workloads are not equivalent to quantum tensor-network contractions. Workloads such as dense and sparse matrix/tensor kernels, SpMV, GEMM, graph analytics, numerical-representation studies, and host-DPU communication frameworks expose relevant memory, locality, and communication constraints, but they do not fully reproduce the dynamic

contraction patterns, complex-valued arithmetic, numerical-accuracy requirements, or circuit-dependent structure of quantum simulation.

Eighth, the reviewed PIM evidence is tied primarily to current digital PIM hardware, especially UPMEM. The conclusions may not generalize to future PIM platforms with different memory hierarchies, floating-point support, inter-DPU communication mechanisms, programming models, or host-device transfer characteristics.

Finally, the field is developing rapidly. The final database searches were executed in November 2025, with a publication cutoff of November 18, 2025. Studies published after this date were outside the scope of the review.

11. Conclusion

This Scoping Literature Review mapped the evidence relevant to accelerating tensor-network contraction for quantum circuit simulation using digital Processing-in-Memory architectures. The strict target intersection considered in this review, tensor-network quantum circuit simulation on UPMEM-style PIM, had no direct studies identified by the executed database search and eligibility process. The final corpus therefore combined evidence from three adjacent domains: quantum simulation on PIM, tensor-network contraction and slicing on conventional CPU/GPU or distributed-memory architectures, and UPMEM proxy workloads.

The main conclusion is that tensor-network simulation on digital PIM is unlikely to be addressed by a direct port of conventional CPU/GPU contraction planning. Existing tensor-network methods provide useful foundations for contraction ordering, slicing, tensor-layout optimization, and memory-aware scheduling. However, these methods generally assume hardware features that are not available, or not efficient, on commodity UPMEM systems. In contrast, the PIM literature highlights constraints that any future implementation would need to address: limited effective host-DPU bandwidth, small WRAM capacity, MRAM-WRAM movement, costly floating-point emulation, and host-mediated communication.

The review therefore motivates the hypothesis of a transfer-aware Host-PIM execution model. In such a model, the host CPU would perform global orchestration, including contraction-path selection, slicing, tensor-layout planning, scheduling, and synchronization, while DPUs would execute local tiled kernels near memory. This would shift the optimization target from minimizing only FLOPs or peak intermediate tensor size toward minimizing data movement, improving locality, and maximizing useful local reuse.

The evidence does not show that PIM is automatically superior for tensor-network quantum simulation. Rather, it identifies the conditions that future work must test: whether suitable circuit structures, slicing strategies, tensor layouts, and numerical representations can make selected parts of exact tensor-network simulation more locality-efficient on digital PIM than on conventional architectures.

Overall, the review shows that the research gap is not simply the absence of an implementation identified by the search. The gap is the absence of a validated PIM-aware contraction model that connects tensor-network planning with the physical constraints of commodity digital PIM hardware. This motivates future prototype work focused on transfer-aware contraction planning, WRAM-aware tiling, numerical representation, and host-mediated execution.

Bibliography

- [1] R. Feynman, “Simulating Physics with Computers,” *International Journal of Theoretical Physics*, vol. 21, no. 6, pp. 467–488, 1982.

- [2] J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018, doi: 10.22331/q-2018-08-06-79.
- [3] A. Cicero, M. A. Maleki, M. W. Azhar, A. F. Kockum, and P. Trancoso, “Simulation of Quantum Computers: Review and Acceleration Opportunities,” *ACM Transactions on Quantum Computing*, vol. 7, no. 1, Nov. 2025, doi: 10.1145/3762672.
- [4] F. Arute *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019, doi: 10.1038/s41586-019-1666-5.
- [5] R. Fu *et al.*, “Surpassing Sycamore: Achieving Energetic Superiority Through System-Level Circuit Simulation,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, in SC '24. Atlanta, GA, USA: IEEE Press, 2024, doi: 10.1109/SC41406.2024.00085.
- [6] J. Gomez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, “Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System,” *IEEE Access*, vol. 10, pp. 52565–52608, 2022, doi: 10.1109/ACCESS.2022.3174101.
- [7] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, “A Modern Primer on Processing in Memory,” in *Emerging Computing: From Devices to Systems*, M. M. S. Aly and A. Chattopadhyay, Eds., Singapore: Springer Nature Singapore, 2023, pp. 171–243. doi: 10.1007/978-981-16-7487-7_7.
- [8] F. Devaux, “The true Processing In Memory accelerator,” in *2019 IEEE Hot Chips 31 Symposium (HCS)*, Cupertino, CA, USA: IEEE, Aug. 2019, pp. 1–24. doi: 10.1109/HOTCHIPS.2019.8875680.
- [9] D. Lee, E. Jang, S. Choi, J. An, C. Kim, and W. W. Ro, “PIMutation: Exploring the Potential of Real PIM Architecture for Quantum Circuit Simulation,” in *Proc Asia South Pac Des Autom Conf*, Institute of Electrical, Electronics Engineers Inc., 2025, pp. 483–490. doi: 10.1145/3658617.3697583.
- [10] Y. Liu *et al.*, “Closing the “Quantum Supremacy” Gap: Achieving Real-Time Simulation of a Random Quantum Circuit Using a New Sunway Supercomputer,” in *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2021, pp. 1–12. doi: 10.1145/3458817.3487399.
- [11] Y. Chen *et al.*, “Lifetime-Based Optimization for Simulating Quantum Circuits on a New Sunway Supercomputer,” in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, in PPOPP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 148–159. doi: 10.1145/3572848.3577529.
- [12] F. Pan, H. Gu, L. Kuang, B. Liu, and P. Zhang, “Efficient Quantum Circuit Simulation by Tensor Network Methods on Modern GPUs,” *ACM Transactions on Quantum Computing*, vol. 5, no. 4, Nov. 2024, doi: 10.1145/3696465.
- [13] L. Ma, J. Ye, and E. Solomonik, “AutoHOOT: Automatic High-Order Optimization for Tensors,” in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, in PACT '20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 125–137. doi: 10.1145/3410463.3414647.
- [14] F. Li *et al.*, “SW_Qsim: a minimize-memory quantum simulator with high-performance on a new Sunway supercomputer,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '21. New York, NY, USA: Association for Computing Machinery, 2021. doi: 10.1145/3458817.3476161.

- [15] R. Kanakagiri and E. Solomonik, “Minimum Cost Loop Nests for Contraction of a Sparse Tensor with a Tensor Network,” in *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures*, in SPAA '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 169–181. doi: 10.1145/3626183.3659985.
- [16] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, “The tensor algebra compiler,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Oct. 2017, doi: 10.1145/3133901.
- [17] R. Yadav, A. Aiken, and F. Kjolstad, “DISTAL: the distributed tensor algebra compiler,” in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, in PLDI 2022. New York, NY, USA: Association for Computing Machinery, 2022, pp. 286–300. doi: 10.1145/3519939.3523437.
- [18] D. Lee, B. Hyun, T. Kim, and M. Rhu, “Analysis of Data Transfer Bottlenecks in Commercial PIM Systems: A Study With UPMEM-PIM,” *IEEE Computer Architecture Letters*, vol. 23, no. 2, pp. 179–182, Jul. 2024, doi: 10.1109/LCA.2024.3387472.
- [19] Y. Shin, D. Kang, and H. Sung, “ATiM: Autotuning tensor programs for processing-in-DRAM,” 2024.
- [20] M. Barkhordar *et al.*, “ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System,” in *2025 IEEE International Symposium on Workload Characterization (IISWC)*, Oct. 2025, pp. 257–271. doi: 10.1109/IISWC66894.2025.00030.
- [21] M. Item, G. F. Oliveira, J. Gómez-Luna, M. Sadrosadati, Y. Guo, and O. Mutlu, “TransPimLib: Efficient Transcendental Functions for Processing-in-Memory Systems,” in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2023, pp. 235–247. doi: 10.1109/ISPASS57527.2023.00031.
- [22] S. U. Noh *et al.*, “PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2024, pp. 245–260. doi: 10.1109/ISCA59077.2024.00027.
- [23] F. Yang, Z. Zhou, C. Li, T.-Y. Ho, and M.-C. Yang, “Pac-PIM: A Parallel Communication Framework for Commodity Processing-in-memory Systems,” *ACM Trans. Archit. Code Optim.*, Nov. 2025, doi: 10.1145/3776751.
- [24] H. Kim *et al.*, “Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices,” in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, in CGO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 225–240. doi: 10.1145/3696443.3708953.
- [25] C. Giannoula, I. Fernandez, J. G. Luna, N. Koziris, G. Goumas, and O. Mutlu, “SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 1, Feb. 2022, doi: 10.1145/3508041.
- [26] C. Giannoula *et al.*, “PyGim : An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 3, p. 43:1–43:36, Dec. 2024, doi: 10.1145/3700434.
- [27] D. Pacheco, L. Sousa, and A. Ilic, “PRISM: Processing-In-Memory Sparse MTTKRP for Tensor Decomposition Acceleration,” in *2025 IEEE International Symposium on Workload Characterization (IISWC)*, Oct. 2025, pp. 246–256. doi: 10.1109/IISWC66894.2025.00029.

- [28] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, “SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory,” in *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, Oct. 2023, pp. 99–111. doi: 10.1109/PACT58117.2023.00017.
- [29] M.-H. Jang, M.-K. Shin, T. Park, Y. Park, and S.-W. Kim, “An Efficient PIM-Based Graph Engine on a Single Machine,” in *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, in CIKM '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 4832–4836. doi: 10.1145/3746252.3760936.
- [30] M. Pellauer *et al.*, “Symphony: Orchestrating Sparse and Dense Tensors with Hierarchical Heterogeneous Processing,” *ACM Trans. Comput. Syst.*, vol. 41, no. 1, Dec. 2023, doi: 10.1145/3630007.
- [31] Z. Wu, H. Zhao, H. Liu, W. Wen, and J. Li, “gHyPart: GPU-friendly End-to-End Hypergraph Partitioner,” *ACM Trans. Archit. Code Optim.*, vol. 22, no. 1, Mar. 2025, doi: 10.1145/3711925.
- [32] T. Nguyen, D. Lyakh, E. Dumitrescu, D. Clark, J. Larkin, and A. McCaskey, “Tensor Network Quantum Virtual Machine for Simulating Quantum Circuits at Exascale,” *ACM Transactions on Quantum Computing*, vol. 4, no. 1, Oct. 2022, doi: 10.1145/3547334.
- [33] B. Friesel, M. Lütke Dreimann, and O. Spinczyk, “A Full-System Perspective on UPMEM Performance,” in *DIMES - Proc. Workshop Disrupt. Memory Syst., Part: SOS*, Association for Computing Machinery, Inc., 2023, pp. 1–7. doi: 10.1145/3609308.3625266.
- [34] D. Chen *et al.*, “A General Offloading Approach for Near-DRAM Processing-In-Memory Architectures,” in *Proc. - IEEE Int. Parallel Distrib. Process. Symp., IPDPS*, Institute of Electrical, Electronics Engineers Inc., 2022, pp. 246–257. doi: 10.1109/IPDPS53621.2022.00032.
- [35] H. Kal, C. Yoo, and W. W. Ro, “AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, in MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 815–827. doi: 10.1145/3613424.3614314.
- [36] F. A. Siddique *et al.*, “Architectural Modeling and Benchmarking for Digital DRAM PIM,” in *Proc. - IEEE Int. Symp. Workload Charact., IISWC*, Institute of Electrical, Electronics Engineers Inc., 2024, pp. 247–261. doi: 10.1109/IISWC63097.2024.00030.
- [37] J. Gomez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, “Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-In-Memory Hardware,” in *Int. Green Sustain. Comput. Conf., IGSC*, Institute of Electrical, Electronics Engineers Inc., 2021. doi: 10.1109/IGSC54211.2021.9651614.
- [38] A. A. Khan, H. Farzaneh, K. F. A. Friebel, C. Fournier, L. Chelini, and J. Castrillon, “CINM (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms,” in *Int Conf Archit Support Program Lang Oper Syst ASPLOS*, Association for Computing Machinery, 2025, pp. 31–46. doi: 10.1145/3622781.3674189.
- [39] S. Yu, H. Kim, K. Jeun, S. Hwang, S. Cho, and E. Lee, “ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, in MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 49–62. doi: 10.1145/3725843.3756017.

- [40] S. Shi, X. Li, Z. Pan, P. Zhang, and N. Sun, “CoPIM: A Collaborative Scheduling Framework for Commodity Processing-in-memory Systems,” in *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, Nov. 2024, pp. 44–51. doi: 10.1109/ICCD63220.2024.00018.
- [41] B. Hyun, T. Kim, D. Lee, and M. Rhu, “Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Mar. 2024, pp. 263–279. doi: 10.1109/HPCA57654.2024.00029.
- [42] G. Jonatan *et al.*, “Scalability Limitations of Processing-in-Memory using Real System Evaluations,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 1, Feb. 2024, doi: 10.1145/3639046.
- [43] C. Silvano *et al.*, “A Survey on Deep Learning Hardware Accelerators for Heterogeneous HPC Platforms,” *ACM Comput. Surv.*, vol. 57, no. 11, Jun. 2025, doi: 10.1145/3729215.
- [44] J. Huang, C. D. Yu, and R. A. van de Geijn, “Strassen’s Algorithm Reloaded on GPUs,” *ACM Trans. Math. Softw.*, vol. 46, no. 1, Mar. 2020, doi: 10.1145/3372419.
- [45] L. Liang *et al.*, “Matrix: Multi-Cipher Structures Dataflow for Parallel and Pipelined TFHE Accelerator,” *ACM Trans. Archit. Code Optim.*, vol. 22, no. 3, Sep. 2025, doi: 10.1145/3750446.
- [46] B. Marker, R. van de Geijn, and D. Batory, “DSLs, DLA, DxT, and MDE in CSE,” in *Proceedings of the 5th International Workshop on Software Engineering for Computational Science and Engineering*, in SE-CSE '13. San Francisco, California: IEEE Press, 2013, pp. 84–87.
- [47] B. A. Sanders, R. Bartlett, E. Deumens, V. Lotrich, and M. Ponton, “A Block-Oriented Language and Runtime System for Tensor Algebra with Very Large Arrays,” in *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '10. USA: IEEE Computer Society, 2010, pp. 1–11. doi: 10.1109/SC.2010.3.
- [48] G. Bisbas *et al.*, “A shared compilation stack for distributed-memory parallelism in stencil DSLs,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, in ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 38–56. doi: 10.1145/3620666.3651344.
- [49] D. B. Larkins, J. Snyder, and J. Dinan, “Accelerated Work Stealing,” in *Proceedings of the 48th International Conference on Parallel Processing*, in ICPP '19. New York, NY, USA: Association for Computing Machinery, 2019. doi: 10.1145/3337821.3337878.
- [50] D. Kempf, R. Heß, S. Müthing, and P. Bastian, “Automatic Code Generation for High-performance Discontinuous Galerkin Methods on Modern Architectures,” *ACM Trans. Math. Softw.*, vol. 47, no. 1, Dec. 2020, doi: 10.1145/3424144.
- [51] S. Soldavini, K. Friebel, M. Tibaldi, G. Hempel, J. Castrillon, and C. Pilato, “Automatic Creation of High-bandwidth Memory Architectures from Domain-specific Languages: The Case of Computational Fluid Dynamics,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 2, Mar. 2023, doi: 10.1145/3563553.
- [52] M. G. Knepley and A. R. Terrel, “Finite Element Integration on GPUs,” *ACM Trans. Math. Softw.*, vol. 39, no. 2, Feb. 2013, doi: 10.1145/2427023.2427027.
- [53] S. Krishnamoorthy, U. Catalyurek, J. Nieplocha, A. Rountev, and P. Sadayappan, “Hypergraph partitioning for automatic memory hierarchy management,” in *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, in SC '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 98–es. doi: 10.1145/1188455.1188558.

- [54] Z. Xie, J. Liu, J. Li, and D. Li, “Merchandise: Data Placement on Heterogeneous Memory for Task-Parallel HPC Applications with Load-Balance Awareness,” in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, in PPoPP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 204–217. doi: 10.1145/3572848.3577497.
- [55] H. Ushijima-Mwesigwa, R. Shaydulin, C. F. A. Negre, S. M. Mniszewski, Y. Alexeev, and I. Safro, “Multilevel Combinatorial Optimization across Quantum Architectures,” *ACM Transactions on Quantum Computing*, vol. 2, no. 1, Feb. 2021, doi: 10.1145/3425607.
- [56] J. S. Firoz *et al.*, “Optimizing Data Distribution and Kernel Performance for Efficient Training of Chemistry Foundation Models: A Case Study with MACE,” in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, in HPDC '25. New York, NY, USA: Association for Computing Machinery, 2025. doi: 10.1145/3731545.3731594.
- [57] M. Metcalf, P. Andrés-Martínez, and N. Fitzpatrick, “Realizing Quantum Kernel Models at Scale with Matrix Product State Simulation,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, in SC '24. Atlanta, GA, USA: IEEE Press, 2024. doi: 10.1109/SC41406.2024.00086.
- [58] M. Hall *et al.*, “Scheduling Language Chronology: Past, Present, and Future,” *ACM Trans. Archit. Code Optim.*, vol. 22, no. 3, Sep. 2025, doi: 10.1145/3743135.
- [59] R. Dorozhinskii and M. Bader, “SeisSol on Distributed Multi-GPU Systems: CUDA Code Generation for the Modal Discontinuous Galerkin Method,” in *The International Conference on High Performance Computing in Asia-Pacific Region*, in HPCAsia '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 69–82. doi: 10.1145/3432261.3436753.
- [60] D. Park, H. Kim, J. Kim, T. Kim, and J. Lee, “SnuQS: scaling quantum circuit simulation using storage devices,” in *Proceedings of the 36th ACM International Conference on Supercomputing*, in ICS '22. New York, NY, USA: Association for Computing Machinery, 2022. doi: 10.1145/3524059.3532375.
- [61] M. Wang, C.-chin Huang, and J. Li, “Supporting Very Large Models using Automatic Dataflow Graph Partitioning,” in *Proceedings of the Fourteenth EuroSys Conference 2019*, in EuroSys '19. New York, NY, USA: Association for Computing Machinery, 2019. doi: 10.1145/3302424.3303953.
- [62] E. Georganas *et al.*, “Tensor processing primitives: a programming abstraction for efficiency and portability in deep learning workloads,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '21. New York, NY, USA: Association for Computing Machinery, 2021. doi: 10.1145/3458817.3476206.
- [63] S. Bavikadi, P. R. Sutradhar, J. Thangellamudi, and S. Manoj Pudukotai Dinakarrao, “3D-PLANE: A 3D-stacked DRAM-based Programmable SLM Accelerator Capable of Near-Memory and Energy-Efficient Parallel Processing,” in *Proceedings of the Great Lakes Symposium on VLSI 2025*, in GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 540–546. doi: 10.1145/3716368.3735285.
- [64] B. Li, J. R. Doppa, P. P. Pande, K. Chakrabarty, J. X. Qiu, and H. (H. Li, “3D-ReG: A 3D ReRAM-based Heterogeneous Architecture for Training Deep Neural Networks,” *J. Emerg. Technol. Comput. Syst.*, vol. 16, no. 2, Jan. 2020, doi: 10.1145/3375699.

- [65] H. K. Ahn, S. Lee, and S.-O. Jung, "A CNN-Based Super-Resolution Processor With Short-Term Caching for Real-Time UHD Upscaling," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 3, pp. 1198–1207, Mar. 2024, doi: 10.1109/TCSI.2023.3346440.
- [66] T. Sterling and L. Bergman, "A design analysis of a hybrid technology multithreaded architecture for petaflops scale computation3," in *Proceedings of the 13th International Conference on Supercomputing*, in ICS '99. New York, NY, USA: Association for Computing Machinery, 1999, pp. 286–293. doi: 10.1145/305138.305205.
- [67] B. Wang, Y. Wu, and A. Ivanov, "A Fast Diagnosis Scheme for Distributed Small Embedded SRAMs," in *Proceedings of the Conference on Design, Automation and Test in Europe - Volume 2*, in DATE '05. USA: IEEE Computer Society, 2005, pp. 852–857. doi: 10.1109/DATE.2005.13.
- [68] H. Sharma, P. Dhingra, J. Doppa, U. Ogras, and P. P. Pande, "A Heterogeneous Chiplet Architecture for Accelerating End-to-End Transformer Models," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 3, Apr. 2025, doi: 10.1145/3718487.
- [69] M. M. Ghaffar, C. Sudarshan, C. Weis, M. Jung, and N. Wehn, "A Low Power In-DRAM Architecture for Quantized CNNs using Fast Winograd Convolutions," in *Proceedings of the International Symposium on Memory Systems*, in MEMSYS '20. New York, NY, USA: Association for Computing Machinery, 2021, pp. 158–168. doi: 10.1145/3422575.3422790.
- [70] C. Sudarshan *et al.*, "A Novel DRAM-Based Process-in-Memory Architecture and its Implementation for CNNs," in *2021 26th Asia and South Pacific Design Automation Conference (ASP-DAC)*, Jan. 2021, pp. 35–42. Accessed: Nov. 21, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/9371531/>
- [71] A. Gebregiorgis *et al.*, "A Survey on Memory-centric Computer Architectures," *J. Emerg. Technol. Comput. Syst.*, vol. 18, no. 4, Oct. 2022, doi: 10.1145/3544974.
- [72] A. Baumstark, M. A. Jibril, and K.-U. Sattler, "Accelerating Large Table Scan using Processing-In-Memory Technology," in *Lect. Notes Informatics (LNI), Proc. - Series Ges. Inform. (GI)*, Gesellschaft fur Informatik (GI), 2023, pp. 797–814. doi: 10.18420/BTW2023-51.
- [73] R. Ma *et al.*, "Accelerating Regular Path Queries over Graph Database with Processing-in-Memory," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, in DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. doi: 10.1145/3649329.3656235.
- [74] L.-C. Chen, C.-C. Ho, and Y.-H. Chang, "Accelerating RNA-Seq Quantification on a Real Processing-in-Memory System," *IEEE Transactions on Computers*, vol. 74, no. 7, pp. 2334–2347, 2025, doi: 10.1109/TC.2025.3558075.
- [75] A. Baumstark, M. A. Jibril, and K.-U. Sattler, "Adaptive Query Compilation with Processing-in-Memory," in *2023 IEEE 39th International Conference on Data Engineering Workshops (ICDEW)*, Apr. 2023, pp. 191–197. doi: 10.1109/ICDEW58674.2023.00035.
- [76] C. Zhang, Z. Yuan, X. Li, and G. Sun, "Algorithm-Hardware Co-Design for Energy-Efficient A/D Conversion in ReRAM-Based Accelerators," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2024, pp. 1–6. doi: 10.23919/DATE58400.2024.10546760.
- [77] J. Jeon, S. Park, S. Kashyap, S. Kannan, D. Zhou, and J. Kang, "Analyzing and Enhancing ArckFS: An Anecdotal Example of Benefits of Artifact Evaluation," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, in SOSOP '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1149–1157. doi: 10.1145/3731569.3768291.

- [78] A. Sangiovanni-Vincentelli, Z. Liang, Z. Zhou, and J. Zhang, “Automated Design of Chiplets,” in *Proceedings of the 2023 International Symposium on Physical Design*, in ISPD '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1–8. doi: 10.1145/3569052.3578917.
- [79] C. H. Kim, W. J. Lee, Y. Paik, S. Y. Kim, and S. W. Kim, “BL-PIM: Varying the Burst Length to Realize the All-Bank Performance and Minimize the Multi-Workload Interference for in-DRAM PIM,” *IEEE Access*, vol. 11, pp. 81143–81156, 2023, doi: 10.1109/ACCESS.2023.3300893.
- [80] L. Zhao, Y. Ling, and S. Yang, “Buffer Size Impact on End-to-End DNN Acceleration in Near-Bank DRAM-PIM Systems,” in *Proceedings of the 18th ACM International Systems and Storage Conference*, in SYSTOR '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 220–221. doi: 10.1145/3757347.3759149.
- [81] J. Nider, J. Dagger, N. Gharavi, D. Ng, and A. (S. Fedorova, “Bulk JPEG decoding on in-memory processors,” in *Proceedings of the 15th ACM International Conference on Systems and Storage*, in SYSTOR '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 51–57. doi: 10.1145/3534056.3534946.
- [82] D. Kim, S.-Y. Lee, K. Min, and S. Kang, “Construction of Realistic Place-and-Route Benchmarks for Machine Learning Applications,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 6, pp. 2030–2042, Jun. 2023, doi: 10.1109/TCAD.2022.3209530.
- [83] S. Ollivier, S. Longofono, P. Dutta, J. Hu, S. Bhanja, and A. K. Jones, “CORUSCANT: Fast Efficient Processing-in-Racetrack Memories,” in *Proc. Annu. Int. Symp. Microarchitecture MICRO*, IEEE Computer Society, 2022, pp. 784–798. doi: 10.1109/MICRO56248.2022.00060.
- [84] B. Kim, C. Lee, G. Kim, and E. Park, “Cost-Effective Extension of DRAM-PIM for Group-Wise LLM Quantization,” *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 53–56, 2025, doi: 10.1109/LCA.2025.3532682.
- [85] M. A. Kabir *et al.*, “DA-VinCi: A Deep-Learning Accelerator Overlay Using In-Memory Computing,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, no. 4, Nov. 2025, doi: 10.1145/3770756.
- [86] C. Lim *et al.*, “Design and Analysis of a Processing-in-DIMM Join Algorithm: A Case Study with UPMEM DIMMs,” *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023, doi: 10.1145/3589258.
- [87] S. B. Rai *et al.*, “Design space for scaling-in general purpose computing within the DDR DRAM hierarchy for map-reduce workloads,” in *Proceedings of the 18th ACM International Conference on Computing Frontiers*, in CF '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 113–123. doi: 10.1145/3457388.3458661.
- [88] M. Chen *et al.*, “DRIM-ANN: An Approximate Nearest Neighbor Search Engine based on Commercial DRAM-PIMs,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 820–836. doi: 10.1145/3712285.3759801.
- [89] B. Kim, S. Li, B. Taylor, and Y. Chen, “Efficient and Robust Edge AI: Software, Hardware, and the Co-design,” *ACM Trans. Embed. Comput. Syst.*, vol. 24, no. 3, Apr. 2025, doi: 10.1145/3724396.
- [90] J. Kim and D. Nam, “Enhanced JPEG Decoding Using PIM Architectures with Parallel MCU Processing,” in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, Jun. 2025, pp. 1228–1237. doi: 10.1109/IPDPS64566.2025.00111.

- [91] T. Kim, Y. Han, S. Ha, J. Kim, and L.-S. Kim, “EOD: Enabling Low Latency GNN Inference via Near-Memory Concatenate Aggregation,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, in ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1125–1139. doi: 10.1145/3695053.3731083.
- [92] J. Gómez-Luna *et al.*, “Evaluating Machine Learning Workloads on Memory-Centric Computing Systems,” in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2023, pp. 35–49. doi: 10.1109/ISPASS57527.2023.00013.
- [93] J. Kim, J. Jung, and J. Kang, “Expediting Hazard Pointers with Bounded RCU Critical Sections,” in *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures*, in SPAA '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–13. doi: 10.1145/3626183.3659941.
- [94] C. Shin, J. Song, S. Na, J. Sung, H. Jang, and J. Lee, “FALA: Locality-Aware PIM-Host Cooperation for Graph Processing with Fine-Grained Column Access,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, in MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1520–1534. doi: 10.1145/3725843.3756127.
- [95] Z. Lv, T. Bu, and Q. Yang, “Fast, Transparent and Accurate Simulation of Thousand Processing-in-Memory Cores,” in *Proc. ACM Great Lakes Symp. VLSI GLSVLSI*, Association for Computing Machinery, 2025, pp. 29–34. doi: 10.1145/3716368.3735197.
- [96] P. R. Sutradhar, S. Bavikadi, M. Indovina, S. M. Pudukotai Dinakarrao, and A. Ganguly, “FlutPIM: A Look-up Table-based Processing in Memory Architecture with Floating-point Computation Support for Deep Learning Applications,” in *Proceedings of the Great Lakes Symposium on VLSI 2023*, in GLSVLSI '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 207–211. doi: 10.1145/3583781.3590313.
- [97] A. Morad, L. Yavits, and R. Ginosar, “GP-SIMD Processing-in-Memory,” *ACM Trans. Archit. Code Optim.*, vol. 11, no. 4, Jan. 2015, doi: 10.1145/2686875.
- [98] R. Morimoto, K. Ichinose, and T. Ugawa, “Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory,” in *Int. Symp. Mem. Manage. ISMM*, Association for Computing Machinery, 2025, pp. 122–133. doi: 10.1145/3735950.3735961.
- [99] T. Glint, M. Awasthi, and J. Mekie, “Hardware-Software Co-Design of a Collaborative DNN Accelerator for 3D Stacked Memories with Multi-Channel Data,” in *Proceedings of the 29th Asia and South Pacific Design Automation Conference*, in ASPDAC '24. Incheon, Republic of Korea: IEEE Press, 2024, pp. 454–459. doi: 10.1109/ASP-DAC58780.2024.10473935.
- [100] C. Liu *et al.*, “HeterRAG: Heterogeneous Processing-in-Memory Acceleration for Retrieval-augmented Generation,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, in ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 884–898. doi: 10.1145/3695053.3731089.
- [101] M. Lütke Dreimann, B. Friesel, and O. Spinczyk, “HetSim: A Simulator for Task-based Scheduling on Heterogeneous Hardware,” in *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*, in ICPE '24 Companion. New York, NY, USA: Association for Computing Machinery, 2024, pp. 261–268. doi: 10.1145/3629527.3652275.
- [102] J. Choe, A. Crotty, T. Moreshet, M. Herlihy, and R. I. Bahar, “HybriDS: Cache-Conscious Concurrent Data Structures for Near-Memory Processing Architectures,” in *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures*, in SPAA '22. New York,

- NY, USA: Association for Computing Machinery, 2022, pp. 321–332. doi: 10.1145/3490148.3538591.
- [103] Y. Wu, K. Chen, G. Chen, D. Jiang, H. Li, and L. Shou, “HyperMR: Efficient Hypergraph-enhanced Matrix Storage on Compute-in-Memory Architecture,” *Proc. ACM Manag. Data*, vol. 3, no. 1, Feb. 2025, doi: 10.1145/3709695.
 - [104] P. Das, P. R. Sutradhar, M. Indovina, S. M. P. Dinakarrao, and A. Ganguly, “Implementation and Evaluation of Deep Neural Networks in Commercially Available Processing in Memory Hardware,” in *2022 IEEE 35th International System-on-Chip Conference (SOCC)*, Sep. 2022, pp. 1–6. doi: 10.1109/SOCC56010.2022.9908126.
 - [105] D. Lee *et al.*, “Improving In-Memory Database Operations with Acceleration DIMM (AxDIMM),” in *Proceedings of the 18th International Workshop on Data Management on New Hardware*, in DaMoN '22. New York, NY, USA: Association for Computing Machinery, 2022. doi: 10.1145/3533737.3535093.
 - [106] J. Seo, S. Park, and S. Kang, “Improving Timing & Power Trade-off in Post-place Optimization Using Multi-agent Reinforcement Learning,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3676536.3676702>
 - [107] C. Lin *et al.*, “Ironman: Accelerating Oblivious Transfer Extension for Privacy-Preserving AI with Near-Memory Processing,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, in MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 354–368. doi: 10.1145/3725843.3756025.
 - [108] O. Mutlu, “Lightning Talk: Memory-Centric Computing,” in *Proceedings of the 60th Annual ACM/IEEE Design Automation Conference*, IEEE Press, 2025, pp. 1–2. [Online]. Available: <https://doi.org/10.1109/DAC56929.2023.10247896>
 - [109] Z. Liu, J. Schult, P. Xu, and T. Roscoe, “Mainframe-Style Channel Controllers for Modern Disaggregated Memory Systems,” in *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems*, in APSys '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 82–90. doi: 10.1145/3725783.3764403.
 - [110] J. Park, Q. A. Hoang, J. Ta, S.-H. Kang, K. Sohn, and S.-W. Jun, “MAPLE: Flexible-Precision Processing-In-Memory Architecture for Efficient On-Device ML,” in *Proceedings of the Great Lakes Symposium on VLSI 2025*, in GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 192–199. doi: 10.1145/3716368.3735216.
 - [111] W. Huangfu, X. Li, S. Li, X. Hu, P. Gu, and Y. Xie, “MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, in MICRO-52. New York, NY, USA: Association for Computing Machinery, 2019, pp. 587–599. doi: 10.1145/3352460.3358329.
 - [112] J. B. Brockman, P. M. Kogge, T. L. Sterling, V. W. Freeh, and S. K. Kuntz, “Microservers: a new memory semantics for massively parallel computing,” in *Proceedings of the 13th International Conference on Supercomputing*, in ICS '99. New York, NY, USA: Association for Computing Machinery, 1999, pp. 454–463. doi: 10.1145/305138.305234.
 - [113] M. Jang, J. Rhee, W. Lee, S. Zhao, and J. Kang, “Modular Hardware Design of Pipelined Circuits with Hazards,” *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Jun. 2024, doi: 10.1145/3656378.

- [114] X. Xie, P. Gu, Y. Ding, D. Niu, H. Zheng, and Y. Xie, “MPU: Memory-centric SIMT Processor via In-DRAM Near-bank Computing,” *ACM Trans. Archit. Code Optim.*, vol. 20, no. 3, Jul. 2023, doi: 10.1145/3603113.
- [115] V. Mohith, S. S. Borah, P. Sundaravadivel, and R. Sakthivel, “Multi-Parameter Comparative Study of Selective In-Memory Computing Processor Architectures for Performance Optimization in Accelerated Edge Computing,” in *Proceedings of the Great Lakes Symposium on VLSI 2025*, in GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1002–1007. doi: 10.1145/3716368.3735304.
- [116] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, “NDPBridge: Enabling Cross-Bank Coordination in Near-DRAM-Bank Processing Architectures,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, in ISCA '24. Buenos Aires, Argentina: IEEE Press, 2025, pp. 628–643. doi: 10.1109/ISCA59077.2024.00052.
- [117] S. R. Sundara Raman, L. John, and J. P. Kulkarni, “NEM-GNN: DAC/ADC-less, Scalable, Reconfigurable, Graph and Sparsity-Aware Near-Memory Accelerator for Graph Neural Networks,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, May 2024, doi: 10.1145/3652607.
- [118] C. Li, Z. Zhou, X. Li, G. Sun, and D. Niu, “NMExplorer: An Efficient Exploration Framework for DIMM-Based Near-Memory Tensor Reduction,” in *Proceedings of the 60th Annual ACM/IEEE Design Automation Conference*, IEEE Press, 2025, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/DAC56929.2023.10247764>
- [119] H. Kim *et al.*, “NMP-PaK: Near-Memory Processing Acceleration of Scalable De Novo Genome Assembly,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, in ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1834–1847. doi: 10.1145/3695053.3731056.
- [120] H. Kim, Y. Zhao, A. Pavlo, and P. B. Gibbons, “No Cap, This Memory Slaps: Breaking through the Memory Wall of Transactional Database Systems with Processing-in-Memory,” *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4241–4254, Jul. 2025, doi: 10.14778/3749646.3749690.
- [121] Y. Zhao *et al.*, “Optimal Batch-Dynamic kd-trees for Processing-in-Memory with Applications,” in *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, in SPAA '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 350–366. doi: 10.1145/3694906.3743318.
- [122] C. Sudarshan, M. H. Sadi, C. Weis, and N. Wehn, “Optimization of DRAM based PIM Architecture for Energy-Efficient Deep Neural Network Training,” in *Proc IEEE Int Symp Circuits Syst*, Institute of Electrical, Electronics Engineers Inc., 2022, pp. 1472–1476. doi: 10.1109/ISCAS48785.2022.9937832.
- [123] Y. He *et al.*, “PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 766–782. [Online]. Available: <https://doi.org/10.1145/3676641.3716009>
- [124] M. Mognol, D. Lavenier, and J. Legriel, “Parallelization of the Banded Needleman & Wunsch Algorithm on UPMEM PiM Architecture for Long DNA Sequence Alignment,” in *Proceedings of the 53rd International Conference on Parallel Processing*, in ICPP '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1062–1071. doi: 10.1145/3673038.3673094.

- [125] B. Friesel, M. L. Dreimann, and O. Spinczyk, “Performance Models for Task-based Scheduling with Disruptive Memory Technologies,” in *Proceedings of the 2nd Workshop on Disruptive Memory Systems*, in DIMES '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–8. doi: 10.1145/3698783.3699376.
- [126] Y. Gu *et al.*, “PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 862–881. [Online]. Available: <https://doi.org/10.1145/3676641.3716267>
- [127] I. Hwang *et al.*, “PIM-CARE: A Compiler-Assisted Dynamic Resource Allocation Framework for Real-world DRAM PIM,” in *Proceedings of the 39th ACM International Conference on Supercomputing*, in ICS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 458–472. doi: 10.1145/3721145.3725777.
- [128] J. Kim, D. Kim, S. Kang, B. Hyun, I. Lee, and Y. Park, “PIM-CCA: An Efficient PIM Architecture with Optimized Integration of Configurable Functional Units,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, in MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 63–77. doi: 10.1145/3725843.3756034.
- [129] C. Li *et al.*, “PIM-DL: Expanding the Applicability of Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, in ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 879–896. doi: 10.1145/3620665.3640376.
- [130] D. Lee, B. Hyun, T. Kim, and M. Rhu, “PIM-MMU: A Memory Management Unit for Accelerating Data Transfers in Commercial PIM Systems,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Nov. 2024, pp. 627–642. doi: 10.1109/MICRO61859.2024.00053.
- [131] S. Rhyner *et al.*, “PIM-Opt: Demystifying Distributed Optimization Algorithms on a Real-World Processing-In-Memory System,” in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, in PACT '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 201–218. doi: 10.1145/3656019.3676947.
- [132] J. Alsop, S. Aga, M. Ibrahim, M. Islam, N. Jayasena, and A. McCrabb, “PIM-Potential: Broadening the Acceleration Reach of PIM Architectures,” in *Proceedings of the International Symposium on Memory Systems*, in MEMSYS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–12. doi: 10.1145/3695794.3695795.
- [133] A. Lopes, D. Castro, and P. Romano, “PIM-STM: Software Transactional Memory for Processing-In-Memory Systems,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, in ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 897–911. doi: 10.1145/3620665.3640428.
- [134] H. Kang *et al.*, “PIM-Tree: A Skew-Resistant Index for Processing-in-Memory,” *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 946–958, Dec. 2022, doi: 10.14778/3574245.3574275.
- [135] H. Kang *et al.*, “PIM-trie: A Skew-resistant Trie for Processing-in-Memory,” in *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, in SPAA '23. New

- York, NY, USA: Association for Computing Machinery, 2023, pp. 1–14. doi: 10.1145/3558481.3591070.
- [136] S. Ma *et al.*, “PIMSAB: A Processing-In-Memory System with Spatially-Aware Communication and Bit-Serial-Aware Computation,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 4, Nov. 2024, doi: 10.1145/3690824.
 - [137] D. Christ, L. Steiner, M. Jung, and N. Wehn, “PIMSys: A Virtual Prototype for Processing in Memory,” in *Proceedings of the International Symposium on Memory Systems*, in MEMSYS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 26–33. doi: 10.1145/3695794.3695797.
 - [138] Y. Hu *et al.*, “PLAIN: Leveraging High Internal Bandwidth in PIM for Accelerating Large Language Model Inference via Mixed-Precision Quantization,” in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, Oct. 2025, pp. 1–9. doi: 10.1109/ICCAD66269.2025.11240991.
 - [139] R. Langarita, A. Armejach, P. Ibáñez, J. Alastruey-Benedé, and M. Moretó, “Porting and Optimizing BWA-MEM2 Using the Fujitsu A64FX Processor,” *IEEE/ACM Trans. Comput. Biol. Bioinformatics*, vol. 20, no. 5, pp. 3139–3153, Apr. 2023, doi: 10.1109/TCBB.2023.3264514.
 - [140] P. Carrinho, O. Ferraz, J. D. Ferreira, Y. Falevoz, V. Silva, and G. Falcao, “Processing Multi-Layer Perceptrons In-Memory,” in *IEEE Workshop Signal. Process. Syst. SiPS Des. Implement.*, Institute of Electrical, Electronics Engineers Inc., 2024, pp. 7–12. doi: 10.1109/SiPS62058.2024.00010.
 - [141] A. Baumstark, M. A. Jibril, and K.-U. Sattler, “Processing-in-Memory for Databases: Query Processing and Data Transfer,” in *Proceedings of the 19th International Workshop on Data Management on New Hardware*, in DaMoN '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 107–111. doi: 10.1145/3592980.3595323.
 - [142] M. Saed, P. J. Nair, and T. M. Aamodt, “RayN: Ray Tracing Acceleration with Near-memory Computing,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, in MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 277–291. doi: 10.1145/3725843.3756067.
 - [143] H. Shin, S. Bang, H. Park, and D. Kim, “SAFE: Sharing-Aware Prefetching for Efficient GPU Memory Management With Unified Virtual Memory,” *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 117–120, Jan. 2025, doi: 10.1109/LCA.2025.3553143.
 - [144] T. W. Kim, Y. Kwon, and J. Kang, “Scalable Address Spaces using Concurrent Interval Skiplist,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, in SOSP '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1131–1148. doi: 10.1145/3731569.3764807.
 - [145] A. Mamdouh, H. Geng, M. Niemier, X. Sharon Hu, and D. Reis, “Shared-PIM: Enabling Concurrent Computation and Data Flow for Faster Processing-in-DRAM,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 11, pp. 4395–4404, 2025, doi: 10.1109/TCAD.2025.3565466.
 - [146] M. Besta *et al.*, “SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, in MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 282–297. doi: 10.1145/3466752.3480133.

- [147] W. Xu, H. Choi, P.-K. Hsu, S. Yu, and T. Simunic, “SLIM: A Heterogeneous Accelerator for Edge Inference of Sparse Large Language Model via Adaptive Thresholding,” *ACM Trans. Embed. Comput. Syst.*, Jul. 2025, doi: 10.1145/3750727.
- [148] T. Kang, G. Choi, T. Suh, and G. Koo, “SparsePIM: An Efficient HBM-Based PIM Architecture for Sparse Matrix-Vector Multiplications,” in *Proceedings of the 39th ACM International Conference on Supercomputing*, in ICS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 495–512. doi: 10.1145/3721145.3735111.
- [149] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, “SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, in ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 950–965. doi: 10.1145/3620666.3651352.
- [150] S. Lee *et al.*, “SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs,” *Proc. ACM Manag. Data*, vol. 2, no. 6, Dec. 2024, doi: 10.1145/3698827.
- [151] B. Yoon, S. Han, G. Park, and J.-J. Kim, “SplitSync: Bank Group-Level Split-Synchronization for High-Performance DRAM PIM,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, Jun. 2025, pp. 1–7. doi: 10.1109/DAC63849.2025.11132821.
- [152] K. Gogineni *et al.*, “SwiftRL: Towards Efficient Reinforcement Learning on Real Processing-In-Memory Systems,” in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, May 2024, pp. 217–229. doi: 10.1109/ISPASS61541.2024.00029.
- [153] P. Bogdan *et al.*, “Taming extreme heterogeneity via machine learning based design of autonomous manycore systems,” in *Proceedings of the International Conference on Hardware/Software Codesign and System Synthesis Companion*, in CODES/ISSS '19. New York, NY, USA: Association for Computing Machinery, 2019. doi: 10.1145/3349567.3357376.
- [154] A. Devic, S. B. Rai, A. Sivasubramaniam, A. Akel, S. Eilert, and J. Eno, “To PIM or not for emerging general purpose processing in DDR memory systems,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, in ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 231–244. doi: 10.1145/3470496.3527431.
- [155] S. Lee, V. Rao, and A. Gavrilovska, “Towards Disaggregated NDP Architectures for Large-scale Graph Analytics,” in *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, in SC-W '24. Atlanta, GA, USA: IEEE Press, 2025, pp. 1622–1629. doi: 10.1109/SCW63240.2024.00202.
- [156] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, “Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures,” in *Perform Eval Rev*, Association for Computing Machinery, 2022, pp. 33–34. doi: 10.1145/3489048.3522661.
- [157] Y. Zhao *et al.*, “UM-PIM: DRAM-Based PIM with Uniform & Shared Memory Space,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, in ISCA '24. Buenos Aires, Argentina: IEEE Press, 2025, pp. 644–659. doi: 10.1109/ISCA59077.2024.00053.
- [158] A. Baumstark, L. Martins, and K.-U. Sattler, “Uncore your Queries: Towards CPU-less Query Processing,” in *Proceedings of the 21st International Workshop on Data Management on New Hardware*, in DaMoN '25. New York, NY, USA: Association for Computing Machinery, 2025. doi: 10.1145/3736227.3736243.

- [159] B. Friesel and O. Spinczyk, “Understanding Product Line Runtime Performance with Behaviour Models and Regression Model Trees,” in *Proceedings of the 29th ACM International Systems and Software Product Line Conference - Volume A*, in SPLC-A '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 142–148. doi: 10.1145/3744915.3748472.
- [160] N. Mavrogeorgis, C. Vasiladiotis, P. Mu, A. Khordadi, B. Franke, and A. Barbalace, “UNIFICO: Thread Migration in Heterogeneous-ISA CPUs without State Transformation,” in *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*, in CC 2024. New York, NY, USA: Association for Computing Machinery, 2024, pp. 86–99. doi: 10.1145/3640537.3641565.
- [161] S. Chen, A. C. Zhou, Y. Shi, Y. Li, and X. Yao, “UpANNS: Enhancing Billion-Scale ANNS Efficiency with Real-World PIM Architecture,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 789–804. doi: 10.1145/3712285.3759777.
- [162] D. Kim *et al.*, “Virtual PIM: Resource-aware Dynamic DPU Allocation and Workload Scheduling Framework for Multi-DPU PIM Architecture,” in *Proceedings of the 32nd International Conference on Parallel Architectures and Compilation Techniques*, in PACT '23. Vienna, AE, Austria: IEEE Press, 2024, pp. 112–123. doi: 10.1109/PACT58117.2023.00018.
- [163] D. Teguia, J. Chen, S. Bitchebe, O. Balmau, and A. Tchana, “vPIM: Processing-in-Memory Virtualization,” in *Middleware - Proc. ACM Int. Middlew. Conf.*, Association for Computing Machinery, Inc, 2024, pp. 417–430. doi: 10.1145/3652892.3700782.
- [164] G. Park *et al.*, “vSPACE: Supporting Parallel Network Packet Processing in Virtualized Environments through Dynamic Core Management,” in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, in PACT '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 14–25. doi: 10.1145/3656019.3689610.

12. Appendix A – PICOC, Search Strings, and Search Validation

12.1. Purpose of the appendix

This appendix documents the PICOC keyword groups, tier-specific search logic, database-level limits, search counts, and exact Boolean search strings used in the review. The main text summarizes the search strategy, while this appendix records the operational details needed for reproducibility.

The PICOC framework was used to define the conceptual scope of the review. The final database searches did not combine all PICOC fields into a single query, because doing so would have made the search too restrictive for a sparse and interdisciplinary topic. Instead, the search first tested the strict target intersection and then used tier-specific proxy searches. Each tier retained the concepts most important to its role in the review and relaxed one part of the strict target intersection.

During revision, the original search strings were also checked against cleaned or normalized versions. These validation searches were used to assess reproducibility, not to silently replace the executed search protocol. Where a normalized query produced a different record set, the difference was treated as a search-validation finding rather than as automatic evidence that the original search should be replaced.

12.2. PICOC keyword groups

The PICOC keyword groups in Table 5 define the conceptual search scope. They should be read as the controlled vocabulary from which the strict and tier-specific searches were derived. Not every

keyword appears in every search string, because the tiered strategy deliberately changes which part of the PICOC framework is emphasized.

PICOC element	Keyword concepts
Population	“quantum circuit simulation”, “quantum simulation”, “quantum circuit”, “tensor network”, “tensor-network”, “tensor contraction”, “tensor algebra”, “Einsum”, “matrix product state”, “projected entangled pair state”, “tensor train”, “state-vector simulation”, “QAOA”
Intervention	“processing in memory”, “processing-in-memory”, “near-data processing”, “near-memory processing”, “Processing-in-Memory”, “PIM”, “DRAM-PIM”, “UPMEM”, “PIM-DIMM”, “DPU”
Comparison	“CPU”, “GPU”, “distributed memory”, “MPI”, “cuTensorNet”, “cuQuantum”, “QuEST”, “state-vector simulator”, “state-vector simulation”
Outcome	“performance”, “speedup”, “throughput”, “energy efficiency”, “memory bandwidth”, “memory wall”, “data movement”, “communication cost”, “arithmetic intensity”, “memory footprint”, “memory capacity”, “intermediate tensor size”, “scalability”, “numerical accuracy”
Context	“digital PIM”, “digital Processing-in-Memory”, “UPMEM-style architecture”, “isolated DPU”, “host-DPU transfer”, “host-mediated communication”, “MRAM”, “WRAM”, “MRAM-WRAM movement”, “integer arithmetic”, “floating-point emulation”, “disjoint memory”

Table 5: PICOC keyword concepts used to define the conceptual search scope.

12.3. From PICOC to tiered searches

A strict target search was first used to test whether the direct research intersection already existed: tensor-network quantum circuit simulation using digital Processing-in-Memory. This strict target search returned no in-scope records in the selected databases. The review therefore proceeded with a tiered proxy strategy.

The tiered search design is summarized in Table 6. Tier 0 was used as a baseline gap test. Tiers 1–3 were used to retrieve adjacent evidence domains that could inform the review questions.

Tier	Purpose	Search logic
Tier 0	Strict target intersection	Tensor-network quantum-simulation terms combined with digital-PIM terms. This tested whether direct studies existed.
Tier 1	Quantum simulation on PIM	Quantum-simulation terms combined with Processing-in-Memory terms. This tier retained quantum simulation and PIM, but did not require a tensor-network representation.
Tier 2	Tensor-network algorithmic analogs	Tensor-network and tensor-contraction terms combined with contraction-planning and execution terms such as slicing, contraction path, partitioning, distributed memory, bandwidth, and communication cost. This tier retained tensor-network algorithmic evidence, but did not require PIM hardware.
Tier 3	UPMEM hardware proxy workloads	UPMEM or DRAM-PIM terms combined with selected proxy-workload and constraint terms. This tier retained digital-PIM hardware evidence, but did not require quantum simulation.

Table 6: Tier-specific mapping from PICOC concepts to database searches.

The comparison and outcome terms from the PICOC framework were not always included as mandatory Boolean clauses. They were used mainly to guide tier construction, screening, data extraction, and synthesis. This avoided over-constraining the searches while preserving traceability between the research questions and the search strategy.

12.4. Database-level limits and search log

Database-level filters were applied before deduplication where supported by IEEE Xplore, the ACM Digital Library, and Scopus. These filters were used to align the exported records with the review scope defined in the inclusion and exclusion criteria. The filters retained records published between January 1, 2016 and November 18, 2025, English-language records, and publication types corresponding to peer-reviewed conference papers, journal articles, or comparable scholarly research outputs. Because filtering options differ across databases, equivalent limits were applied as closely as each database interface allowed.

The formal tiered searches identified 316 raw records. After database-level limits were applied, 186 records remained for deduplication and screening. Counts in Table 7 describe search yield, not final inclusion in the review corpus.

Database	Tier	Raw hits	After limits
IEEE Xplore	1 Quantum simulation on PIM	0	0
IEEE Xplore	2 Algorithmic analogs	1	1
IEEE Xplore	3 Hardware proxies	31	31
ACM Digital Library	1 Quantum simulation on PIM	52	6
ACM Digital Library	2 Algorithmic analogs	77	27
ACM Digital Library	3 Hardware proxies	131	98
Scopus	1 Quantum simulation on PIM	1	1
Scopus	2 Algorithmic analogs	0	0
Scopus	3 Hardware proxies	23	22
Total		316	186

Table 7: Search log summary by database and evidence tier.

12.5. Search-string validation during revision

During revision, the original search strings were compared with normalized versions of the same searches. The normalized strings preserved the original search intent but cleaned Boolean formatting and, where appropriate, tested broader database search fields. The purpose of this validation was to check reproducibility at the level of record identity, not only at the level of hit counts.

Three categories of searches were distinguished:

Original executed search The exact search string used during the original database search.

Normalized validation search A cleaned version of the original string, used to check whether formatting or field restrictions changed the retrieved record set.

Expanded sensitivity search A broader search used only to test whether additional proxy terminology, such as GEMM, matrix multiplication, tensor kernels, host-DPU communication, collective communication, or data transfer, would retrieve additional potentially relevant records.

Normalized searches were not automatically treated as replacements for the original executed searches. If a normalized search produced the same in-scope record set, it was considered equivalent for reproducibility purposes. If it produced different records, the discrepancy was recorded as a search-validation finding. Expanded sensitivity searches were used only to assess possible search-string limitations; their results were not merged into the final corpus unless the full deduplication, screening, selection, quality appraisal, and data extraction process was repeated.

Because bibliographic databases change over time, validation runs may retrieve records that were not available or not in scope during the original search. Records published after November 18, 2025 were excluded even if they appeared in later validation searches.

12.6. Exact search strings

The following search strings were executed in IEEE Xplore, the ACM Digital Library, and Scopus. Because each database uses a different query syntax, the strings are reported separately. Where no field label is shown, the database's default search behavior was used.

12.6.1. Strict target search

The strict target search combined tensor-network quantum-simulation terms with Processing-in-Memory terms. It was intentionally restrictive because it tested the exact target intersection. It returned zero in-scope records across all three databases.

IEEE Xplore

Records returned: 0.

```
("All Metadata":("Abstract":"tensor network" OR "All Metadata":"Abstract":"tensor contraction" OR "All Metadata":"Abstract":"matrix product state" OR "All Metadata":"Abstract":"projected entangled pair state" OR "All Metadata":"Abstract":"tensor train")
AND
"All Metadata":"Abstract":quantum
AND
"All Metadata":("Abstract":"processing in memory" OR "All Metadata":"Abstract":"processing-in-memory" OR "All Metadata":"Abstract":"near-data processing" OR "All Metadata":"Abstract":UPMEM OR "All Metadata":"Abstract":"PIM-DIMM"))
```

ACM Digital Library

Records returned: 0.

```
Abstract:("tensor network" OR "tensor contraction" OR "matrix product state" OR "projected entangled pair state" OR "tensor train")
AND
Abstract:(quantum)
AND
Abstract:("processing in memory" OR "processing-in-memory" OR "near-data processing" OR "near-memory processing" OR UPMEM OR "PIM-DIMM")
```

Scopus

Records returned: 0.

```
TITLE-ABS-KEY(("tensor network" OR "tensor contraction" OR "matrix product state" OR "projected entangled pair state" OR "tensor train")
AND
quantum
AND
("processing in memory" OR "processing-in-memory" OR "near-data processing" OR "near-memory processing" OR UPMEM OR "PIM-DIMM"))
```

12.6.2. IEEE Xplore

Tier 1: Quantum simulation on PIM

Raw hits: 0. After limits: 0.

```
("Tensor Network" OR "Tensor Contraction" OR "Quantum Circuit Simulation" OR "Quantum Simulation" OR "QAOA")
AND
("Processing-in-Memory" OR "PIM" OR "Near-Data Processing" OR "Near-Memory Processing" OR "UPMEM" OR "DRAM-PIM")
```


Tier 2: Tensor-network algorithmic analogs

Raw hits: 1. After limits: 1.

```
("Tensor Network" OR "Tensor Contraction")
AND
("Slicing" OR "Bond Cutting" OR "Graph Partitioning" OR "Hypergraph Partitioning" OR
"Vertex Elimination" OR "Contraction Path")
AND
("Distributed Memory" OR "Memory Bound" OR "Bandwidth" OR "Communication Cost")
```

Tier 3: UPMEM hardware proxy workloads

Raw hits: 31. After limits: 31.

```
("UPMEM" OR "DRAM-PIM")
AND
("Sparse Matrix" OR "SpMV" OR "Graph Processing" OR "Irregular Applications" OR
"Benchmark" OR "Quantization" OR "Floating Point")
```

12.6.3. ACM Digital Library

Tier 1: Quantum simulation on PIM

Raw hits: 52. After limits: 6.

```
("Tensor Network" OR "Tensor Contraction" OR "Quantum Circuit Simulation" OR "Quantum
Simulation" OR "QAOA")
AND
("Processing-in-Memory" OR "PIM" OR "Near-Data Processing" OR "Near-Memory
Processing" OR "UPMEM" OR "DRAM-PIM")
```

Tier 2: Tensor-network algorithmic analogs

Raw hits: 77. After limits: 27.

```
("Tensor Network" OR "Tensor Contraction")
AND
("Slicing" OR "Bond Cutting" OR "Graph Partitioning" OR "Hypergraph Partitioning" OR
"Vertex Elimination" OR "Contraction Path")
AND
("Distributed Memory" OR "Memory Bound" OR "Bandwidth" OR "Communication Cost")
```

Tier 3: UPMEM hardware proxy workloads

Raw hits: 131. After limits: 98.

```
("UPMEM" OR "DRAM-PIM")
AND
("Sparse Matrix" OR "SpMV" OR "Graph Processing" OR "Irregular Applications" OR
"Benchmark" OR "Quantization" OR "Floating Point")
```

12.6.4. Scopus

Tier 1: Quantum simulation on PIM

Raw hits: 1. After limits: 1.

```
TITLE-ABS-KEY(("Tensor Network*" OR "Tensor Contraction" OR "Quantum Circuit
Simulat*" OR "Quantum Simulat*" OR "QAOA")
AND
("Processing-in-Memory" OR "PIM" OR "Near-Data Processing" OR "Near-Memory
Processing" OR "UPMEM" OR "DRAM-PIM"))
```

Tier 2: Tensor-network algorithmic analogs

Raw hits: 0. After limits: 0.

TITLE-ABS-KEY(("Tensor Network*" OR "Tensor Contraction")
AND
("Slicing" OR "Bond Cutting" OR "Graph Partitioning" OR "Hypergraph Partitioning" OR
"Vertex Elimination" OR "Contraction Path")
AND
("Distributed Memory" OR "Memory Bound" OR "Bandwidth" OR "Communication Cost"))

Tier 3: UPMEM hardware proxy workloads

Raw hits: 23. After limits: 22.

TITLE-ABS-KEY(("UPMEM" OR "DRAM-PIM")
AND
("Sparse Matrix" OR "SpMV" OR "Graph Processing" OR "Irregular Application*" OR
"Benchmark" OR "Quantization" OR "Floating Point"))

13. Appendix B – Screening Log

13.1. Purpose of the screening log

This appendix documents the title-and-abstract screening stage described in Section 5.1. After duplicate removal, 161 unique records were screened against the inclusion and exclusion criteria defined in Section 3.4 and Section 3.5. When titles and abstracts were insufficient to determine eligibility, the fallback procedure described in Section 3.7 was applied.

The purpose of this appendix is to record which records advanced to full-text eligibility assessment and which records were excluded during screening. Detailed full-text eligibility decisions are reported separately in Section 14.

13.2. Screening summary by tier

Evidence tier	Records screened	Advanced to full-text	Excluded during screening
Tier 1: Quantum simulation on PIM	6	2	4
Tier 2: Tensor-network algorithmic analogs	28	12	16
Tier 3: UPMEM hardware proxy workloads	127	23	104
Total	161	37	124

Table 8: Summary of title-and-abstract screening outcomes by evidence tier.

13.3. Records advanced to full-text assessment

The records below advanced from title-and-abstract screening to full-text eligibility assessment. At this stage, advancement means that the record appeared potentially relevant based on title, abstract, and fallback inspection where needed. Final inclusion was determined during full-text eligibility assessment.

13.3.1. Tier 1: Quantum simulation on PIM

Record	Screening rationale
PIMutation [9]	Directly addresses state-vector quantum circuit simulation on UPMEM hardware. Advanced because it provides direct evidence for quantum simulation on digital PIM.
Symphony [30]	Addresses sparse and dense tensor workloads using near-memory or PIM-oriented execution concepts. Advanced as an architectural proxy identified during the core search.

13.3.2. Tier 2: Tensor-network algorithmic analogs

Record	Screening rationale
AutoHOOT [13]	Address the Constrained Contraction Path Construction explicitly. Passes IC-Tier2. Addresses the constrained contraction path construction explicitly, which provides the mathematical foundation needed before porting to PIM.
Closing the “Quantum Supremacy” Gap [10]	Address QC Simulation and TN. IC1,3 are good. Passes IC-Tier2. Provides a highly relevant near-optimal slicing scheme and path-optimization strategy for tensor-based quantum circuit simulation, which can be adapted for PIM constraints.
DISTAL [17]	Passes IC-Tier2. Address TN with hardware averseness, which can be adapted for PIM.
Efficient Quantum Circuit Simulation by Tensor Network Methods on Modern GPUs [12]	Passes IC-Tier2. Perfect paper on TN tailored for GPU - use this to adapt for PIM.
gHyPart [31]	Passes IC-Tier2. Provides a modern algorithmic approach to hypergraph partitioning. Since hypergraph partitioning is a core technique for slicing tensor networks, understanding its memory mapping on GPUs provides a critical mathematical foundation for adapting these heuristics to PIM.
Lifetime-Based Optimization for Simulating Quantum [11]	Passes IC-Tier2. Precisely on the topic “circuit simulator tensor network contraction”
Minimum Cost Loop Nests for Contraction of a Sparse Tensor with a Tensor Network [15]	Passes IC-Tier2. Provides algorithmic theory for optimizing loop nests in tensor network contractions. This loop-level optimization is critical for controlling data movement and memory access patterns, which can be adapted for the strict WRAM constraints of PIM DPUs.
Simulation of Quantum Computers: Review and Acceleration Opportunities [3]	Passes IC-Tier2. Comprehensive survey that analyzes state-of-the-art tensor slicing, contraction ordering, and data format optimizations on CPUs and GPUs, providing critical baselines for the memory wall in quantum simulations. Overview of simulation methods for quantum computing.

Record	Screening rationale
Surpassing Sycamore [5]	Passes IC-Tier2. Precise efficient quantum simulation using SOTA techniques.
SW_Qsim [14]	Passes IC-Tier2. Simulation of quantum computer using tensors methods. Proposes a ‘minimize-memory’ quantum simulation strategy. The algorithms used to reduce memory footprints and optimize distributed communication offer excellent mathematical foundations for memory-constrained PIM environments.
Tensor Network Quantum Virtual Machine [32]	Passes IC-Tier2. TN for QC. Explores Tensor Network contraction across massive distributed-memory systems. The algorithmic strategies for slicing and distributing tasks across isolated memory nodes are highly relevant for PIM WRAM constraints.
The tensor algebra compiler (TACO) [16]	Passes IC-Tier2. The foundational paper for automatically generating efficient code for sparse and dense tensor contractions; provides the critical data layout and compiler abstractions needed before porting tensor math to PIM.

13.3.3. Tier 3: UPMEM hardware proxy workloads

Record	Screening rationale
A Full-System Perspective on UPMEM Performance [33]	Passes IC3 - overview on UPMEM “We selected all UPMEM-suitable workloads as well as GEMV , SpMV , and TS for our extension. With these, we examine how addressing the challenges mentioned in the previous section affects the speedup of off loading via UP-MEM PIM over a CPU baseline. Source code , data, and anal-ysis scripts are available online at https://ess.cs.uos.de/git/artifacts/dimes23-upmem-performance.4 “
A General Offloading Approach for Near-DRAM Processing-In-Memory Architectures [34]	Passes IC-Tier3. Provides a general-purpose instruction-level offloading technique for near-DRAM PIM. The analysis of computation locality and host-to-PIM offloading is highly relevant for designing the software execution model for UPMEM DPUs.
Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices [24]	Passes IC-Tier3. While the application domain is LLMs, the paper survives EC2 because it specifically develops an efficient GEMM (General Matrix Multiply) library for real-world PIM. Since GEMM is the foundational building block for tensor network contractions, this provides the exact architectural proxy needed to map quantum tensor algebra to PIM.
AESPA: Asynchronous Execution Scheme to Exploit	Passes IC-Tier3. Proposes an asynchronous execution scheme to maximize Bank-Level Parallelism (BLP) in PIM

Record	Screening rationale
Bank-Level Parallelism of Processing-in-Memory [35]	architectures. Understanding how to manage asynchronous task offloading and memory bank concurrency provides a highly relevant architectural blueprint for optimizing tensor network contractions across isolated UPMEM DPUs. UPMEM + GEMV
ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System [20]	Passes IC-Tier3. Evaluates linear algebraic graph processing (fundamentally driven by SpMV) on real PIM hardware. Because graph processing exhibits the same highly irregular, memory-bound access patterns as sliced tensor network contractions, this paper provides a direct architectural proxy for managing data layouts and bandwidth on isolated DPUs.
An Efficient PIM-Based Graph Engine on a Single Machine [29]	Passes IC-Tier3. Proposes a high-performance graph engine leveraging PIM architecture. Since graph processing shares the highly irregular, memory-bound access patterns of tensor network contractions, this paper provides critical architectural proxies for managing data layouts and bandwidth optimization across isolated DPUs.
Analysis of Data Transfer Bottlenecks in Commercial PIM Systems: A Study With UPMEM-PIM [18]	Passes IC-Tier3. Explicitly targets real-world UPMEM architecture. By detailing the physical and software bottlenecks of Host-to-DPU data transfers (including memory mapping and bank access constraints), this paper provides the exact architectural blueprint required to optimize the transfer of sliced tensor networks to isolated PIM nodes.
Architectural Modeling and Benchmarking for Digital DRAM PIM [36]	Passes IC-Tier3. Provides comprehensive architectural modeling and general benchmarking for Digital DRAM PIM. This serves as a foundational hardware proxy for understanding execution cycles, memory bandwidth utilization, and physical DPU constraints, which are necessary for modeling tensor contraction performance.
ATiM: Autotuning Tensor Programs for Processing-in-DRAM [19]	Passes IC-Tier3. While motivated by ML domains, it explicitly bypasses EC2 because it develops an autotuning compiler framework for general tensor programs on PIM. This provides a direct, highly optimized software blueprint for mapping, tiling, and executing tensor algebraic operations (the core of quantum tensor networks) within the constrained memory hierarchy of PIM DPUs.
Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System [6]	Passes IC-Tier3. This is the foundational paper characterizing real-world UPMEM architecture. It introduces the PrIM benchmark suite (including SpMV and GEMV), providing the definitive architectural proxy measurements for WRAM bandwidth, instruction throughput, and Host-DPU data transfer bottlenecks necessary to model tensor network execution.

Record	Screening rationale
Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-In-Memory Hardware [37]	Passes IC-Tier3. Directly analyzes real-world PIM hardware limits. Serves as a critical architectural baseline for understanding the physical execution cycles and memory latency walls that will dictate the performance of offloaded tensor math.
CINM (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms [38]	Passes IC-Tier3. While it addresses both analog and digital paradigms, it explicitly models and compiles down to real-world UPMEM architecture (Compute Near-Memory). By providing a compilation infrastructure and Intermediate Representation (IR) for UPMEM, this paper offers a critical software-engineering proxy. It details how generalized computational loops and memory transfers can be optimally scheduled and lowered to fit the strict WRAM constraints of isolated DPUs, directly informing the implementation of tensor network code.
ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration [39]	Passes IC-Tier3. Addresses the critical Host-to-DPU communication bottleneck. By proposing a dedicated protocol and scheduling architecture for CPU-PIM collaboration, this paper provides a direct blueprint for how to efficiently orchestrate and dispatch tensor network slices to isolated PIM nodes without stalling the host.
CoPIM: A Collaborative Scheduling Framework for Commodity Processing-in-memory Systems [40]	Passes IC-Tier3. Explicitly targets UPMEM systems and addresses the critical challenge of CPU-PIM collaborative scheduling. By using computational graphs and performance modeling to minimize Host-to-DPU communication overhead, it provides a highly relevant architectural proxy for distributing and scheduling sliced tensor network workloads.
Pac-PIM: A Parallel Communication Framework for Commodity Processing-in-memory Systems [23]	Passes IC-Tier3. Explicitly targets commodity UPMEM hardware and addresses the critical host-to-DPU communication bottleneck. By combining rank-level and process-level parallelism for data transfers, it provides a direct, actionable software framework for efficiently orchestrating and dispatching tensor network slices to isolated DPUs.
Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology [41]	Passes IC-Tier3. While its primary output is a software simulator (uPIMulator), the paper survives the simulation trap because it explicitly reverse-engineers and demystifies existing UPMEM hardware. It provides a cycle-accurate architectural breakdown of physical DPU pipeline depths, instruction latencies, and memory bottlenecks, offering a critical low-level blueprint for optimizing bare-metal tensor C-code.

Record	Screening rationale
PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices [22]	Passes IC-Tier3. Explicitly targets commodity UPMEM hardware. Because UPMEM DPUs are physically isolated, distributed tensor contractions require massive host-relayed synchronization. This paper provides an optimized collective communication API (specifically Broadcast for shared tensor slices, and AllReduce for accumulating partial sums), offering a critical software blueprint for multi-DPU tensor algebra.
PRISM: Processing-In-Memory Sparse MTTKRP for Tensor Decomposition Acceleration [27]	Passes IC-Tier3. Explicitly targets UPMEM architecture to accelerate Sparse MTTKRP (Matricized Tensor Times Khatri-Rao Product). Because MTTKRP is a foundational multidimensional tensor contraction, this paper provides a direct, highly optimized software blueprint for partitioning, mapping, and computing sparse tensor algebra across isolated DPUs.
PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures [26]	Passes IC-Tier3. While targeting Graph Neural Networks (GNNs), the full text confirms it survives the AI trap (EC2) because the core contribution is a highly optimized bare-metal library for Sparse Matrix-Matrix Multiplication (SpMM) and Dense Matrix Multiplication (GEMM) on UPMEM DPUs. It provides an actionable, hardware-validated C-code blueprint for load-balancing irregular sparse data, WRAM tiling, and Host-to-DPU communication, all of which are mandatory for distributing quantum tensor contractions.
Scalability Limitations of Processing-in-Memory using Real System Evaluations [42]	Passes IC-Tier3. Evaluates real-world UPMEM hardware to empirically characterize the bandwidth saturation walls of Host-centric collective communication. Because UPMEM DPUs are physically isolated, distributed tensor contractions require massive Host-relayed Broadcast (dispatching operators) and Reduce (accumulating partial sums) operations. This paper provides the critical physical scaling laws and topology limits required to properly size tensor slices and prevent memory-bus starvation when scaling the simulator to thousands of DPUs.
SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory [28]	Passes IC-Tier3. Explicitly targets real-world UPMEM hardware and provides an optimized software framework/API for managing arrays of arbitrary elements. Crucially, it abstracts and optimizes the primitives for Host-to-DPU and Inter-DPU communication, offering an essential systems engineering blueprint for orchestrating distributed quantum tensor slices without relying on raw, unoptimized UPMEM SDK boilerplate.
SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real	Passes IC-Tier3. Explicitly targets real-world UPMEM hardware and provides the foundational, comprehensive software analysis for SpMV (Sparse Matrix-Vector

Record	Screening rationale
Processing-In-Memory Architectures [25]	Multiplication). Because SpMV is a core computational primitive for sparse tensor network contractions, this paper offers a highly actionable, bare-metal C-code blueprint for mapping sparse data structures (e.g., CSR, COO), balancing loads across isolated DPUs, and managing WRAM constraints.
TransPimLib: Efficient Transcendental Functions for Processing-in-Memory Systems [21]	Passes IC-Tier3. Explicitly targets real-world constrained PIM hardware. Quantum circuit simulation relies heavily on complex amplitudes, phase shifts, and rotation gates, necessitating extensive use of transcendental functions (sine, cosine, exponentiation). Because UPMEM DPUs possess a simplified RISC ALU lacking native hardware support for these operations, standard library calls result in severe software-emulation cycle penalties. This paper provides the critical architectural blueprint for bypassing this bottleneck, offering optimized, bare-metal CORDIC (shift-and-add) and WRAM-aware Look-Up Table (LUT) C-code implementations to execute quantum gate mathematics efficiently on isolated DPUs.

13.4. Records excluded during screening

The records listed below were excluded during title-and-abstract screening. The failed criteria column reports the main criterion or criteria responsible for exclusion. When several criteria applied, only the most relevant criteria are listed.

13.4.1. Tier 1: Quantum simulation on PIM

Record	Failed criteria	Reason for exclusion
A Survey on Deep Learning [43]	EC2	Focuses on deep-learning hardware accelerators rather than quantum simulation, tensor-network contraction, or transferable PIM proxy workloads.
Strassen's Algorithm Reloaded on GPUs [44]	IC2 / EC4	Focuses on GPU execution and relies on shared-memory hierarchy assumptions that are not transferable to isolated UPMEM DPUs.
Matrix: Multi-Cipher Structures Dataflow [45]	IC1	Focuses on fully homomorphic encryption acceleration rather than quantum circuit simulation, tensor networks, or general PIM proxy workloads.
DSLs, DLA, DxT, and MDE in CSE [46]	IC1 / IC4	Focuses on software-engineering methodology for dense linear algebra and falls outside the publication window used in the review.

13.4.2. Tier 2: Tensor-network algorithmic analogs

Record	Failed criteria	Reason for exclusion
A Block-Oriented Language [47]	IC1, IC4	Quantum chemistry, 2010 year. Is on tensors but not relevant enough.
A shared compilation [48]	IC1,3	Focuses on stencil computations (PDE solvers) for standard HPC clusters; irrelevant to Tensor Networks and PIM architectures.
Accelerated Work Stealing [49]	IC1,3	Work scheduling isn't in the scope of my thesis.
Automatic Code Generation [50]	IC1	Focuses on Finite Element Methods (Galerkin) and SIMD vectorization for standard architectures; irrelevant to Tensor Networks and PIM.
Automatic creation [51]	IC1	Focuses on computational fluid dynamics(CFD) for standard architectures; irrelevant to IC1
Finite Element Integration on GPUs [52]	IC1	"Finite element method (FEM) is a popular method for numerically solving differential equations arising in engineering and mathematical modeling." not applicable in my thesis e.g. EC1
Hypergraph partitioning for automatic memory hierarchy management [53]	IC4	Published in 2006 (fails >2016 date criteria). Excluded from SLR data extraction, but MAY BE retained for historical context in the Background chapter.
Merchandiser [54]	EC1	It is about persistent memory - "We use Intel Optane Per-sistent Memory (PM) and DRAM as an example of HM in our study."
Multilevel Combinatorial Optimization across Quantum Architectures [55]	IC1 / IC2	Focuses on using quantum hardware (D-Wave/ IBM Q) to solve classical graph partitioning problems. It does not address simulating quantum circuits/tensor networks, nor does it target PIM architectures.
Optimizing Data Distribution [56]	EC2	Neural networks - excluded by EC2, not on quantum or general tensor algebra
Realizing Quantum Kernel Models at Scale with Matrix Product State Simulation [57]	IC3	Focuses on benchmarking a quantum machine learning application using an existing MPS simulator on CPUs/GPUs. Lacks novel algorithmic proposals for tensor slicing or partitioning required for Tier 2. Good for background?
Scheduling Language Chronology [58]	IC1	Focuses on parallel Domain Specific Language - complete miss on quantum computing.

Record	Failed criteria	Reason for exclusion
SeisSo [59]	IC1	Modal Discontinuous Galerkin Method (irrelevant to tensor networks)
SnuQS [60]	IC3	Focuses on out-of-core SSD storage management for state-vector simulations. Lacks both PIM architecture targeting and tensor network contraction/slicing algorithms.
Supporting Very Large [61]	EC2	Neural networks, DNN, CNN, RNN
Tensor processing primitives: a programming abstraction for efficiency... in deep learning [62]	EC2 / IC1	Focuses on optimizing 2D tensor primitives specifically for Deep Learning (DNN) workloads on standard CPUs; irrelevant to Quantum Circuits and PIM.

13.4.3. Tier 3: UPMEM hardware proxy workloads

Record	Failed criteria	Reason for exclusion
3D-PLANE: A 3D-stacked DRAM-based Programmable SLM Accelerator... [63]	EC2	Passes IC2 (3D-stacked DRAM PIM), but triggers EC2 because it exclusively targets Small Language Models (Transformers) rather than general sparse tensor or graph workloads.
3D-ReG [64]	EC1,2	Paper on Deep Neural Networks (EC2) on ReRAM (EC1)
A CNN-Based Super-Resolution Processor [65]	EC1,2	Focused on Neural networks (EC2) on proposed SR processor not PIM (EC1)
A design analysis of a hybrid technology multithreaded architecture [66]	EC5, IC4	published: 01 May 1999; there is a section “4.3 PIM Main Memory”
A fast diagnosis scheme for distributed small embedded SRAMs [67]	EC5, IC4, EC1	published: 21 March 2005; Work on embedded SRAMs
A Heterogeneous Chiplet Architecture for Accelerating End-to-End Transformer Models [68]	EC2	Focused on ML, DL, Transformers
A Low Power In-DRAM Architecture for Quantized CNNs using Fast Winograd Convolutions [69]	EC2	Neural Networks, Quantized CNN
A Novel DRAM-Based Process-in-Memory	EC2	CNN. “In this paper, we present for the first time a novel DRAM-based PIM architecture for multi-bit-

Record	Failed criteria	Reason for exclusion
Architecture and its Implementation for CNNs [70]		precision CNN inference that functions both as a CNN accelerator and as a high-density host memory. Our neural network inference engine relies on the one-hot and two-hot quantization of neural network weights that transforms multiplication operations into shift operations.”
A Survey on Memory-centric Computer Architectures [71]	EC1	Broad tutorial that heavily covers memristors, resistive computing, and emerging non-volatile memory (NVM) technologies. Excluded as it focuses on analog/NVM hardware rather than digital PIM workload mapping.
Accelerating Large Table Scan Using Processing-In-Memory Technology [72]	EC2	This paper accelerates a Database Table Scan. A table scan is a highly regular, predictable, sequential memory access pattern. It behaves completely differently in hardware than tensor contractions. Therefore, the algorithms they use to map data to the DPU will be useless for quantum simulator.
Accelerating Neural Network Inference With Processing-in-DRAM: From the Edge to the Cloud @oliveira_accelerating_2022	EC2	Evaluates PIM architectures (including UPMEM), but focuses exclusively on Neural Network (NN) inference. Excluded as it lacks the general sparse matrix or tensor algebra workloads required for a quantum simulation proxy.
Accelerating Regular Path Queries over Graph Database with Processing-in-Memory [73]	EC2	Work on Database without general algebra in any kind.
Accelerating RNA-Seq Quantification on a Real Processing-in-Memory System [74]	EC2	Not on Neural Networks but RNA sequencing also not applicable to general algebra or quantum simulations etc.
Adaptive Query Compilation with Processing-in-Memory [75]	EC2	Query not general algebra, nothing useful for quantum simulations.
Algorithm-hardware co-design for Energy-Efficient A/D conversion in ReRAM-based accelerators [76]	EC1,2	Focuses on analog ReRAM for Deep Neural Network, “ReRAM-based accelerator shows potential in accelerating DNN”
Analyzing and Enhancing ArckFS: An Anecdotal	EC1, IC2 / Workload Mismatch	Focuses on Non-Volatile Memory (NVM) file system architectures and multi-inode operations. Excluded because it does not target Digital PIM

Record	Failed criteria	Reason for exclusion
Example of Benefits of Artifact Evaluation [77]		compute (fails IC2) and lacks a computational proxy workload relevant to tensor networks.
Automated Design of Chiplets [78]	EC2, IC2 / Workload Mismatch	Focuses on Electronic Design Automation (EDA) and manufacturing methodologies for multi-die chiplets. Excluded because it does not target PIM architectures and lacks computational proxy workloads relevant to tensor algebra.
BL-PIM: Varying the Burst Length to Realize the All-Bank Performance and Minimize the Multi-Workload Interference for in-DRAM PIM [79]	EC2	Focuses on DRAM burst length optimizations but heavily tailors its motivation, evaluation, and framework to Deep Neural Networks (DNN/CNN). While it includes minor general matrix multiplication concepts, it lacks actionable architectural proxies for irregular sparse tensor networks. Excluded under EC2 to maintain a strict boundary against AI-specific hardware profiling.
Buffer Size Impact on End-to-End DNN Acceleration in Near-Bank DRAM-PIM Systems [80]	EC2	Focuses exclusively on Deep Neural Network (DNN) acceleration and hardware tuning for specific activation functions (ReLU/GELU). Excluded because it lacks general tensor algebra or irregular proxy workloads.
Bulk JPEG decoding on in-memory processors [81]	EC2 / Workload Mismatch	Executes on PIM, but accelerates JPEG decoding. Excluded because image decoding relies on highly regular, predictable, and block-based operations, making it an invalid proxy for the irregular memory patterns of tensor networks.
Construction of Realistic Place-and-Route Benchmarks for Machine Learning Applications [82]	EC1, IC2 / Workload Mismatch	Focuses on Electronic Design Automation (EDA) and generating place-and-route (P&R) layouts for chip manufacturing. Excluded because it does not target PIM architecture and lacks a computational proxy workload relevant to tensor networks.
CORUSCANT: Fast Efficient Processing-in-Racetrack Memories [83]	EC1 / IC2	Proposes a processing-in-memory architecture based on Spintronic Domain-Wall (Racetrack) Memory. Excluded because it relies on emerging magnetic memory paradigms rather than commodity digital DRAM PIM (UPMEM), failing to provide a compatible architectural proxy.
Cost-Effective Extension of DRAM-PIM for Group-Wise LLM Quantization [84]	EC2	Targets DRAM-PIM but proposes custom hardware extensions (INT2FP converters) specifically for low-precision Large Language Model (LLM) weight quantization. Excluded under the AI Trap (EC2) because low-bit quantization hardware provides no actionable proxy for the

Record	Failed criteria	Reason for exclusion
		high-precision floating-point requirements of quantum tensor networks.
DA-VinCi: A Deep-Learning Accelerator Overlay Using In-Memory Computing [85]	EC2	Focused on Neural Networks, not applicable to tensor contractions.
Design and Analysis of a Processing-in-DIMM Join Algorithm: A Case Study with UPMEM DIMMs [86]	EC2, IC2 / Workload Mismatch	Accelerates Database Hash Joins on UPMEM. Excluded because the underlying data structures (hash tables, SQL tuples) are too distant from the multidimensional floating-point arrays required for quantum tensor algebra, offering no practical code proxy.
Design space for scaling-in general purpose computing within the DDR DRAM hierarchy for map-reduce workloads [87]	IC2 / Hardware Simulation	Focuses on theoretical design space exploration for future DRAM hierarchies. Excluded because it relies on hardware simulation rather than providing actionable software/scheduling blueprints for existing commodity UPMEM hardware.
DRIM-ANN: An Approximate Nearest Neighbor Search Engine based on Commercial DRAM-PIMs [88]	EC2 / Workload Mismatch	Targets UPMEM but focuses on Approximate Nearest Neighbor (ANN) search indexes. Excluded because search-based memory management provides no actionable software or architectural proxy for deterministic tensor network contractions.
Efficient and Robust Edge AI: Software, Hardware, and the Co-design [89]	EC1 / EC2	Focuses on Edge AI and emerging resistive memory (VRRAM). Excluded because it relies on non-commodity analog memory and focuses entirely on Neural Architecture Search rather than generalized tensor algebra proxies.
Enhanced JPEG Decoding using PIM Architectures with Parallel MCU Processing [90]	EC2 / Workload Mismatch	Targets UPMEM but accelerates JPEG decoding. Excluded because image block decoding relies on highly regular, predictable operations (like DCT), making it an invalid architectural proxy for the sparse/irregular memory patterns of tensor networks.
EOD: Enabling Low Latency GNN Inference via Near-Memory Concatenate Aggregation [91]	EC2 / IC2	Proposes a custom near-memory hardware architecture designed specifically for Graph Neural Network (GNN) aggregation. Excluded because it relies on custom AI hardware rather than commodity UPMEM DPUs, providing no actionable software proxies for general tensor algebra.

Record	Failed criteria	Reason for exclusion
Evaluating Machine Learning Workloads on Memory-Centric Computing Systems [92]	EC2 / Workload Mismatch	Evaluates commercial UPMEM hardware but focuses entirely on classic ML workloads (Linear/ Logistic Regression, Decision Trees, K-Means). Excluded because these workloads do not serve as a valid mathematical or architectural proxy for the heavy sparse/dense matrix multiplications required in tensor network contractions.
Expediting Hazard Pointers with Bounded RCU Critical Sections [93]	EC4 / Hardware Trap	Focuses on Hazard Pointers and RCU synchronization. Excluded because these primitives rely heavily on cache-coherence and shared memory, which are fundamentally incompatible with the isolated, non-coherent architecture of UPMEM DPUs.
FALA: Locality-Aware PIM-Host Cooperation for Graph Processing with Fine-Grained Column Access [94]	IC2 / Hardware Modification	While it evaluates graph processing, it relies on proposing custom microarchitectural modifications to DRAM column access logic. Excluded because this cannot be implemented via software on locked commodity UPMEM hardware.
Fast, Transparent and Accurate Simulation of Thousand Processing-in-Memory Cores [95]	IC2 / Simulation Trap	Focuses on developing a software simulator for massive-scale PIM systems. Excluded because it does not provide algorithmic proxies or physical hardware benchmarking for actual UPMEM code implementation.
FlutPIM: A Look-up Table-based Processing in Memory Architecture with Floating-point Computation Support for Deep Learning Applications[96]	EC2 / IC2	Focuses entirely on accelerating Deep Learning workloads. Additionally, proposes a custom Look-up Table (LUT) hardware architecture rather than utilizing commodity off-the-shelf UPMEM DPUs.
GP-SIMD Processing-in-Memory [97]	EC5 / IC2	Published before 2016. Furthermore, proposes a theoretical custom SIMD architecture to replace the CPU Last Level Cache (LLC), providing no actionable software proxy for commodity UPMEM hardware.
Gray-in-Young: A Generational Garbage Collection for Processing-in-Memory [98]	EC2 / Workload Mismatch	Targets UPMEM but focuses on designing a generational garbage collector. Excluded because high-performance bare-metal tensor computation relies on explicit memory management, rendering automated garbage collection frameworks irrelevant.
Hardware-Software Co-Design of a Collaborative	EC2 / IC2	Proposes custom hardware design for a Deep Neural Network (DNN) accelerator using HBM3.

Record	Failed criteria	Reason for exclusion
DNN Accelerator for 3D Stacked Memories with Multi-Channel Data [99]		Excluded because it falls into the AI trap and relies on custom non-commodity hardware, offering no software blueprints for UPMEM.
HeterRAG: Heterogeneous Processing-in-Memory Acceleration for Retrieval-augmented Generation [100]	EC2 / Workload Mismatch	Accelerates Retrieval-Augmented Generation (RAG) for LLMs using vector similarity search. Excluded because search-based AI workloads do not serve as a deterministic proxy for sparse/dense tensor network contractions.
HetSim: A Simulator for Task-based Scheduling on Heterogeneous Hardware[101]	EC2, IC2 / Simulation Trap	Proposes a software simulator for heterogeneous task scheduling. Excluded under the rigorous timeline filter because it lacks actionable physical UPMEM benchmarks or concrete code proxies.
HybriDS: Cache-Conscious Concurrent Data Structures for Near-Memory Processing Architectures [102]	EC4 / Workload Mismatch	Focuses on cache-conscious database data structures (B-trees/Hash tables). Excluded because it relies on cache-coherence and shared memory, offering no proxy for the manually managed scratchpad (WRAM) architecture of UPMEM DPUs.
HyperMR: Efficient Hypergraph-enhanced Matrix Storage on Compute-in-Memory Architecture [103]	EC1 / Workload Mismatch	Focuses on matrix storage formats for data warehouses using Compute-in-Memory (typically analog/mixed-signal crossbars). Excluded because it does not target commodity digital DRAM-PIM (UPMEM).
Implementation and Evaluation of Deep Neural Networks in Commercially Available Processing in Memory Hardware [104]	EC2	Targets UPMEM but focuses exclusively on mapping Convolutional Neural Networks (CNNs) and Binarized Neural Networks. Excluded under EC2 because sliding-window convolutions offer less value than generalized GEMM proxies for quantum circuits.
Improving In-Memory Database Operations with Acceleration DIMM (AxDIMM) [105]	EC2 / Workload Mismatch	Evaluates database operations on AxDIMM. Excluded under the strict timeline filter because relational database workloads do not serve as a valid proxy for multi-dimensional tensor contractions.
Improving Timing & Power Trade-off in Post-place Optimization Using Multi-agent Reinforcement Learning [106]	EC2 / Domain Mismatch	Focuses on Electronic Design Automation (EDA) and chip manufacturing layouts using Reinforcement Learning. Excluded because it does not target PIM architecture or software implementation.

Record	Failed criteria	Reason for exclusion
Ironman: Accelerating Oblivious Transfer Extension for Privacy-Preserving AI with Near-Memory Processing [107]	EC2 / Workload Mismatch	Focuses on accelerating cryptographic protocols (Oblivious Transfer) for privacy-preserving AI. Excluded because cryptographic bitwise logic provides no actionable proxy for high-precision quantum tensor algebra.
Lightning Talk: Memory-Centric Computing [108]	EC5 / Format Trap	This is a 1-page extended abstract for a lightning talk. Excluded because it provides only a high-level overview of the memory wall, lacking the actionable architectural blueprints or mathematical proxies required for implementation.
Mainframe-Style Channel Controllers for Modern Disaggregated Memory Systems [109]	IC2	Focuses on CXL and Disaggregated Memory pools for datacenters. Excluded because network-attached memory pooling fundamentally differs from the local, non-coherent Processing-in-Memory architecture of UPMEM DPUs.
MAPLE: Flexible-Precision Processing-In-Memory Architecture for Efficient On-Device ML [110]	EC2	Focuses entirely on flexible-precision quantization for on-device Machine Learning. Excluded under the irrelevant workload trap (EC2) because low-bit AI quantization provides no valid proxy for high-precision quantum tensor algebra.
MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm [111]	EC2	Focuses on bioinformatics and DNA seeding algorithms. Excluded under the irrelevant workload trap (EC2) because string-matching genomic algorithms do not serve as a deterministic proxy for multidimensional tensor contractions.
Microservers: a new memory semantics for massively parallel computing [112]	EC5	Published in 1999. Excluded under the date trap (EC5) because it predates modern commodity digital PIM hardware and offers no actionable blueprints for current architectures.
Modular Hardware Design of Pipelined Circuits with Hazards [113]	EC2	Focuses on Electronic Design Automation (EDA) and hardware description languages for circuit synthesis. Excluded under the irrelevant domain trap (EC2) because it involves chip manufacturing design rather than PIM software implementation.
MPU: Memory-centric SIMT Processor via In-DRAM Near-bank Computing [114]	IC2	Proposes a custom SIMT processor design embedded within DRAM. Excluded because it relies on theoretical hardware modifications rather than providing actionable software/scheduling proxies for existing commodity UPMEM hardware.

Record	Failed criteria	Reason for exclusion
Multi-Parameter Comparative Study of Selective In-Memory Computing Processor Architectures for Performance Optimization in Accelerated Edge Computing [115]	EC2	Focuses on comparative architecture studies for Edge AI computing. Excluded under the irrelevant workload trap (EC2) because low-power Edge AI provides no valid proxy for high-precision quantum tensor algebra on UPMEM.
NDPBridge: Enabling Cross-Bank Coordination in Near-DRAM-Bank Processing Architectures [116]	IC2	Proposes physical hardware modifications (hardware bridges) to enable cross-bank communication. Excluded because these architectural changes cannot be implemented via software on locked commodity UPMEM hardware.
NEM-GNN: DAC/ADC-less, Scalable, Reconfigurable, Graph and Sparsity-Aware Near-Memory Accelerator for Graph Neural Networks [117]	EC2	Proposes a custom hardware accelerator specifically for Graph Neural Networks (GNNs). Excluded under the irrelevant workload trap (EC2) because custom AI hardware provides no actionable C-code proxies for commodity UPMEM.
NMExplorer: An Efficient Exploration Framework for DIMM-based Near-Memory Tensor Reduction [118]	IC2	Proposes a software exploration framework to simulate and evaluate theoretical DIMM-NMP hardware configurations. Excluded because it relies on hardware simulation rather than providing actionable software/scheduling blueprints for existing commodity UPMEM hardware.
NMP-PaK: Near-Memory Processing Acceleration of Scalable De Novo Genome Assembly [119]	EC2	Focuses on bioinformatics and de novo genome assembly. Excluded under the irrelevant workload trap (EC2) because genomic sequence alignment algorithms do not serve as a valid deterministic proxy for multidimensional tensor contractions.
No Cap, This Memory Slaps: Breaking through the Memory Wall of Transactional Database Systems with Processing-in-Memory [120]	EC2	Focuses on transactional database systems. Excluded under the irrelevant workload trap (EC2) because database transaction management is fundamentally incompatible with the multidimensional floating-point requirements of tensor networks.
Optimal Batch-Dynamic kd-trees for Processing-in-Memory with Applications [121]	EC2	Focuses on kd-trees for spatial queries and search algorithms. Excluded under the irrelevant workload trap (EC2) because search-based tree structures offer no valid architectural proxy for deterministic dense/sparse matrix math.

Record	Failed criteria	Reason for exclusion
Optimization of DRAM based PIM Architecture for Energy-Efficient Deep Neural Network Training [122]	EC2	Focuses entirely on energy-efficient Deep Neural Network (DNN) training architectures. Excluded under the irrelevant workload trap (EC2) because AI training optimizations do not translate to general quantum tensor contractions.
PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System [123]	EC2	Focuses on Large Language Model (LLM) decoding and speculative execution. Excluded under the irrelevant workload trap (EC2) because generative text generation provides no actionable deterministic proxy for quantum tensor algebra.
Parallelization of the Banded Needleman & Wunsch Algorithm on UPMEM PIM Architecture for Long DNA Sequence Alignment [124]	EC2	Targets UPMEM but focuses on bioinformatics and DNA sequence alignment. Excluded under the irrelevant workload trap (EC2) because dynamic programming string-matching algorithms do not map to multidimensional tensor contractions.
Performance Models for Task-based Scheduling with Disruptive Memory Technologies [125]	IC2	Focuses on theoretical performance models built for the HetSim simulator. Excluded because it relies on software simulation rather than providing actionable C-code proxies or benchmarks for existing hardware.
PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference [126]	EC2	Focuses entirely on Large Language Model (LLM) inference. Excluded under the irrelevant workload trap (EC2) because generative text decoding provides no valid deterministic proxy for quantum tensor algebra.
PIM-CARE: A Compiler-Assisted Dynamic Resource Allocation Framework for Real-world DRAM PIM [127]	EC2	Focuses on OS-level dynamic resource allocation for multi-tenant applications. While it evaluates GEMV, it does so to profile inter-application scalability rather than proposing novel intra-application mathematical chunking. Excluded under the OS/Systems trap (EC2) because multi-tenancy schedulers provide no actionable architectural proxy for a monolithic, single-tenant HPC quantum simulator.
PIM-CCA: An Efficient PIM Architecture with Optimized Integration of Configurable Functional Units [128]	IC2	Proposes a novel PIM architecture integrating custom Configurable Compute Accelerators (CCA). Excluded because theoretical custom silicon designs offer no practical C-code implementation value for commodity UPMEM DPUs.

Record	Failed criteria	Reason for exclusion
PIM-DL: Expanding the Applicability of Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization [129]	EC2	Focuses exclusively on algorithm-system co-optimization for Deep Learning. Excluded under the irrelevant workload trap (EC2) because AI-specific operator offloading offers no valid proxy for general tensor algebra.
PIM-MMU: A Memory Management Unit for Accelerating Data Transfers in Commercial PIM Systems [130]	IC2	Proposes a custom hardware Memory Management Unit (MMU) for data transfers. Excluded because it relies on architectural modifications rather than providing actionable software blueprints for existing, locked commodity UPMEM hardware.
PIM-Opt: Demystifying Distributed Optimization Algorithms on a Real-World Processing-In-Memory System [131]	EC2	Focuses on Stochastic Gradient Descent (SGD) for ML training. Excluded under the irrelevant workload trap (EC2) because gradient-based AI optimization algorithms do not serve as a deterministic proxy for quantum tensor contractions.
PIM-Potential: Broadening the Acceleration Reach of PIM Architectures [132]	IC2	Analyzes hardware constraints such as row activation overheads for commercial PIM architectures. Excluded because it relies on memory-controller level commands (typical of HBM-PIM) rather than the DMA-based MRAM-to-WRAM programming model of commodity UPMEM DPUs, offering no actionable software blueprints.
PIM-STM: Software Transactional Memory for Processing-In-Memory Systems [133]	EC2	Targets UPMEM but focuses on Software Transactional Memory (STM). Excluded under the irrelevant workload trap (EC2) because OS-level concurrency control mechanisms (locks/rollbacks) are unnecessary overhead for deterministic, single-tenant quantum physics workloads.
PIM-tree: A Skew-resistant Index for Processing-in-Memory [134]	EC2	Focuses on ordered indexes and skip lists for spatial/database queries. Excluded under the irrelevant workload trap (EC2) because tree traversal algorithms are fundamentally incompatible with the multidimensional floating-point block accesses of tensor networks.
PIM-trie: A Skew-resistant Trie for Processing-in-Memory [135]	EC2	Focuses on radix-based indexes and trie matching. Excluded under the irrelevant workload trap (EC2) because database search data structures offer no

Record	Failed criteria	Reason for exclusion
		valid architectural proxy for deterministic dense/sparse matrix math.
PIMSAB: A Processing-In-Memory System with Spatially-Aware Communication and Bit-Serial-Aware Computation [136]	EC2	Proposes a custom PIM architecture featuring a physical spatial communication network and bit-serial ALUs. Excluded because (1) bit-serial computation is for low-precision AI and cannot support high-precision quantum floating-point math (EC2), and (2) custom spatial silicon networks provide no actionable software routing proxies for physically isolated commodity UPMEM DPUs (IC2).
PIMSys: A Virtual Prototype for Processing in Memory [137]	IC2	Develops a gem5-based software simulator for Samsung’s HBM-PIM architecture. Excluded because theoretical simulations of non-UPMEM hardware paradigms offer no actionable C-code blueprints for commodity digital DRAM-PIM.
PLAIN: Leveraging High Internal Bandwidth in PIM for Accelerating Large Language Model Inference via Mixed-Precision Quantization [138]	EC2	Focuses on Large Language Model (LLM) inference and mixed-precision quantization. Excluded under the AI trap (EC2) because low-bit quantization is fundamentally incompatible with the high-precision floating-point requirements of quantum tensor networks.
Porting and Optimizing BWA-MEM2 Using the Fujitsu A64FX Processor [139]	EC2	Focuses on bioinformatics (BWA-MEM2 sequence alignment) mapped to the Fujitsu A64FX CPU. Excluded under the irrelevant workload trap (EC2) because genomic string-matching offers no valid proxy for multidimensional tensor algebra.
Processing Multi-Layer Perceptrons In-Memory [140]	EC2	Targets UPMEM but focuses exclusively on Multi-Layer Perceptrons (MLPs). Excluded under the irrelevant workload trap (EC2) because AI-specific network implementations do not provide clean, generalized mathematical proxies for bare-metal quantum physics computations.
Processing-in-Memory for Databases: Query Processing and Data Transfer [141]	EC2	Targets commercial PIM but focuses on accelerating relational database query processing. Excluded under the irrelevant workload trap (EC2) because database scans and queries do not serve as a valid architectural proxy for multidimensional tensor contractions.
RayN: Ray Tracing Acceleration with Near-memory Computing [142]	EC2	Focuses on accelerating ray tracing and BVH tree traversals. Excluded under the irrelevant workload trap (EC2) because geometric ray tracing

Record	Failed criteria	Reason for exclusion
		algorithms provide no valid deterministic proxy for generalized quantum tensor algebra. Furthermore, relies on custom 3D-stacked hardware (IC2).
SAFE: Sharing-Aware Prefetching for Efficient GPU Memory Management With Unified Virtual Memory [143]	IC2 / EC4	Focuses on GPU Unified Virtual Memory (UVM) and hardware prefetchers. Excluded because it relies on shared virtual address spaces and hardware cache management, which are fundamentally incompatible with the isolated, explicit DMA-based memory architecture of UPMEM DPUs.
Scalable Address Spaces using Concurrent Interval Skiplist [144]	EC2 / IC2	Focuses on optimizing Linux kernel virtual memory operations (mmap/munmap) for multi-core CPUs. Excluded because UPMEM DPUs operate on physical memory without OS-level virtual address spaces, rendering kernel-level locking optimizations completely irrelevant.
Shared-PIM: Enabling Concurrent Computation and Data Flow for Faster Processing-in-DRAM [145]	IC2	Proposes a custom in-DRAM architecture requiring physical modifications to memory banks and peripherals. Excluded under the hardware trap (IC2) because architectural silicon modifications cannot be implemented via software on existing commodity UPMEM hardware.
SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems [146]	IC2	Proposes a novel Instruction Set Architecture (ISA) for graph mining. Excluded under the hardware trap (IC2) because proposing theoretical, custom silicon instructions provides no actionable C-code proxies for the locked 32-bit RISC pipeline of commodity UPMEM DPUs.
SLIM: A Heterogeneous Accelerator for Edge Inference of Sparse Large Language Model via Adaptive Thresholding [147]	EC2	Focuses on Edge LLM inference using adaptive thresholding. Excluded under the AI trap (EC2) because neural network approximation algorithms destroy the high-precision floating-point accuracy required for quantum state amplitudes. Furthermore, proposes custom hardware rather than UPMEM (IC2).
SparsePIM: An Efficient HBM-Based PIM Architecture for Sparse Matrix-Vector Multiplications [148]	IC2	While the workload is SpMV, the paper proposes a novel, custom HBM-based PIM hardware architecture. Excluded under the hardware trap (IC2) because theoretical 3D-stacked silicon designs provide no actionable software blueprints for commodity DDR4-based UPMEM DPUs.

Record	Failed criteria	Reason for exclusion
SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration [149]	EC2	Focuses on speculative inference for Large Language Models (LLMs). Excluded under the AI trap (EC2) because generative text decoding and token verification provide no actionable deterministic proxy for quantum tensor algebra.
SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs [150]	EC2	Targets UPMEM but focuses on accelerating relational database joins. Excluded under the irrelevant workload trap (EC2) because SQL hash-joins provide no valid architectural proxy for deterministic multidimensional tensor math.
SplitSync: Bank Group-Level Split-Synchronization for High-Performance DRAM PIM [151]	IC2	Proposes a custom DRAM PIM hardware design to enable asynchronous bank-group execution. Excluded under the hardware trap (IC2) because architectural silicon modifications cannot be implemented via software on existing commodity UPMEM hardware.
SwiftRL: Towards Efficient Reinforcement Learning on Real Processing-In-Memory Systems [152]	EC2	Targets real PIM but focuses entirely on Reinforcement Learning (RL). Excluded under the AI trap (EC2) because the stochastic memory access patterns of RL experience replay buffers provide no actionable deterministic proxy for quantum tensor algebra.
Taming Extreme Heterogeneity via Machine Learning based Design of Autonomous Manycore Systems [153]	IC2 / EC2	Proposes a Machine Learning framework to design custom heterogeneous manycore hardware. Excluded because theoretical silicon design frameworks offer no practical software implementation value for existing commodity UPMEM hardware.
To PIM or Not for Emerging General Purpose Processing in DDR Memory Systems [154]	IC2	While it proposes a software compilation framework, it relies on adding custom vector engine extensions to the PIM cores. Excluded under the hardware trap (IC2) because software targeting hypothetical vector extensions cannot be executed on the locked 32-bit RISC architecture of UPMEM.
Towards Disaggregated NDP Architectures for Large-scale Graph Analytics [155]	IC2	Focuses on Disaggregated Near-Data Processing (network-attached memory pools). Excluded under the hardware trap (IC2) because network-pooled memory relies on fundamentally different communication fabrics (e.g., CXL) than local, in-server commodity UPMEM DIMMs.

Record	Failed criteria	Reason for exclusion
Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures [156]	EC5	This is a 2-page extended abstract for the previously included “SparseP” paper. Excluded under the quality/format trap (EC5) to avoid literature duplication and because extended abstracts lack the comprehensive methodology and code required for extraction.
UM-PIM: DRAM-based PIM with Uniform & Shared Memory Space [157]	EC4	Focuses on enabling uniform and shared memory spaces for PIM. Excluded under the shared-memory trap (EC4) because relying on cache-coherence or shared virtual address spaces violates the physically isolated, explicit DMA-based architecture of commodity UPMEM DPUs.
Uncore your Queries: Towards CPU-less Query Processing [158]	IC2 / EC2	Focuses on accelerating database queries using Intel Xeon Data Accelerator Complex (IAA). Excluded because CPU-attached accelerators (IC2) and relational database scans (EC2) provide no actionable software blueprints for commodity UPMEM tensor algebra.
Understanding Product Line Runtime Performance with Behaviour Models and Regression Model Trees [159]	EC2	Focuses on software engineering and performance modeling for software product lines. Excluded under the irrelevant domain trap (EC2) because it does not involve PIM architectures or high-performance tensor computing.
UNIFICO: Thread Migration in Heterogeneous-ISA CPUs without State Transformation [160]	IC2	Focuses on OS-level thread migration compilers for heterogeneous-ISA CPUs. Excluded because UPMEM relies on explicit, statically compiled Host/DPU binary dispatching rather than dynamic runtime thread migration.
UpANNS: Enhancing Billion-Scale ANNS Efficiency with Real-World PIM Architecture [161]	EC2	Targets real UPMEM hardware but focuses on Approximate Nearest Neighbor Search (ANNS). Excluded under the irrelevant workload trap (EC2) because vector distance calculations and top-K pruning algorithms offer no valid mathematical proxy for deterministic tensor algebra.
Virtual PIM: Resource-aware Dynamic DPU Allocation and Workload Scheduling Framework for Multi-DPU PIM Architecture [162]	EC2	Focuses on OS-level dynamic DPU allocation and spatial multitasking for concurrent applications. Excluded under the OS/Systems trap (EC2) because multi-tenancy schedulers provide no actionable architectural proxy for a monolithic, single-tenant HPC workload.
vPIM: Processing-in-Memory Virtualization [163]	EC2	Focuses on virtualizing UPMEM DPUs for cloud hypervisors (Firecracker). Excluded under the OS/

Record	Failed criteria	Reason for exclusion
		Systems trap (EC2) because multi-tenant cloud virtualization frameworks provide no actionable architectural proxy for bare-metal, single-tenant HPC quantum simulators.
vSPACE: Supporting Parallel Network Packet Processing in Virtualized Environments through Dynamic Core Management [164]	EC2	Focuses on CPU core management and network packet processing for virtual machines. Excluded under the irrelevant domain trap (EC2) because it does not involve PIM architectures or high-performance memory computing.

14. Appendix C – Full-Text Review

14.1. Purpose of the full-text review

This appendix documents the full-text eligibility assessment described in Section 5.2. The 37 records that advanced from title-and-abstract screening were assessed against the tier-specific eligibility logic defined in Section 3.6.

The purpose of this appendix is to record the final eligibility decisions after full-text review. Studies listed as included here form the final corpus used for quality appraisal, data extraction, and evidence synthesis.

14.2. Full-text eligibility summary

Evidence tier	Full texts assessed	Included	Excluded
Tier 1: Quantum simulation on PIM	2	1	1
Tier 2: Tensor-network algorithmic analogs	12	10	2
Tier 3: UPMEM hardware proxy workloads	23	22	1
Total	37	33	4

Table 9: Summary of full-text eligibility decisions by evidence tier.

14.3. Studies included after full-text review

The studies below satisfied the tier-specific eligibility rules after full-text assessment and formed the final included corpus.

14.3.1. Tier 1: Quantum simulation on PIM

Study	Eligibility rationale
PIMutation [9]	Included because it directly evaluates quantum circuit simulation on commodity UPMEM hardware. It provides the only direct Tier 1 evidence for adapting quantum simulation to digital PIM.

14.3.2. Tier 2: Tensor-network algorithmic analogs

Study	Eligibility rationale
AutoHOOT [13]	Included because it addresses high-order tensor optimization, contraction-path selection, dimension-tree construction, and memory-footprint reduction. These methods provide transferable algorithmic evidence for memory-constrained tensor contraction.
Closing the Quantum Supremacy Gap [10]	Included because it performs large-scale random quantum circuit simulation using tensor-network contraction and memory-aware slicing. Its contraction planning and distributed execution strategy are relevant to memory-constrained simulation.
DISTAL [17]	Included because it provides a distributed tensor algebra compiler with explicit data distribution and communication scheduling. This offers transferable evidence for orchestrating tensor operations across separated memory domains.
Efficient Quantum Circuit Simulation by Tensor Network Methods on Modern GPUs [12]	Included because it studies tensor-network quantum circuit simulation on GPUs, including Einsum-to-GEMM transformation and contraction-path feedback. It provides a conventional-architecture baseline for comparison with PIM-aware execution.
Lifetime-Based Optimization for Simulating Quantum Circuits [11]	Included because it proposes lifetime-based slicing and contraction-path refinement for tensor-network quantum circuit simulation under explicit memory constraints. These methods are directly relevant to memory-capped contraction planning.
Minimum Cost Loop Nests for Contraction of a Sparse Tensor with a Tensor Network [15]	Included because it studies loop-nest optimization for sparse-dense tensor-network contraction, including buffer and cache behavior. This provides algorithmic evidence for reducing memory traffic in tensor workloads.
Surpassing Sycamore [5]	Included because it demonstrates large-scale tensor-network quantum circuit simulation using memory-constrained system-level optimization. Its communication and data-layout strategies provide relevant evidence for transfer-aware simulation design.
SW_Qsim [14]	Included because it proposes a minimize-memory quantum simulation strategy based on tensor-network contraction and recursive slicing. The work is relevant to reducing intermediate storage requirements in constrained-memory environments.
Tensor Network Quantum Virtual Machine [32]	Included because it supports exact and approximate quantum circuit simulation using tensor networks and distributed-memory execution. Its contraction-path and slicing strategies provide relevant evidence for scalable tensor-network simulation.

Study	Eligibility rationale
The Tensor Algebra Compiler [16]	Included because it provides compiler abstractions for dense and sparse tensor algebra through iteration graphs and merge lattices. These abstractions are relevant for generating memory-aware tensor kernels.

14.3.3. Tier 3: UPMEM hardware proxy workloads

Study	Eligibility rationale
A Full-System Perspective on UPMEM Performance [33]	Included because it evaluates UPMEM performance from a full-system perspective and characterizes offloading overheads, DPU memory constraints, and host-mediated data movement.
A General Offloading Approach for Near-DRAM Processing-In-Memory Architectures [34]	Included because it studies general offloading policies for near-DRAM PIM and analyzes locality and data movement. It provides transferable evidence for host-to-PIM orchestration.
Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices [24]	Included because it develops an efficient GEMM library for real-world PIM devices. Although the application is LLM acceleration, GEMM is a relevant proxy for tensor-contraction kernels.
AESPA [35]	Included because it studies asynchronous execution and bank-level parallelism in PIM architectures. The work provides architectural evidence for scheduling memory-bound vector and matrix operations.
ALPHA-PIM [20]	Included because it evaluates graph and linear-algebra workloads on real UPMEM hardware, including SpMV partitioning and host-PIM transfer behavior. These constraints are relevant to sparse tensor workloads.
An Efficient PIM-Based Graph Engine on a Single Machine [29]	Included because it studies graph processing on PIM and provides evidence about data movement, partitioning, and irregular memory access on digital PIM systems.
Analysis of Data Transfer Bottlenecks in Commercial PIM Systems [18]	Included because it directly measures data-transfer bottlenecks on commercial UPMEM systems. These measurements are central to modeling host-DPU transfer costs for tensor workloads.
Architectural Modeling and Benchmarking for Digital DRAM PIM [36]	Included because it provides modeling and benchmarking infrastructure for digital DRAM PIM systems. This supports reasoning about performance and energy behavior of memory-bound workloads.
ATiM [19]	Included because it provides an autotuning framework for tensor programs on Processing-in-DRAM. It is directly relevant to mapping tensor operations onto PIM-aware execution models.

Study	Eligibility rationale
Benchmarking a New Paradigm [6]	Included because it provides comprehensive experimental characterization of real UPMEM hardware, including memory hierarchy, SPMD execution, and host-mediated communication.
CINM [38]	Included because it provides a compilation infrastructure for compute-in-memory and compute-near-memory systems, including lowering to UPMEM-style execution. It is relevant to automated host-DPU code generation.
ComPASS [39]	Included because it studies processor-PIM collaboration and scheduling for digital PIM architectures. It provides systems-level evidence for coordinating work between host and PIM units.
CoPIM [40]	Included because it targets commodity PIM systems and studies CPU-PIM collaborative scheduling. Its modeling of host-PIM data transfers is relevant to sliced tensor execution.
Pac-PIM [23]	Included because it proposes a communication framework for commodity PIM systems. It directly addresses host-to-DPU communication bottlenecks relevant to distributed tensor workloads.
Pathfinding Future PIM Architectures [41]	Included because it demystifies commercial PIM technology and provides simulator-supported architectural characterization of UPMEM-style systems.
PID-Comm [22]	Included because it provides collective communication support for commodity Processing-in-DIMM devices. This is relevant to reductions and synchronization in sliced tensor contraction.
PRISM [27]	Included because it implements sparse MTTKRP on UPMEM PIM. MTTKRP is a relevant sparse tensor-operation proxy for tensor-network execution.
PyGim [26]	Included because it evaluates graph neural network kernels on real PIM hardware, including SpMM and GEMM-style operations. These kernels provide relevant sparse/dense matrix proxies for tensor workloads.
Scalability Limitations of Processing-in-Memory using Real System Evaluations [42]	Included because it analyzes scaling limits and communication patterns on real UPMEM systems. Its findings are relevant to multi-DPU execution and host-mediated communication.
SimplePIM [28]	Included because it provides a software framework for programming PIM systems and managing host-PIM interaction. It is relevant to productivity and execution orchestration on UPMEM.
SparseP [25]	Included because it provides a comprehensive SpMV library and evaluation on real UPMEM hardware. It is a strong proxy for irregular sparse tensor operations.

Study	Eligibility rationale
TransPimLib [21]	Included because it implements transcendental functions for constrained PIM hardware. This is relevant to complex-valued quantum simulation operations that require sine, cosine, and exponential-like functionality.

14.4. Studies excluded after full-text review

The studies below advanced to full-text assessment but were excluded after detailed eligibility review.

Study	Tier	Failed criteria	Reason for exclusion
Symphony [30]	T1	IC1 / IC2	Excluded because the full text proposes a custom heterogeneous tensor-processing architecture rather than evaluating commodity digital PIM or distributed-memory execution. It also does not address quantum simulation workloads.
gHyPart [31]	T2	IC1 / IC3 / EC2	Excluded because the full text focuses on VLSI-oriented hypergraph partitioning rather than tensor-network contraction, tensor slicing, or transferable memory-constrained tensor execution.
Simulation of Quantum Computers [3]	T2	IC3 / EC5	Excluded from the final corpus because it is a broad survey and does not provide a specific algorithmic method, implementation, or experimental setup suitable for data extraction under the Tier 2 eligibility rules. It remains useful as background literature.
Benchmarking Memory-Centric Computing Systems [37]	T3	EC5	Excluded because it is an invited summary that omits the full experimental methodology and is superseded by the more complete 2022 journal version [6].

15. Appendix D – Quality Appraisal

15.1. Purpose of the quality appraisal appendix

This appendix reports the per-study quality appraisal records used to generate the summary statistics in Section 6. Each study in the final corpus was evaluated using an eight-criterion Quality Appraisal (QA) rubric. The rubric was designed to assess whether each study provides reliable, reproducible, and transferable evidence for the review questions.

Each criterion was scored on a 0–2 scale: 0 indicates that the criterion is not satisfied, 1 indicates partial satisfaction, and 2 indicates full satisfaction. The maximum possible score is therefore 16. To improve separation between studies, the final classification used stricter score bands: studies scoring

14–16 points were classified as high quality, studies scoring 10–13 points were classified as medium quality, and studies scoring below 10 points were classified as low quality.

15.2. Quality appraisal rubric

- QR1** *Aim clarity* — The study clearly defines its research objectives, target workload, and evaluation scope.
- QR2** *Reproducibility* — The study provides sufficient methodological detail, algorithms, pseudocode, mathematical definitions, or experimental setup information to support replication.
- QR3** *Baseline comparison* — The study compares its method against relevant CPU, GPU, distributed-memory, prior PIM, or established software baselines.
- QR4** *Data movement* — The study explicitly analyzes or optimizes memory bandwidth, host-device transfer, DMA movement, communication cost, cache behavior, or locality.
- QR5** *Hardware realism* — The study reflects realistic hardware constraints, such as WRAM capacity, MRAM-WRAM movement, lack of native floating-point support, isolated DPUs, physical PIM devices, or validated simulation parameters.
- QR6** *Open artifacts* — The study provides public code, datasets, benchmark suites, simulation frameworks, or sufficient artifact material for reuse.
- QR7** *Limitations* — The study provides a critical discussion of limitations, bottlenecks, scalability constraints, or threats to validity.
- QR8** *Contribution depth* — The study provides substantial scientific or engineering value for this review, such as a reproducible baseline, a novel method, a reusable framework, or foundational theory.

15.3. Quality appraisal cards

The following cards report the score assigned to each included study, together with brief evidence supporting each criterion and a short note on its relevance to the review.

Tier 1: Quantum simulation on PIM

PIMutation: Exploring the Potential of Real PIM Architecture for Quantum Circuit Simulation			
		[9] Tier 1 – PIM	
QR1 Aim clarity	2	p.1 aims PIM UPMEM	
QR2 Reproducibility	1	p.4-5 description, no pseudo-code	
QR3 Baseline	2	p.6-7 vs QuEST	
QR4 Data movement	2	p.4-5, 7 WRAM/DMA breakdown	
QR5 Hardware realism	2	p.3,6 20 UPMEM DIMMs = 2,560 DPU	
QR6 Open artifacts	0	none – no code repository	
QR7 Limitations	2	p.2 integer only; p.2 no inter-DPU communication; p.5 vector partitioning needs separable states	
QR8 Contribution	2	b – new method: gate merging, row swapping, vector partitioning	

Score: 13 / 16 (81% – Medium)

Review relevance: Direct PIM baseline for state-vector; proves VP fails on entanglement – justifies tensor approach.

Tier 2: Tensor-network algorithmic analogs

AutoHOOT: Automatic High-Order Optimization for Tensors

[13] Tier 2 – Tensor-GPU

QR1 Aim clarity	2	p.1 aims AD (automatic differentiation) for tensor optimization
QR2 Reproducibility	2	p.4 algorithms, p.2 code available
QR3 Baseline	2	p.9-10 vs JAX, TensorFlow, TensorLy, scikit-tensor, Quimb
QR4 Data movement	1	p.8 optimizes FLOPs only, cost model ignores communication
QR5 Hardware realism	1	p.10 K80/Titan X GPU, Stampede2 KNL CPU
QR6 Open artifacts	2	p.2 arXiv v2 – code at github.com/LinjianMa/AutoHOOT ; URL omitted in PACT PDF
QR7 Limitations	1	p.6 notes NP-hard search, p.8 notes hardware-oblivious assumption; no dedicated limitations section
QR8 Contribution	2	p.2 new method - a new Jacobian/Hessian kernel, computational graph optimization module

Score: 13 / 16 (81% – Medium)

Review relevance: Code URL appears in arXiv v2 page 1 but is omitted from PACT PDF; repository on github accessed 21 April 2026; plan to fork and adapt for UPMEM memory constraints

Closing the “Quantum Supremacy” Gap: Achieving Real-Time Simulation of a Random Quantum Circuit Using a New Sunway Supercomputer

[10] Tier 2 – Tensor-Supercomputer

QR1 Aim clarity	2	p.1 high-performance tensor-based simulator for random quantum circuits
QR2 Reproducibility	2	p.5-9 details slicing and contraction path algorithms
QR3 Baseline	2	p.10-11 vs qFlex, Alibaba, Google Sycamore baseline
QR4 Data movement	1	p.6-7 optimizes for compute density; p.11 data movement hidden in permutations
QR5 Hardware realism	1	p.10-11 Sunway many-core CPU, not PIM
QR6 Open artifacts	2	p.11 https://github.com/Daisyforest/SWQsim (no code?)
QR7 Limitations	1	p.11 low efficiency for CoTenGra-generated tensors; no general portability discussion
QR8 Contribution	2	p.5-9 architecture-aware slicing, fused permutation-multiplication, mixed-precision

Score: 13 / 16 (81% – Medium)

Review relevance: Repository available; plan to distill architecture-aware contraction path methods for UPMEM Host compiler

DISTAL: The Distributed Tensor Algebra Compiler

[17] Tier 2 – Tensor-Supercomputer

QR1 Aim clarity	2	p.1 machine-aware compiler for dense tensor algebra
QR2 Reproducibility	1	p.7-8 pseudocode for distribution algorithms
QR3 Baseline	2	p.10-13 vs Cyclops Tensor Framework, COSMA, ScaLAPACK
QR4 Data movement	1	p.3-4 data distribution per node with locality awareness
QR5 Hardware realism	1	p.10 Lassen supercomputer – IBM Power9 CPU + NVIDIA V100 GPU
QR6 Open artifacts	0	no public artifact linked
QR7 Limitations	1	p.14 limited to dense tensors; supercomputer target only
QR8 Contribution	2	p.3-6 separation of data distribution and computation scheduling via abstract processor grid
Score: 10 / 16 (63% – Medium)		
Review relevance: Core abstraction for mapping tensor algebra to hierarchical machines; highly relevant for designing the Host-to-PIM data distribution and scheduling.		

Efficient Quantum Circuit Simulation by Tensor Network Methods on Modern GPUs [12]

Tier 2 – Tensor-GPU

QR1 Aim clarity	2	p.1 transform Einsum to GEMM and use extended/mixed precision for GPU tensor cores
QR2 Reproducibility	1	p.8-14 equations for path finding, slicing, and precision conversion; no implementation pseudocode
QR3 Baseline	2	p.19 vs state-of-the-art CPU [28] and GPU [25,42] baselines on Sycamore
QR4 Data movement	0	p.7 discusses arithmetic intensity; no model of data movement or communication
QR5 Hardware realism	1	p.17 DGX-A100 with eight NVIDIA A100-SXM-80GB GPUs
QR6 Open artifacts	0	no public artifact
QR7 Limitations	1	p.19-20 need to optimize index arrangement; rescaling required for low precision; GPU-only design
QR8 Contribution	2	p.8-10 path-finding with dynamic slicing; p.10-13 Einsum-to-GEMM conversion; p.13-14 extended/mixed precision scheme
Score: 9 / 16 (56% – Low)		
Review relevance: Einsum-to-GEMM and precision-scaling adaptable to UPMEM integer cores; evaluate on real hardware gives credible performance baseline		

Lifetime-based Optimization for Simulating Quantum Circuits on a New Sunway Supercomputer

[11] Tier 2 – Tensor-Supercomputer

QR1 Aim clarity	2	p.1 lifetime-based methods to reduce slicing overhead in tensor network contraction
QR2 Reproducibility	1	p.7 pseudocode for in-place slice finder
QR3 Baseline	2	p.10 vs Cotengra, Alibaba, and Liu et al. 2021 Gordon Bell
QR4 Data movement	2	p.8-9 secondary slicing fuses thread-level communication to eliminate discrete LDM access
QR5 Hardware realism	1	p.4 Sunway SW26010Pro processor, 41M cores

QR6 Open artifacts	0	no public artifact
QR7 Limitations	1	p.11 permutation via LDM access still limits performance vs peak
QR8 Contribution	2	p.2-3 lifetime analysis of tensor edges; in-place slicing; architecture-aware path refiner
Score: 11 / 16 (69% – Medium)		
Review relevance: Direct follow-on to Liu 2021 [10]; demonstrates how lifetime tracking reduces slicing overhead – reusable for UPMEM memory lifetime modeling		

Minimum Cost Loop Nests for Contraction of a Sparse Tensor with a Tensor Network [15]

Tier 2 – Sparse Tensor

QR1 Aim clarity	2	p.1 framework for SpTTN kernels in decomposition and completion
QR2 Reproducibility	1	p.7-8 pseudocode for DP(Dynamic Programming)-based loop nest enumeration
QR3 Baseline	2	p.10 vs TACO, SparseLNR, SPLATT, CTF
QR4 Data movement	1	p.6-7 cost model for buffer size and total cache misses
QR5 Hardware realism	1	p.9-10 Stampede2 KNL (Xeon Phi 7250) and Ice Lake CPU testbed
QR6 Open artifacts	0	no public artifact
QR7 Limitations	1	p.11 limited to SpTTN kernels; partially-fused nests not explored
QR8 Contribution	2	p.7-8 DP algorithm finds minimum-cost loop order for given contraction paths
Score: 10 / 16 (63% – Medium)		
Review relevance: Cache-miss cost model and DP enumeration transferable to PIM loop ordering; sparse-only scope limits direct use		

Surpassing Sycamore: Achieving Energetic Superiority Through System-Level Circuit Simulation

[5] *Tier 2 – Tensor-Supercomputer*

QR1 Aim clarity	2	p.1 surpass Sycamore in time-to-solution and energy
QR2 Reproducibility	2	p.5 three-level parallelization scheme pseudocode
QR3 Baseline	2	p.1-2 vs Sycamore, Liu 2021, Pan 2022
QR4 Data movement	2	p.5-6 customized low-precision communication inter-node
QR5 Hardware realism	1	p.9 80GB A100 cluster, up to 2304 GPUs
QR6 Open artifacts	2	p.1 github.com/DeepLink-org/OpenTenNet
QR7 Limitations	1	p.12 system optimized for GPU clusters only
QR8 Contribution	2	p.5 global-node-device parallelization with quantized communication
Score: 14 / 16 (88% – High)		
Review relevance: Current SOTA for quantum simulation; low-precision communication directly relevant to PIM bandwidth constraints		

SW_Qsim: A Minimize-Memory Quantum Simulator with High-Performance on a New Sunway Supercomputer

[14] *Tier 2 – Tensor-Supercomputer*

QR1 Aim clarity	2	p.1 tensor-network simulator with two-level parallelism for Sunway
QR2 Reproducibility	1	p.4-8 minimize-memory path and thread-level optimizations
QR3 Baseline	1	p.11 vs Guochu et al. (previous state of art)
QR4 Data movement	1	p.5 minimize-memory contraction path and thread parallelism
QR5 Hardware realism	1	p.3 Sunway SW26016pro many-core processor and interconnect (abstract cites SW26010pro)
QR6 Open artifacts	1	p.13 Appendix repo - not working but found one that works
QR7 Limitations	1	p.1-2 PEPS-based simulation tailored for Sunway
QR8 Contribution	2	p.1,3-4 process-level and thread-level parallel framework
Score: 10 / 16 (63% – Medium)		
Review relevance: Good example of parallelism – but topology designed for Sunway and PEPS - found related repo: https://github.com/Daisyforest/SWQsim		

Tensor Network Quantum Virtual Machine for Simulating Quantum Circuits at Exascale

[32] Tier 2 – Tensor

QR1 Aim clarity	2	p.1 modernized TNQVM with exact and approximate modes
QR2 Reproducibility	2	p.14-17 code snippets for MPS and contraction
QR3 Baseline	0	no quantitative comparison
QR4 Data movement	0	no explicit (PIM-relevant) data movement
QR5 Hardware realism	2	p.12 Summit 384 V100s, DGX-A100 8×80GB
QR6 Open artifacts	2	p.4,17 open source public domain, https://github.com/ORNL-QCI/tnqvm/tree/master/examples/sycamore
QR7 Limitations	1	p.7 accuracy-memory tradeoff for approximations
QR8 Contribution	2	p.6 multimodal TN with ExaTN backend
Score: 11 / 16 (69% – Medium)		
Review relevance: General HPC TN framework; approximate methods and lack of PIM-specific data movement optimization place it outside thesis core		

The Tensor Algebra Compiler

[16] Tier 2 – Tensor

QR1 Aim clarity	2	p.1 automatic generation of kernels for tensor algebra in taco
QR2 Reproducibility	2	p.2-15 algorithm, code snippets, descriptions
QR3 Baseline	2	p.16 vs Eigen, uBLAS, Gmm++, OSKI, MKL
QR4 Data movement	1	p.4-10 data storage formats explicitly targeted
QR5 Hardware realism	2	p.17 2×12-core Xeon E5-2695 v2, 128GB, 30MB L3
QR6 Open artifacts	2	p.2,25 MIT license, tensor-compiler.org , github.com/tensor-compiler/taco
QR7 Limitations	2	p.26 future work: GPUs, formats, tiling, autotuning, JIT
QR8 Contribution	2	p.2 tensor formats, iteration graphs, merge lattices, codegen

Score: 15 / 16 (94% – High)

Review relevance: Auto code generation base; cited by AutoHOOT and DISTAL; potential foundation for PIM tensor code generation

Tier 3: UPMEM hardware proxy workloads

A Full-System Perspective on UPMEM Performance

[33] Tier 3 – PIM-Critique

QR1 Aim clarity	2	p.1 examine UPMEM PIM performance with full-system view
QR2 Reproducibility	1	p.4-6 description of optimizations and benchmarks
QR3 Baseline	2	p.6 vs dual-socket Xeon Gold 6152, 44c/88t, 384GB
QR4 Data movement	2	p.5 explicit data transfer to/from PIM
QR5 Hardware realism	2	p.5 2304 DPUs, 16 tasklets each, 350MHz
QR6 Open artifacts	2	p.4 https://ess.cs.uos.de/git/artifacts/dimes23-upmem-performance
QR7 Limitations	2	p.1-2 memory controller interleaving, 4K instruction limit
QR8 Contribution	1	p.6 optimization and reconstruction of extended benchmarks

Score: 14 / 16 (88% – High)

Review relevance: Workshop paper showing CPU domination; critical for realistic PIM expectations but not core technique

A General Offloading Approach for Near-DRAM Processing-In-Memory Architectures

[34] Tier 3 – Near-DRAM

QR1 Aim clarity	2	p.1 general-purpose instruction-level offloading for near-DRAM PIM, IOTPIM
QR2 Reproducibility	1	p.5 overview of IOTPIM system
QR3 Baseline	2	p.2,8 vs LLCMIRO, GraphPIM, TDO-CIM, PRIMO
QR4 Data movement	2	p.5-7 hierarchy-aware workload offloading
QR5 Hardware realism	2	p.7-9 Ramulator-PIM, 16 cores@2.5GHz, PIM@1.2GHz, 1KB buffer/core
QR6 Open artifacts	0	no public artifacts
QR7 Limitations	1	p.6,10 simulation-only; 3.4× faster than real UPMEM; small dataset gap
QR8 Contribution	2	p.2 identifies PIM offloading inaccuracy; proposes policy and prediction model

Score: 12 / 16 (75% – Medium)

Review relevance: Simulation with inflated 1.2GHz PIM (vs 350MHz real); 1KB buffer matches UPMEM WRAM limits; 1.28-1.51× speedup not realizable on hardware

Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices

[24] Tier 3 – UPMEM-LLM

QR1 Aim clarity	2	p.1 PIM-LLM framework to accelerate LLMs using tiled GEMM on real PIM
QR2 Reproducibility	2	p.4-5 PGEMMLib with tiling and Tile-Selector analytical model

QR3 Baseline	2	p.1,10 vs TVM baseline, CPU; up to 45.75× on LLaMA-7b
QR4 Data movement	2	p.1 minimizing host-PIM transfer cost is crucial for LLM serving
QR5 Hardware realism	2	p.9 20×UPMEM modules, 160GB, 450MHz DPUs, 2×Xeon Gold 5222
QR6 Open artifacts	0	no public artifacts
QR7 Limitations	2	p.9 no native FP, requires int8 quantization; p.10 MRAM limit forces CPU fallback
QR8 Contribution	2	p.1 PGEMMLib tiling considers architecture-specific characteristics
Score: 14 / 16 (88% – High)		
Review relevance: CRITICAL: 45.75× speedup requires int8 quantization — UPMEM lacks FP32/FP64; 64MB/DPU MRAM limit forces CPU offload even with 160GB total; proves UPMEM structurally hostile to quantum TN workloads		

AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory

[35] Tier 3 – PIM-Arch

QR1 Aim clarity	2	p.1 asynchronous execution to exploit bank-level parallelism in near-bank PIM
QR2 Reproducibility	2	p.2-3 per-bank long-data command vs all-bank synchronous execution
QR3 Baseline	2	p.1 vs PIM-HBM and Newton baselines; 33.5% and 59.5% speedup
QR4 Data movement	2	p.1-2 mitigates activation/precharge overhead (43-70% of execution time)
QR5 Hardware realism	2	p.9 DRAMsim3 simulation; 4×HBM2 stacks, 16-lane units, 256B RF/bank
QR6 Open artifacts	0	no public artifacts
QR7 Limitations	2	p.7-8 requires shared accumulator per bank group; prevents tFAW; 11.7% area overhead
QR8 Contribution	2	p.2-3 hides activation latency by computing multiple DRAM columns per bank
Score: 14 / 16 (88% – High)		
Review relevance: Simulation-only; requires hardware mod (shared accumulator) impossible on isolated UPMEM DPUs; HBM-specific		

ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System

[20] Tier 3 – UPMEM-Graph

QR1 Aim clarity	2	p.1 explore capabilities of graph algorithms on real-world PIM
QR2 Reproducibility	1	p.4-6 SpMV and SpMSpV with 1D/2D partitioning; adaptive format selection
QR3 Baseline	2	p.2,9 vs CPU (10.2×/48.8×/3.6× kernel) but GPU outperforms both by 100×+
QR4 Data movement	2	p.1 optimal data partitioning strategies across PIM critical
QR5 Hardware realism	2	p.7-8 20×UPMEM DIMMs (160GB, 2560 DPUs), 2×Xeon Silver 4110; peak 4.66 GFLOPS
QR6 Open artifacts	0	no public artifacts
QR7 Limitations	2	p.10 recommend non-blocking DMA, intra-thread forwarding, better sync
QR8 Contribution	2	p.8 PPR slow due to software-emulated FP multiplication (no hardware FP)

Score: 13 / 16 (81% – Medium)

Review relevance: Real UPMEM: GPU beats UPMEM 116× (BFS 2.08ms vs 241ms); UPMEM uses 524× more energy than GPU (131J vs 0.25J); software FP emulation fatal for quantum TN

An Efficient PIM-Based Graph Engine on a Single Machine

[29] Tier 3 – UPMEM-Graph

QR1 Aim clarity	2	p.1 process large graphs that don't fit in main memory using PIM
QR2 Reproducibility	1	p.3 asynchronous I/O + column-wise partitioning; details insufficient for reproduction
QR3 Baseline	0	p.4 invalid comparisons: strawman (42× vs self), memory-starved CPU (16GB vs 90GB), 2012 PowerGraph
QR4 Data movement	2	p.1 reduces CPU workloads to issue I/O more frequently
QR5 Hardware realism	2	p.2 20×UPMEM DIMMs (2560 DPUs, 64MB MRAM each), 4×DRAM DIMMs
QR6 Open artifacts	0	GitHub exists but contains only .md files, no source code
QR7 Limitations	0	p.3 claims "10MB limit" but still gives PIM 5.6× memory advantage; no real limitations discussed
QR8 Contribution	1	p.3-4 I/O optimization only; no compute analysis

Score: 8 / 16 (50% – Low)

Review relevance: CRITICAL FLAWS: Memory starvation persists even with 10MB limit (90GB vs 16GB), strawman baseline, undisclosed CPU, no GPU/energy data; artificial constraint hides true PIM costs

Analysis of Data Transfer Bottlenecks in Commercial PIM Systems: A Study With UPMEM-PIM

[18] Tier 3 – UPMEM-Characterization

QR1 Aim clarity	2	p.1 characterize DRAM↔PIM data transfer overheads on commercial UPMEM
QR2 Reproducibility	2	p.3 PrIM CPU-DPU microbenchmark (1MB/bank, 384 banks) + custom AVX512 memcpy
QR3 Baseline	2	p.3-4 vs theoretical peak bandwidth; vs conventional DRAM memory mapping
QR4 Data movement	2	p.1-2 separate DRAM/PIM address spaces force manual transfers in critical path
QR5 Hardware realism	2	p.3 Intel Xeon Gold 5222, 3×DDR4-3200 (76.8GB/s), 3×DDR4-2400 UPMEM (57.6GB/s)
QR6 Open artifacts	0	no artifacts released
QR7 Limitations	2	p.3-4 software thread scheduling causes imbalance; PIM BIOS disables interleaving
QR8 Contribution	2	p.3 quantifies CPU utilization, power (+70W), and per-channel bandwidth

Score: 14 / 16 (88% – High)

Review relevance: Critical real-hardware results: DRAM read 11.6% peak (8.9/76.8 GB/s), PIM write 15.5% peak (8.9/57.6 GB/s), +70W power, 100% CPU utilization, DRAM BW drops to 30% with PIM BIOS

Architectural Modeling and Benchmarking for Digital DRAM PIM

[36] Tier 3 – Simulation-Tools

QR1 Aim clarity	2	p.1 introduce PIMbench suite, PIMeval framework, and PIM API for portable PIM evaluation
QR2 Reproducibility	2	p.6-7 detailed simulator architecture, API specification, open-source C++ implementation
QR3 Baseline	2	p.8-10 compares 3 PIM architectures (bit-serial, Fulcrum, bank-level) vs CPU/GPU baselines
QR4 Data movement	2	p.6-7 models data movement, energy, and performance within PIM memory hierarchy
QR5 Hardware realism	1	p.7 Table II: simulation parameters only; no real PIM hardware (CPU/GPU baselines may be real)
QR6 Open artifacts	2	p.1 https://github.com/UVA-LavaLab/PIMeval-PIMbench , MIT license, Zenodo 10.5281/zenodo.13243685
QR7 Limitations	2	p.4 discusses limitations: no MAT-level modeling, digital-only, subarrays modeled as monolithic
QR8 Contribution	2	p.1-7 comprehensive framework: 17 benchmarks across 9 domains, detailed energy models, sensitivity analysis
Score: 15 / 16 (94% – High)		
Review relevance: Open-source PIM simulation framework with portable benchmark suite; enables architecture exploration but provides no real-hardware validation		

ATiM: Autotuning Tensor Programs for Processing-in-DRAM

[19] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 automatically optimize tensor kernels for UPMEM via joint host-kernel search
QR2 Reproducibility	2	p.5-8 full system design extending TVM with PIM-aware lowering
QR3 Baseline	2	p.9 PrIM benchmark as baseline; compares vs PrIM, SimplePIM, CPU
QR4 Data movement	2	p.7-8 PIM-aware optimizations for boundary conditions and DMA
QR5 Hardware realism	2	p.9,16 Intel Xeon Gold 5220R with 20 DDR4-2400 PIM modules (2048 DPUs)
QR6 Open artifacts	2	p.2 https://github.com/SNU-CODElab/atim
QR7 Limitations	2	p.12-13 discusses UPMEM variability, MAC-PIM limitations, MLIR integration challenges
QR8 Contribution	2	p.13 flexible autotuning-driven code generation for UPMEM
Score: 16 / 16 (100% – High)		
Review relevance: First autotuning compiler for real UPMEM; strong systems work but no TN/quantum relevance		

Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System

[6] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 first comprehensive analysis of real UPMEM PIM system; introduces PrIM benchmark suite
QR2 Reproducibility	2	p.5-16 detailed methodology; microbenchmarks for compute, memory, communication
QR3 Baseline	2	p.29-31 vs Intel Xeon CPU and Nvidia Titan V GPU across 16 benchmarks

QR4 Data movement	2	p.14-16 CPU↔DPU bandwidth characterization; 6.68 GB/s H2D, 4.74 GB/s D2H
QR5 Hardware realism	2	p.5,30 2,556-DPU and 640-DPU systems; Intel Xeon E3-1240; Titan V GPU
QR6 Open artifacts	2	p.1,43 https://github.com/CMU-SAFARI/prim-benchmarks
QR7 Limitations	2	p.31-32 discusses comparison difficulties; hardware immaturity; frequency limitations
QR8 Contribution	2	p.1-44 44-page deep dive; 16 benchmarks; roofline analysis; energy measurements
Score: 16 / 16 (100% – High)		
Review relevance: Landmark UPMEM characterization; establishes PRIM suite; essential baseline for all PIM systems work - criticized in some sense by [33]		

CINM (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms

[38] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 end-to-end compilation flow for heterogeneous CIM/CNM using MLIR
QR2 Reproducibility	2	p.4-10 MLIR dialects (cinm, cnm, cim); progressive lowering; cost models
QR3 Baseline	2	p.10-11 vs PRIM, OCC; evaluates on UPMEM and memristor crossbar
QR4 Data movement	2	p.9-10 models host-device transfers; data placement across DPUs
QR5 Hardware realism	2	p.10 real UPMEM (2048 DPUs) but uses very old Xeon E5-2630 v2 host
QR6 Open artifacts	2	p.10 https://github.com/tud-ccc/Cinnamon.git
QR7 Limitations	1	p.10 no heterogeneous setup; limited evaluation; p.12 future work
QR8 Contribution	2	p.1-16 full compiler stack; supports UPMEM, AiM, FIMDRAM, memristors
Score: 15 / 16 (94% – High)		
Review relevance: MLIR-based compiler for heterogeneous PIM; supports UPMEM but targets general workloads not TN/quantum. CPU baseline is severely outdated.		

ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration

[39] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 PIM-ACT command for multi-bank activation; compatible protocol across PIM architectures
QR2 Reproducibility	2	p.4-6 PIM-ACT encoding for DDR5/LPDDR5/HBM3; PIM request generator in MC
QR3 Baseline	2	p.9-10 vs FRFCFS, Batch, TCM, ST-BLC; evaluates scheduling policies
QR4 Data movement	2	p.5-6 reduces mode-switch overhead; models host-PIM data transfers
QR5 Hardware realism	1	p.9 simulation only; LPDDR5-6400 PIM (no real hardware)
QR6 Open artifacts	0	p.13 references PIMSimulator but no ComPASS code released
QR7 Limitations	1	p.1-2 discusses existing PIM limitations; minimal self-critique
QR8 Contribution	2	p.1-14 detailed protocol design; JEDEC-compatible; MICRO '25

Score: 12 / 16 (75% – Medium)

Review relevance: Protocol-level work for PIM compatibility; simulation-only evaluation targeting LLM GEMV; no TN/quantum relevance

CoPIM: A Collaborative Scheduling Framework for Commodity Processing-in-memory Systems

[40] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 collaborative CPU-PIM scheduling using computational graphs
QR2 Reproducibility	2	p.4-5 lightweight performance model with metaheuristic offload ratio search
QR3 Baseline	2	p.6 compares vs CPU-only and DPU-only on 4 benchmarks
QR4 Data movement	2	p.7 achieves 82.1% reduction in CPU-PIM data transfers
QR5 Hardware realism	2	p.6 real hardware: dual Xeon Silver 4216 + 20 UPMEM DIMMs (2530 DPUs)
QR6 Open artifacts	0	p.7 no artifact release; measurements via Intel PCM
QR7 Limitations	1	p.8 acknowledges scheduling overhead; argues offline cost is negligible
QR8 Contribution	2	p.6-7 evaluates FFT, Gauss elimination, and synthetic DAG workloads

Score: 13 / 16 (81% – Medium)

Review relevance: CPU-PIM collaborative scheduler; 4× avg speedup on real UPMEM via intelligent partitioning; integer-only due to lack of FP hardware

Pac-PIM: A Parallel Communication Framework for Commodity Processing-in-memory Systems

[23] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 combines rank-level and process-level parallelism for host-PIM communication
QR2 Reproducibility	2	p.4-9 IRA (intelligent rank allocation) and adaptive MP mechanism
QR3 Baseline	2	p.13-19 vs NCF/MP/IRA; evaluates integration w/ SimplePIM, UpPipe, PIMLex
QR4 Data movement	2	p.1 communication accounts for 72% of execution time; targets Host↔DIMMs transfers
QR5 Hardware realism	2	p.12 real UPMEM: 2× Xeon Silver 4110, 8 ranks, 128 GB DDR4, SDK 2023.1.0
QR6 Open artifacts	0	no artifacts released (references PID-Comm GitHub, but not Pac-PIM)
QR7 Limitations	2	p.3-4 discusses RLP/PLP limitations; diminishing returns with few ranks
QR8 Contribution	1	p.1,23 software-centric host-PIM communication framework; systems proxy but no TN/quantum relevance

Score: 13 / 16 (81% – Medium)

Review relevance: Communication framework for UPMEM; 1.53× avg speedup via parallel host-PIM transfers; no TN/quantum relevance

Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology

[41] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 develop uPIMulator cycle-level simulator because hardware counters are insufficient
-----------------	---	---

QR2 Reproducibility	2	p.4-5 ISA-compatible simulation using UPMEM LLVM; models 14-stage pipeline
QR3 Baseline	1	p.5-8 validates simulator against real UPMEM via end-to-end runtime (not counters)
QR4 Data movement	2	p.7-8 analyzes Host→DPU and DPU→Host communication across 16 PRIM benchmarks
QR5 Hardware realism	1	p.4 simulates UPMEM: 350 MHz DPU, DDR4-2400, 64 MB MRAM per DPU
QR6 Open artifacts	1	p.2 code promised at github.com/VIA-Research/uPIMulator
QR7 Limitations	2	p.2,7,9-10 identifies 11-cycle pipeline bubbles and RF bank conflicts; (p.9-10) shows MRAM bandwidth limits
QR8 Contribution	1	p.1-17 detailed microarchitectural characterization; proposes SIMT and cache-centric PIM
Score: 12 / 16 (75% – Medium)		
Review relevance: Cycle-accurate UPMEM simulator; validates against real hardware runtime; identifies pipeline and RF hazards; no TN/quantum content		

PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices

[22] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 inter-PE collective communication framework for commodity PIM-enabled DIMMs
QR2 Reproducibility	2	p.4-7 PE-Assisted Reordering, In-Register Modulation, Cross-Domain Modulation; virtual hypercube
QR3 Baseline	2	p.8-10 vs SimplePIM baseline and existing implementations across 8 primitives
QR4 Data movement	2	p.1, p.4 eliminates host memory access and domain transfers for inter-PE communication
QR5 Hardware realism	2	p.8 real UPMEM: Xeon Gold 5215, 4 channels × 4 ranks = 1024 PEs
QR6 Open artifacts	2	p.1 https://github.com/AIS-SNU/PID-Comm
QR7 Limitations	2	p.11-12 UPMEM-specific; host still mediates; no direct inter-PE paths in commodity DRAM
QR8 Contribution	2	p.4-7,9 novel data modulation techniques; primitives: 5.19× max (2.83× geomean); applications: 3.99× max (1.99× geomean)
Score: 16 / 16 (100% – High)		
Review relevance: Collective communication library for UPMEM; eliminates host-mediated bottlenecks; strong systems proxy for PIM data movement		

PRISM: Processing-In-Memory Sparse MTTKRP for Tensor Decomposition Acceleration

[27] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 optimize spMTTKRP for UPMEM PIM using fixed-point arithmetic
QR2 Reproducibility	1	p.4-6 novel sparse tensor format and dimension-based partitioning
QR3 Baseline	2	p.7-9 vs CPU (ALTO) and GPU (A100) SOTA; heterogeneous CPU+PIM
QR4 Data movement	1	p.4-6 partitions tensor across DPUs; replicates factor matrices
QR5 Hardware realism	2	p.7 UPMEM: Xeon Silver 4210, up to 2176 DPUs; GPU: A100 80GB

QR6 Open artifacts	0	no artifacts released
QR7 Limitations	1	p.10 limited by lack of global memory and inter-DPU communication
QR8 Contribution	1	p.1,10 optimization technique up to 2.37× (PIM) and 2.64× (heterogeneous) over CPU
Score: 10 / 16 (63% – Medium)		
Review relevance: Sparse MTTKRP kernel on UPMEM; fixed-point optimization; systems proxy for PIM tensor workloads		

PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures

[26] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1-2 GNN library with intelligent parallelization for real PIM systems; Python API
QR2 Reproducibility	2	p.7-14 PaF aggregation, lightweight tuner, vertex/edge parallelism; detailed algorithms
QR3 Baseline	2	p.15-22 vs PyTorch CPU/GPU, GraNDe, prior PIM schemes (SP1, SP2)
QR4 Data movement	2	p.9-11 optimizes Host-PIM transfers, PIM core clustering, 2D tiling for partial results
QR5 Hardware realism	2	p.14 2× Xeon (32 threads), 128GB DDR4, 16 PIM DIMMs, 1992 cores @350MHz
QR6 Open artifacts	2	p.2 https://github.com/CMU-SAFARI/PyGim
QR7 Limitations	0	p.24-25 no explicit limitations; discusses future HBM-PIM/AiM but not current constraints
QR8 Contribution	2	p.7-14 first ML library for near-bank PIM with adaptive parallelization and performance modeling
Score: 14 / 16 (88% – High)		
Review relevance: Open-source GNN library for UPMEM; comprehensive PaF parallelization; strong systems artifact		

Scalability Limitations of Processing-in-Memory using Real System Evaluations [42] Tier 3 – Hardware Proxy

QR1 Aim clarity	1	p.1 evaluate scalability of UPMEM PIM for collective communication workloads
QR2 Reproducibility	2	p.4-5 embedding tables and NTT kernels; data partitioning strategies
QR3 Baseline	0	p.9-20 varies DPU counts (1-256); no CPU/GPU baselines
QR4 Data movement	1	p.9-20 measures Host-PIM bandwidth, MRAM-WRAM transfers, inter-DPU costs
QR5 Hardware realism	0	p.6 Fig.2c mentions UPMEM server but no CPU model/specs provided
QR6 Open artifacts	0	no artifacts released
QR7 Limitations	2	p.20-22 detailed analysis of interconnect bottleneck; host-mediated communication limits
QR8 Contribution	2	p.3 implements and characterizes kernels; identifies scalability walls

Score: 8 / 16 (50% – Low)

Review relevance: UPMEM scalability study for collectives; highlights interconnect limits; background reference

SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory

[28] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1 software framework to aid programming real PIM systems
QR2 Reproducibility	2	p.3-5 iterator-based API, communication interface, broadcast/scatter/gather
QR3 Baseline	2	p.8-10 six apps vs hand-optimized: 4.4× LoC reduction, comparable performance
QR4 Data movement	2	p.3-5 abstracts Host-PIM and PIM-PIM transfers; handles alignment constraints
QR5 Hardware realism	2	p.8 UPMEM with 2,432 cores (no host specs provided)
QR6 Open artifacts	2	p.1 https://github.com/CMU-SAFARI/SimplePIM
QR7 Limitations	1	p.10-11 no limitations discussed; only generalizability to other PIM
QR8 Contribution	2	p.3-5 engineering contribution: iterator abstraction for PIM, not architectural novelty

Score: 15 / 16 (94% – High)

Review relevance: Useful framework; good engineering but lacks critical self-evaluation and hardware details

SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures

[25] Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1,3 first comprehensive SpMV analysis on real PIM; SparseP library
QR2 Reproducibility	2	p.8-14 25 kernels, 4 formats (CSR/COO/BCSR/BCOO), 1D/2D partitioning
QR3 Baseline	2	p.1,32-34 vs Intel Xeon Silver 4110 CPU and NVIDIA Tesla V100 GPU
QR4 Data movement	2	p.8-14 parallel transfers, alignment constraints, synchronization primitives
QR5 Hardware realism	2	p.14 2× Xeon Silver 4110, 128GB DDR4, 20 DIMMs, 2560 DPU (2528 used)
QR6 Open artifacts	2	p.1 https://github.com/CMU-SAFARI/SparseP
QR7 Limitations	1	p.37 discusses UPMEM transfer limitations and padding overhead; no self-critique
QR8 Contribution	2	p.1,8-14 systematic exploration of load balancing and data partitioning for SpMV

Score: 15 / 16 (94% – High)

Review relevance: Landmark SpMV work; comprehensive but dated (2022); useful for PIM patterns not TN-specific

TransPimLib: Efficient Transcendental Functions for Processing-in-Memory Systems [21]

Tier 3 – Hardware Proxy

QR1 Aim clarity	2	p.1-2 CORDIC and LUT-based methods for transcendental functions on UPMEM
QR2 Reproducibility	2	p.3-6 library architecture, 8 methods, combined CORDIC+LUT approaches
QR3 Baseline	1	p.9 vs 1-core and 32-core CPU; vs polynomial approximation on PIM (no GPU)
QR4 Data movement	2	p.5 LUTs as data-movement optimization; precompute to avoid PIM compute

QR5 Hardware realism	2	p.6 2× Xeon (32 cores @2.1GHz), 128GB, 20 UPMEM DIMMs, 2545 cores @350MHz
QR6 Open artifacts	2	p.1 https://github.com/CMU-SAFARI/transpimlib
QR7 Limitations	0	no limitations discussed; only method tradeoffs
QR8 Contribution	1	p.1-2 engineering contribution: PIM-specific tuning of classic methods, not novel algorithms
Score: 12 / 16 (75% – Medium)		
Review relevance: Useful transcendental library; solid engineering but limited baselines and no self-critique		

16. Appendix E – Data Extraction Cards

16.1. Purpose of the data extraction appendix

This appendix reports the standardized data extraction records for the 33 studies included in the final corpus. The records provide the detailed evidence base used for the synthesis in Section 8 and the design implications in Section 9.

The extraction records are organized according to the three evidence tiers used throughout the review: Tier 1 covers quantum simulation on Processing-in-Memory, Tier 2 covers tensor-network algorithmic analogs, and Tier 3 covers UPMEM hardware proxy workloads. Each record summarizes the study type, computational domain, aim, method, platform, constraints, evaluation evidence, reported results, limitations, and relevance to the review questions.

16.2. Extraction schema

The extraction schema was designed to capture both algorithmic and architectural evidence. In particular, the schema emphasizes data movement, memory-capacity constraints, hardware realism, and transferability to PIM-aware tensor-network simulation.

Field	Purpose
Study type	Identifies whether the work is a research article, framework, compiler, benchmark, characterization study, or systems paper.
Domain	Records the computational area addressed by the study, such as quantum simulation, tensor-network contraction, sparse tensor algebra, graph processing, or PIM communication.
Aim and RQ link	Summarizes the study objective and indicates which review question it primarily informs.
Method	Captures the algorithmic, compiler, architectural, or systems technique proposed or evaluated.
Platform	Records the hardware or simulation environment used in the evaluation.
Constraints	Extracts explicit memory, communication, arithmetic, and data-movement constraints.
Evaluation	Summarizes baselines, benchmarks, workloads, and comparison methodology.
Results	Records the main reported performance, energy, scalability, or accuracy results.
Reported limitations	Captures limitations stated by the authors or evident from the evaluation design.
Review relevance	Explains how the study contributes to the review questions and to the design of PIM-aware tensor-network simulation.

Table 10: Data extraction schema used for included studies.

16.3. Extraction cards

Tier 1: Quantum simulation on PIM

PIMutation: Exploring the Potential of Real PIM Architecture for Quantum Circuit Simulation [9]	
Study type	Research Article – Framework Proposal
Domain	State-Vector Quantum Circuit Simulation on PIM
Aim and RQ link	Targets the memory wall in state-vector simulation [p.1]
Method	Gate Merging, Row Swapping, and Vector Partitioning to reduce costly operations and exploit integer-oriented execution [p.4-5]
Platform	UPMEM 2,560 DPUs (20 DIMMs), 64KB WRAM, Xeon Silver 4215 host [p.6]
Constraints	DPUs are integer-oriented and lack direct inter-DPU communication [p.2-3]
Evaluation	BB84, BV, EDC, HS, QRNG, XOR; 2–32 qubits; compared against QuEST v3.7.0 [p.6-7]
Results	Reports $2.99\times$ and $16.51\times$ speedup, and 25.23% and 75.29% energy reduction versus QuEST CPU for 16- and 32-qubit cases respectively [p.1,7-8]
Reported limitations	Vector Partitioning works only for separable quantum states; host reconstruction dominates runtime; evaluated scalability remains in the low-30-qubit range due to memory capacity [p.5, p.7]
Review relevance	First implementation of quantum circuit simulation on commercial Processing-in-Memory. Provides direct Tier 1 evidence for RQ3 and shows that integer-oriented arithmetic, memory capacity, and lack of inter-DPU communication constrain scaling for entangled circuits. Motivates investigation of tensor-network and partitioning-based alternatives [RQ1, RQ3] [p.8]

Tier 2: Tensor-network algorithmic analogs

AutoHOOT: Automatic High-Order Optimization for Tensors [13]	
Study type	Research Article – Framework/Compiler Proposal
Domain	Tensor Network and Tensor Decomposition Optimization
Aim and RQ link	To automatically generate optimized high-order tensor computations [p.1]
Method	Reverse-mode automatic differentiation with Einsum fusion, distribution, and constrained contraction path construction [p.4-8]
Platform	KNL CPU (68 cores), NVIDIA Tesla K80 / Titan X GPUs [p.9-10]
Constraints	Hardware-oblivious path selection; assumes execution time proportional to floating-point operation counts [p.8]
Evaluation	CP/Tucker decomposition and DMRG evaluated against JAX, TensorFlow, TensorLy, and Quimb [p.9-11]
Results	Speedup over, or performance comparable to, state-of-the-art solutions [Table I, Fig. 5-7]
Reported limitations	Optimal contraction path search is NP-hard, so authors use greedy heuristics [p.6]; cost model assumes time is proportional to operation count and explicitly does not consider communication costs [p.8]; work targets dense tensors
Review relevance	Provides dimension-tree and constrained contraction-path formulations relevant to host-side optimization before possible dispatch to UPMEM-style execution [RQ2]

Closing the “Quantum Supremacy” Gap: Achieving Real-Time Simulation of a Random Quantum Circuit Using a New Sunway Supercomputer

[10]

Study type	Research Article – Gordon Bell Prize
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To develop a high-performance tensor-based simulator for random quantum circuits [p.1]
Method	Slicing scheme with compute-density heuristic; architecture-aware parallelization; fused permutation-multiplication; mixed precision [p.5-9]
Platform	New Sunway supercomputer, up to 41.9 million cores [p.1, p.10]
Constraints	Hardware-specific to Sunway memory hierarchy; assumes sufficient on-chip memory for slices [p.6-7]
Evaluation	Time-to-solution compared with Google Sycamore, Summit, and qFlex [p.10-11]
Results	304 seconds for Sycamore circuit; 1.2 Eflops single-precision, 4.4 Eflops mixed-precision [p.11]
Reported limitations	Low efficiency for tensors from CoTenGra path finder; limited portability beyond Sunway [p.11]
Review relevance	Provides architecture-aware contraction and slicing heuristics relevant to host-side path selection in a future PIM-aware planner [RQ1, RQ2]

DISTAL: The Distributed Tensor Algebra Compiler

[17]

Study type	Research Article – Compiler
Domain	Dense Tensor Algebra on Distributed Systems
Aim and RQ link	To compile dense tensor algebra to distributed heterogeneous machines using independent data and computation mappings [p.1]
Method	Separate format language for data distribution and scheduling language for computation; maps to abstract processor grid [p.3-4]
Platform	Lassen supercomputer: IBM Power9 (40 cores) + NVIDIA Volta V100 GPUs, up to 256 nodes [p.10]
Constraints	Designed for distributed supercomputers; dense tensors only; assumes Legion runtime [p.3, p.14]
Evaluation	Distributed matrix multiplication and higher-order operations versus CTF [p.10-13]
Results	1.8× to 3.7× speedup over CTF on higher-order operations; competitive with COSMA on MatMul [p.13-14]
Reported limitations	No support for sparse tensors; no evaluation on non-HPC architectures [p.14]
Review relevance	Provides a data-computation separation model relevant to Host-PIM partitioning and scheduling, although the evaluated system is distributed HPC rather than PIM [RQ2]

Efficient Quantum Circuit Simulation by Tensor Network Methods on Modern GPUs

[12]

Study type	Research Article – Optimization
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To accelerate tensor network contraction on GPUs by converting Einsum to GEMM and applying extended and mixed precision [p.1]

Method	Contraction path finding with dynamic slicing; sparse-state contraction; Einsum-to-GEMM transformation; extended precision for accuracy and mixed precision for Tensor Cores [p.8-14]
Platform	NVIDIA DGX-A100, eight A100-SXM-80GB GPUs [p.17]
Constraints	Requires NVIDIA Tensor Cores; for peak performance matrix dimensions must be multiples of 8 [p.7]; method converts contractions to dense GEMM operations [p.10-13]
Evaluation	53-qubit 18-cycle and 20-cycle Sycamore circuits compared with prior CPU and GPU implementations [p.19]
Results	3.96× faster verification for 18-cycle; 12.5× versus CPU baseline and 4.48–6.78× versus GPU baselines for 20-cycle; sustains more than 21 TFLOPS per A100 [p.1]
Reported limitations	Index arrangement not optimized; low-precision formats require tensor rescaling; no portability beyond NVIDIA GPUs [p.19-20]
Review relevance	Provides precision-scaling and Einsum-to-GEMM techniques relevant to evaluating whether dense tensor kernels can be adapted to UPMEM-style constraints [RQ1, RQ2]

Lifetime-based Optimization for Simulating Quantum Circuits on a New Sunway Supercomputer [11]

Study type	Research Article – Optimization
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To reduce slicing overhead in tensor network contraction using tensor-edge lifetime analysis on Sunway [p.1]
Method	Lifetime analysis to select minimal slicing set; in-place slicing strategy; secondary slicing for CPE thread-level fusion; adaptive path refiner for Sunway [p.2-3, p.6-9]
Platform	Sunway new generation, SW26010Pro, 64 CPEs per core group with 256KB LDM [p.4]
Constraints	Requires Sunway architecture; assumes manual DMA management; optimized for 53-qubit Sycamore RQC [p.4, p.8]
Evaluation	Sycamore RQC versus Cotengra, Alibaba, and Liu 2021 [10] [p.10]
Results	96.1s for 1M correlated samples; 308.6 Pflops single-precision sustained; more than 5× over Liu 2021 [10] (60.4 Pflops) [abstract]
Reported limitations	Permutation operations via LDM access create gap to theoretical peak [p.11]
Review relevance	Provides lifetime-based slicing heuristics relevant to host-side memory planning for future PIM-aware offloading [RQ1, RQ2]

Minimum Cost Loop Nests for Contraction of a Sparse Tensor with a Tensor Network [15]

Study type	Research Article – Framework
Domain	Sparse Tensor Decomposition and Completion
Aim and RQ link	To automatically identify minimum-cost loop nests for SpTTN kernels using buffer-size and cache-miss models [p.1]
Method	Dynamic programming enumeration of loop nests; cost models for cache misses; runtime selection without user guidance; distributed-memory parallelization [p.6-8]
Platform	Stampede2: Intel Xeon Phi 7250 (KNL) 68 cores, 96GB DDR4 + 16GB MCDRAM; also Intel Xeon Silver 4314 Ice Lake [p.9-10]

Constraints	Applies only to SpTTN, where one sparse tensor is contracted with a dense network; assumes static sparsity pattern [p.1]
Evaluation	Real-world and synthetic tensors versus TACO, SPLATT, CTF, SparseLNR [p.10]
Results	Outperforms generalized libraries; matches specialized hand-tuned codes [abstract]
Reported limitations	Only CPU evaluation; excludes partially fused nests from search [p.11]
Review relevance	Dynamic-programming-based cost modeling provides a useful analog for automated memory-aware loop selection in future PIM-aware tensor execution [RQ1, RQ2]

Surpassing Sycamore: Achieving Energetic Superiority Through System-Level Circuit Simulation

[5]

Study type	Research Article – Framework
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To surpass Sycamore time-to-solution and energy via system-level optimization [p.1]
Method	Three-level parallelization; customized low-precision inter-node communication; complex-half einsum expansion [p.5-8]
Platform	NVIDIA A100 80GB cluster with NVLink and InfiniBand, up to 2304 GPUs [p.9]
Constraints	Requires GPU cluster with high-speed interconnect; optimized for Sycamore RQC [p.12]
Evaluation	Sycamore RQC compared with Arute 2019, Liu 2021, and Pan 2022 [p.1-2]
Results	17.18s, 0.29 kWh at XEB 0.002; 14.22s, 2.39 kWh [p.1]
Reported limitations	GPU-only implementation; no PIM or CPU evaluation [p.12]
Review relevance	Low-precision communication strategy is relevant to future evaluation of PIM host-device bandwidth and numerical-representation tradeoffs [RQ2, RQ4]

SW_Qsim: A Minimize-Memory Quantum Simulator with High-Performance on a New Sunway Supercomputer

[14]

Study type	Research Article – Framework
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To develop a minimize-memory quantum simulator with two-level parallelism for New Sunway [p.1]
Method	Minimize-memory contraction path for rectangular grids; process-level and thread-level parallelism; tensor operation optimization; fault-tolerance mechanism [p.4-8]
Platform	New Sunway Supercomputer: SW26016pro many-core processor, interconnect network chips [p.3]
Constraints	PEPS-based simulation; memory-limited capacity; Sunway-specific architecture optimizations [p.1-2]
Evaluation	Compared with Guochu et al. previous state of the art [p.11]
Results	Simulates RQCs up to 400 qubits; near-linear strong and weak scaling to 28.75 million cores [p.1]
Reported limitations	No public artifact; tightly coupled to Sunway PEPS topology; memory constraints limit generality [p.1-2]
Review relevance	Two-level parallelism model is relevant to PIM host-DPU coordination; minimize-memory approach is relevant to limited DPU memory [RQ1, RQ2]

Tensor Network Quantum Virtual Machine for Simulating Quantum Circuits at Exascale [32]

Study type	Research Article – Framework
Domain	Tensor Network Quantum Circuit Simulation
Aim and RQ link	To modernize TNQVM with exact and approximate modes for exascale platforms [p.1]
Method	ExaTN backend; MPS, TTN, and full contraction; Kraus operators for noise; code examples [p.14-17]
Platform	Summit up to 384 V100 GPUs, DGX-A100 8×80GB, DGX-1, laptops to exascale [p.12]
Constraints	Accuracy-memory tradeoff for approximations; general HPC architecture [p.7]
Evaluation	No quantitative comparison to baselines
Results	Demonstrates Sycamore circuits and distributed execution; no speedup versus competitors reported
Reported limitations	Approximate methods; no PIM-specific data movement; accuracy-memory tradeoff limits exact simulation [p.7]
Review relevance	General HPC tensor-network simulation framework. Useful as background for tensor-network execution modes and accuracy-memory tradeoffs, but provides limited direct evidence for PIM-aware execution [Tier 2]

The Tensor Algebra Compiler

[16]

Study type	Research Article – Compiler
Domain	Tensor Algebra Compilation
Aim and RQ link	To automatically generate kernels for any tensor algebra on dense and sparse tensors [p.1]
Method	Tensor formats, iteration graphs, merge lattices, code generation algorithm [p.2-15]
Platform	2×12-core Xeon E5-2695 v2, 128GB RAM, 30MB L3 cache per socket [p.17]
Constraints	Shared-memory only; dense/sparse formats only; no GPU support [p.26]
Evaluation	Compared with Eigen, uBLAS, Gmm++, OSKI, and Intel MKL [p.16]
Results	Competitive with MKL [p.18]; 10× faster than Tensor Toolbox [p.21]; within 1.53× of SPLATT [p.21]
Reported limitations	No GPU/TPU, no tiling, no autotuning, no JIT, limited formats, no transposes [p.26]
Review relevance	Foundational tensor-algebra compiler cited by AutoHOOT [13] and DISTAL [17]. Provides transferable ideas for tensor code generation, but does not address PIM-specific memory movement or UPMEM constraints [Tier 2]

Tier 3: UPMEM hardware proxy workloads

A Full-System Perspective on UPMEM Performance

[33]

Study type	Research Article – System Evaluation
Domain	PIM Performance Analysis
Aim and RQ link	To examine UPMEM PIM performance with a full-system view including data transfer [p.1]

Method	Extension of PrIM benchmark suite with data transfer and reconfiguration measurements [p.4-6]
Platform	2×Xeon Silver 4215, 128GB DRAM, 160GB UPMEM, 2304 DPUs@350MHz, 16 tasklets [p.5]
Constraints	Explicit CPU↔PIM transfer; 4K instruction limit; memory interleaving; offloading model differs from idealized in-memory execution [p.1-2]
Evaluation	Compared with dual-socket Xeon Gold 6152 (44c/88t, 384GB) and original PrIM [p.6]
Results	9/11 benchmarks favor CPU; only GEMV, HST, TS benefit; transfer cuts speedup 10×; 44ms/rank allocation overhead
Reported limitations	Shows that UPMEM performance is affected by host-device transfer, memory interleaving, instruction limits, reconfiguration overhead, and resource-allocation costs [p.1-2,6]
Review relevance	Tier 3 hardware proxy evidence. Provides an important full-system baseline showing that data transfer and system-level overhead can dominate UPMEM performance

A General Offloading Approach for Near-DRAM Processing-In-Memory Architectures [34]

Study type	Research Article – Algorithm
Domain	PIM Offloading
Aim and RQ link	To present IOTPIM for instruction-level offloading on near-DRAM PIM [p.1]
Method	Hierarchy-aware policy + linear regression prediction model [p.5-6]
Platform	Ramulator-PIM, 16 cores@2.5GHz, PIM@1.2GHz, 1KB buffer/core [p.7,9]
Constraints	Explicit CPU-PIM transfer; cache asymmetry; 1KB PIM buffer versus 16MB LLC [p.7-8]
Evaluation	Compared with LLCMIRO, GraphPIM, TDO-CIM, PRIMO, CAIRO [p.8-9]
Results	1.28–1.51× speedup; 91.89% accuracy; 1.22–1.74× energy improvement [p.1,8-9]
Reported limitations	Simulation-only at higher clock rate than real UPMEM; 11.62× training overhead [p.6]; no hardware validation
Review relevance	Tier 3 PIM proxy evidence. Provides useful offloading-policy ideas, but transferability to commodity UPMEM is limited because the evaluation is simulation-only and assumes a faster PIM clock than real UPMEM

Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices [24]

Study type	Research Article – System
Domain	LLM Acceleration on PIM
Aim and RQ link	To accelerate LLMs using a tiled GEMM library on real UPMEM PIM [p.1]
Method	PGEMMLib with architecture-aware tiling + Tile-Selector analytical model [p.4-5]
Platform	20×UPMEM modules, 160GB total, 450MHz DPUs, 2×Xeon Gold 5222 [p.9]
Constraints	No native FP operations; 64MB MRAM per DPU; explicit host-PIM transfer [p.9-10]
Evaluation	Compared with TVM baseline and CPU; evaluated on BERT, GPT2, LLaMA-7b [p.1,10]
Results	45.75× versus TVM on LLaMA-7b; 10.09× average over baseline [p.1,10]
Reported limitations	Requires int8 quantization; MRAM limit forces CPU fallback for large GEMMs [p.9-10]
Review relevance	Tier 3 hardware proxy evidence. Provides real-UPMEM evidence for tiled GEMM and target-aware optimization, while highlighting limitations relevant to quantum tensor-

network workloads: lack of native floating point, MRAM capacity limits, and possible CPU fallback for large tensors

AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory

[35]

Study type	Research Article – Architecture
Domain	PIM Bank-Level Parallelism
Aim and RQ link	To exploit bank-level parallelism via asynchronous execution in near-bank PIM [p.1]
Method	Per-bank long-data command computing entire DRAM rows; hides activation latency [p.2-3]
Platform	DRAMsim3 simulation; 4×HBM2 stacks, 16-lane units, 256B RF/bank, 1GHz [p.9]
Constraints	Activation/precharge overhead (43–70% time); requires shared accumulator per bank group [p.1,7]
Evaluation	Compared with PIM-HBM and Newton baselines [p.1]
Results	33.5% speedup on HBM2 versus PIM-HBM; 59.5% on GDDR6 versus Newton [p.1]
Reported limitations	Requires hardware modification through shared accumulator; 11.7% area overhead; prevents tFAW [p.7-8,12]; simulation-only
Review relevance	Tier 3 architectural proxy evidence. Useful for understanding bank-level parallelism, but transferability to commodity UPMEM is limited because the proposed mechanism requires architectural support such as shared accumulation that isolated UPMEM DPUs do not provide

ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System

[20]

Study type	Research Article – System
Domain	Graph Processing on PIM
Aim and RQ link	To analyze graph algorithms on real UPMEM PIM and characterize bottlenecks [p.1]
Method	SpMV and SpMSpV with 1D/2D partitioning; adaptive format selection [p.4-6]
Platform	20×UPMEM DIMMs (160GB, 2560 DPUs), 2×Xeon Silver 4110 @2.1GHz [p.7]
Constraints	No inter-DPU communication; software-emulated FP; CPU-mediated transfers [p.8-9]
Evaluation	Compared with CPU and GPU; Table IV p.9-10
Results	Versus CPU: kernel 10.2×/48.8×/3.6×; versus GPU: UPMEM 116× slower for BFS [p.2,9]
Reported limitations	GPU outperforms in time and energy; UPMEM uses substantially more energy than GPU for some workloads; no hardware FP [p.8-9]
Review relevance	Tier 3 hardware proxy evidence. Provides real-UPMEM evidence that floating-point-heavy and communication-sensitive workloads can perform poorly relative to GPUs, highlighting risks for complex-valued tensor-network arithmetic

An Efficient PIM-Based Graph Engine on a Single Machine

[29]

Study type	Research Article – System
Domain	Out-of-Core Graph Processing

Aim and RQ link	To process graphs exceeding main memory using PIM with asynchronous I/O [p.1]
Method	Asynchronous I/O + column-wise partitioning; five-layer architecture [p.3]
Platform	20×UPMEM PIM DIMMs, 4×DRAM DIMMs; CPU: 16 threads, 16GB MM [p.2-3]
Constraints	No direct inter-DPU communication; limited MRAM (10MB/DPU used) [p.2-3]
Evaluation	Compared with naive-RGPIM, TurboGraph, GridGraph, PowerGraph-64node [p.4]
Results	Up to 42× versus naive-RGPIM; 4.5× versus TurboGraph; 1.4× versus PowerGraph-64node [p.4]
Reported limitations	Baseline configuration and workload differences limit direct comparability; no GPU or energy data; CPU model undisclosed
Review relevance	Tier 3 hardware proxy evidence. Demonstrates potential for out-of-core graph processing on PIM, but baseline configuration and workload differences limit direct transferability to tensor-network simulation

Analysis of Data Transfer Bottlenecks in Commercial PIM Systems: A Study With UPMEM-PIM

[18]

Study type	Research Article – Characterization
Domain	PIM Data Transfer
Aim and RQ link	To characterize DRAM↔PIM transfer overheads on commercial UPMEM [p.1]
Method	PrIM microbenchmarks + custom AVX512 memcpy; BIOS mapping analysis [p.3]
Platform	Intel Xeon Gold 5222, 3×DDR4-3200, 3×DDR4-2400 UPMEM [p.3]
Constraints	Separate DRAM/PIM address spaces; no unified memory; software-managed transfers [p.2]
Evaluation	Compared with theoretical peak and conventional memory mapping [p.3-4]
Results	DRAM read 11.6% peak, PIM write 15.5% peak, DRAM bandwidth 30% with PIM BIOS [p.3-4]
Reported limitations	+70W power, 100% CPU utilization during transfers; software scheduling causes imbalance [p.3-4]
Review relevance	Tier 3 hardware characterization evidence. Provides real-system evidence that DRAM↔PIM transfers can dominate execution and substantially reduce effective PIM speedup

Architectural Modeling and Benchmarking for Digital DRAM PIM

[36]

Study type	Research Article – Tools
Domain	PIM Benchmarking
Aim and RQ link	To provide a standardized benchmark suite and modeling framework for digital DRAM PIM [p.1]
Method	PIMeval C++ simulator + PIMbench (17 benchmarks); models bit-serial, Fulcrum, bank-level PIM [p.1,6]
Platform	Simulation only: 32GB DDR4, 32 ranks, 128 banks/rank, 8192-bit row buffers [p.7]; no physical PIM
Constraints	Models subarrays as monolithic; digital bit-serial only; no analog support [p.4]
Evaluation	Compared with CPU (AMD EPYC 9124) and GPU (NVIDIA A100) baselines; factors out PCIe/CXL transfer [p.7-8]

Results	Simulated speedups over CPU, but “for many benchmarks, none of the PIM architectures consistently outperform an A100 GPU” [p.10]; bit-serial best among PIM variants
Reported limitations	No real PIM hardware; CPU/GPU baselines likely run on real hardware but PIM is simulation-based
Review relevance	Tier 3 benchmarking proxy evidence. Provides useful open-source infrastructure for digital DRAM-PIM evaluation, while showing that simulated PIM architectures do not consistently outperform high-end GPU baselines across all workloads

ATiM: Autotuning Tensor Programs for Processing-in-DRAM

[19]

Study type	Research Article – Compiler
Domain	PIM Systems
Aim and RQ link	To build an autotuning tensor compiler for UPMEM and automate host-kernel optimization [p.1]
Method	Extends TVM with joint search space, PIM-aware TIR lowering, evolutionary search [p.5-6]
Platform	UPMEM DRAM-PIM real hardware; Intel Xeon Gold 5220R host [p.9]
Constraints	UPMEM-specific; no TN/quantum support; limited to TVM operators [p.13]
Evaluation	Compared with PrIM, SimplePIM, CPU; includes H2D/D2H in measurements [p.10]
Results	6.18× over PrIM, 8.21× on GPT-J, 23.3× over CPU for large tensors [p.1,11]
Reported limitations	Open-source; reproducible on UPMEM; artifact available https://github.com/SNU-CODElab/atim
Review relevance	Tier 3 systems proxy evidence. Provides digital-PIM compiler infrastructure for general tensor operators on UPMEM, but does not directly evaluate tensor-network contraction or quantum simulation

Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System

[6]

Study type	Research Article – Characterization
Domain	PIM Systems
Aim and RQ link	To characterize real UPMEM PIM architecture and provide benchmark suite [p.1]
Method	Microbenchmarks and 16 workloads; roofline analysis; performance and energy [p.5-16]
Platform	UPMEM (2,556 and 640 DPUs); Intel Xeon E3 CPUs; Nvidia Titan V GPU [p.30]
Constraints	UPMEM immature (267–350 MHz); limited to integer ops; no FP division [p.31-32]
Evaluation	Compared with CPU and GPU; includes data transfer costs; energy measurements [p.29-31]
Results	UPMEM 2.54× faster than GPU on suitable workloads; 5.23× energy savings versus CPU [p.30-31]
Reported limitations	Fully open-source PrIM benchmarks; reproducible on UPMEM hardware
Review relevance	Tier 3 foundational UPMEM characterization. Provides a systems baseline for PIM data movement, energy, arithmetic, and workload-suitability constraints

CINM (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms

[38]

Study type	Research Article – Compiler
Domain	PIM Compilation
Aim and RQ link	To build unified MLIR compilation flow for heterogeneous CIM/CNM systems [p.1]
Method	MLIR dialects with progressive lowering; device-aware abstractions; cost models [p.4-10]
Platform	UPMEM real hardware (2048 DPUs); Xeon E5-2630 v2 host; memristor simulator [p.10]
Constraints	No heterogeneous multi-device evaluation; limited to specific PIM variants [p.10]
Evaluation	Compared with PrIM and OCC; includes data transfer costs [p.10-11]
Results	Supports tensor contractions; 47% speedup from optimizations [p.11-12]
Reported limitations	Open-source on GitHub; reproducible with UPMEM hardware
Review relevance	Tier 3 systems proxy evidence. Provides general-purpose PIM compiler infrastructure and includes tensor operations, but is not specific to tensor-network quantum simulation

ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration

[39]

Study type	Research Article – Architecture
Domain	PIM Protocols
Aim and RQ link	To design compatible PIM command protocol and scheduling for processor-PIM co-execution [p.1]
Method	PIM-ACT command; MC-based request generator; ST-BLC/AT-BLC scheduling [p.4-8]
Platform	Simulated LPDDR5-PIM and LPDDR-AiM; no real hardware [p.9]
Constraints	Simulation only; no hardware validation; limited to GEMV workloads [p.9-10]
Evaluation	Compared with FRFCFS, Batch, TCM schedulers; includes mode-switch overhead [p.9-10]
Results	10.75× GEMV speedup for GPT-3; meets token latency QoS [p.1,10]
Reported limitations	No code released; uses external PIMSimulator
Review relevance	Tier 3 architectural proxy evidence. Provides PIM protocol and scheduling ideas relevant to host-PIM coordination, but the evaluation is simulation-based and does not address tensor-network or quantum workloads

CoPIM: A Collaborative Scheduling Framework for Commodity Processing-in-memory Systems

[40]

Study type	Research Article – Scheduling Framework
Domain	Heterogeneous CPU-PIM Systems
Aim and RQ link	To automate workload partitioning between host CPU and UPMEM DPUs [p.1]
Method	Graph-based modeling; performance estimation; metaheuristic parameter search [p.4-5]
Platform	UPMEM (2530 DPUs @ 350 MHz); 2× Xeon Silver 4216 host server [p.6]
Constraints	No FP support; integer-oriented execution; no public code; scheduling overhead [p.6-8]
Evaluation	CPU-only baseline and DPU-only baseline; includes transfer costs [p.6]
Results	4.0× average speedup (7.1× peak); 56.6% energy reduction [p.1]
Reported limitations	Closed source; reproducible only with UPMEM hardware

Review relevance	Tier 3 systems proxy evidence. Demonstrates general-purpose CPU-PIM scheduling and data-movement tradeoffs, but does not directly address tensor-network or quantum simulation workloads
-------------------------	--

Pac-PIM: A Parallel Communication Framework for Commodity Processing-in-memory Systems [23]

Study type	Research Article – Systems
Domain	PIM Communication
Aim and RQ link	To reduce host-PIM communication overhead via layered parallelism [p.1]
Method	Intelligent rank allocation; adaptive multi-process; topology-aware scheduling [p.4-9]
Platform	UPMEM real hardware (8 ranks); dual Xeon Silver 4110 host [p.12]
Constraints	Limited by rank count; process overhead; UPMEM-specific optimizations [p.3-4]
Evaluation	Compared with NCF/MP/IRA; evaluates integration with SimplePIM, UpPipe, and PIMLex [p.13-19]
Results	1.53× average speedup (1.71× geomean); scales to 8 ranks [p.1]
Reported limitations	No code released; evaluates integration with PID-Comm/SimplePIM
Review relevance	Tier 3 communication proxy evidence. Addresses host-PIM communication overhead and is relevant to sliced tensor reductions, but does not directly evaluate tensor-network or quantum algorithms

Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology [41]

Study type	Research Article – Simulator
Domain	PIM Characterization
Aim and RQ link	To demystify UPMEM via cycle-level simulation where hardware counters fail [p.1]
Method	uPIMulator: ISA-compatible simulator with pipeline, RF hazard, and DRAM modeling [p.4-5]
Platform	Validated on real UPMEM; simulates 350 MHz DPU with 24 threads [p.5]
Constraints	No power/area modeling; UPMEM-specific; simulator validation limited to runtime [p.13]
Evaluation	Validates against real UPMEM end-to-end execution rather than counters [p.5]
Results	11-cycle pipeline bubbles cause 30–50% idle cycles; RF conflicts limit ILP [p.7]
Reported limitations	Promised open-source at github.com/VIA-Research/uPIMulator [p.2 fn.1]
Review relevance	Tier 3 architecture-characterization proxy evidence. Provides simulation support for UPMEM research, but does not address tensor-network or quantum algorithms directly

PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices [22]

Study type	Research Article – Systems
Domain	PIM Communication
Aim and RQ link	To accelerate inter-PE collective communication on commodity PIM DIMMs [p.1]

Method	Virtual hypercube with PE-assisted reordering and cross-domain modulation [p.4-7]
Platform	UPMEM real hardware (1024 PEs); Xeon Gold 5215 host [p.8]
Constraints	Requires host mediation; UPMEM-specific; limited by DRAM architecture [p.11-12]
Evaluation	Compared with SimplePIM and naive host-mediated baselines [p.8-10]
Results	Primitives: $5.19\times$ max, $2.83\times$ geomean; Applications: $3.99\times$ max, $1.99\times$ geomean [p.9]
Reported limitations	Open-source at github.com/AIS-SNU/PID-Comm ; artifact evaluated
Review relevance	Tier 3 communication proxy evidence. Provides UPMEM collective-communication infrastructure relevant to host-mediated reductions and data movement in future PIM-aware tensor execution

PRISM: Processing-In-Memory Sparse MTTKRP for Tensor Decomposition Acceleration [27]

Study type	Research Article – Systems
Domain	PIM Application
Aim and RQ link	To accelerate sparse MTTKRP on UPMEM for tensor decomposition [p.1]
Method	Fixed-point kernels with dimension partitioning [p.4-6]
Platform	UPMEM (2176 DPUs), Xeon Silver 4210, Nvidia A100 [p.7]
Constraints	No native FP; no inter-DPU communication; UPMEM-specific [p.10]
Evaluation	Compared with CPU (ALTO) and GPU (A100) [p.7-9]
Results	Up to $2.37\times$ PIM-only, $2.64\times$ heterogeneous versus CPU [p.1]
Reported limitations	No code released
Review relevance	Tier 3 tensor-kernel proxy evidence. Demonstrates PIM feasibility for memory-bound sparse tensor operations, but does not directly evaluate tensor-network contraction or quantum simulation

PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures [26]

Study type	Research Article – Systems
Domain	GNN Acceleration
Aim and RQ link	To build an efficient GNN library for real near-bank PIM with autotuning [p.1-2]
Method	PaF aggregation with vertex/edge parallelism and performance models [p.7-14]
Platform	UPMEM (1992 cores), 2× Xeon Gold, 128GB DDR4 [p.14]
Constraints	Limited discussion of UPMEM constraints such as FP support, capacity, and faulty cores [p.24-25]
Evaluation	Compared with PyTorch CPU/GPU, GraNDe, SP1/SP2 PIM baselines [p.15-22]
Results	$3.04\times$ versus PyTorch, $4.38\times$ versus prior PIM, $11.6\times$ utilization gain [p.1]
Reported limitations	Open-source at github.com/CMU-SAFARI/PyGim ; artifact available
Review relevance	Tier 3 systems proxy evidence. Provides reusable kernels and tuning infrastructure for memory-bound graph workloads on PIM, with indirect relevance to irregular tensor-network subtasks

Scalability Limitations of Processing-in-Memory using Real System Evaluations

[42]

Study type	Research Article – Characterization
Domain	PIM Scalability
Aim and RQ link	To evaluate UPMEM scalability for workloads requiring collective communication [p.1]
Method	Embedding lookup and NTT with table/column/row partitioning [p.4-5]
Platform	UPMEM system with unspecified host; up to 256 DPUs evaluated [p.6, p.9-20]
Constraints	Host-mediated communication; no direct PIM-PIM interconnect; bandwidth-bound behavior [p.20-22]
Evaluation	Compared across varying DPU counts for weak and strong scaling; no external baselines [p.9-20]
Results	Scalability limited by host bandwidth; interconnect would improve by 2–4× [p.21-22]
Reported limitations	No code released
Review relevance	Tier 3 scalability proxy evidence. Characterizes UPMEM limits for workloads requiring collective communication and helps explain why inter-DPU communication matters for scaled tensor workloads

SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory

[28]

Study type	Research Article – Systems
Domain	Programming Framework
Aim and RQ link	To provide a productive high-level API for UPMEM PIM programming [p.1]
Method	Iterator-based framework with map/reduce primitives and communication abstractions [p.3-5]
Platform	UPMEM (2,432 cores evaluated; host unspecified) [p.8]
Constraints	Framework overhead not quantified; limited to data-parallel patterns; UPMEM-specific optimizations [p.10-11]
Evaluation	Compared with hand-optimized implementations for 6 applications [p.8-10]
Results	4.4× LoC reduction; performance similar to or better than hand-optimized implementations [p.1-2]
Reported limitations	Open-source at github.com/CMU-SAFARI/SimplePIM
Review relevance	Tier 3 programming-framework proxy evidence. Provides useful abstractions for rapid UPMEM prototyping, but framework overhead would need to be evaluated for tensor-network kernels

SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures

[25]

Study type	Research Article – Systems
Domain	SpMV Library
Aim and RQ link	To design efficient SpMV for real near-bank PIM systems [p.1,3]
Method	25 kernels with 1D/2D partitioning and multiple load-balancing schemes [p.8-14]
Platform	UPMEM (2528 DPUs), 2× Xeon Silver 4110, Tesla V100 [p.14,32]

Constraints	UPMEM transfer size constraints; padding overhead; no inter-DPU communication [p.8,37]
Evaluation	Compared with CPU (Xeon) and GPU (V100) across 22 matrices [p.32-34]
Results	Up to 51.7% of peak versus 0.51% CPU / 0.21% GPU; 25× energy efficiency versus CPU [p.34]
Reported limitations	Open-source at github.com/CMU-SAFARI/SparseP
Review relevance	Tier 3 sparse-kernel proxy evidence. Provides a strong real-UPMEM methodology for sparse matrix workloads, but transferability is limited by its SpMV focus

TransPimLib: Efficient Transcendental Functions for Processing-in-Memory Systems [21]

Study type	Research Article – Systems
Domain	Math Library
Aim and RQ link	To provide efficient transcendental functions for UPMEM PIM [p.1-2]
Method	CORDIC, LUT-based, and hybrid methods with accuracy tuning [p.3-6]
Platform	UPMEM (2545 cores), 2× Xeon 32-core [p.6]
Constraints	Limited discussion of framework overhead or accuracy limitations [p.6]
Evaluation	Compared with CPU (1/32 cores) and polynomial approximation [p.9]
Results	Up to 10× faster than polynomial; 2–3 orders of magnitude less memory than CPU [p.9-10]
Reported limitations	Open-source at github.com/CMU-SAFARI/transpimlib
Review relevance	Tier 3 numerical-library proxy evidence. Provides practical mathematical-function support for UPMEM and informs the numerical-representation constraints relevant to future tensor-network implementations

Study type	Count
Research Article – Framework Proposal	1
Research Article – Framework/Compiler Proposal	1
Research Article – Gordon Bell Prize	1
Research Article – Compiler	4
Research Article – Optimization	2
Research Article – Framework	4
Research Article – System Evaluation	1
Research Article – Algorithm	1
Research Article – System	3
Research Article – Architecture	2
Research Article – Characterization	3
Research Article – Tools	1
Research Article – Scheduling Framework	1
Research Article – Systems	7
Research Article – Simulator	1